ResettingStructures

Code walkthrough: what every file and every line does

Paper plugin for Minecraft 26.2 / 26.3 · author nlTarzan

Code excerpts are taken directly from the project's source files at the time this document was generated.

Contents

About this document

Step 1: setup files

- settings.gradle.kts
- build.gradle.kts
- src/main/resources/plugin.yml
- src/main/resources/monuments/monuments.yml

Step 2: startup, structure modules and settings

- A few Java basics first
- ResettingStructures.java: where the plugin starts
- StructureModule.java: what every structure provides
- Monuments.java: everything Ocean Monument
- MonumentConfig.java: reading monuments.yml
- The info package: status.txt and commands.txt

Step 3: the commands

- StructureCommand.java: the /structure root
- MonumentCommand.java: /structure monument ...

Step 4: how a reset happens

- util/Durations.java: times like 1d, 10s, 5t
- util/Weighted.java: weighted random picks
- ResetScheduler.java: the timer
- MonumentArena.java: the area around the monument
- StructureLibrary.java: the .nbt templates
- CurrentMonument.java: what's built right now
- MonumentResetter.java: one check, and placing the new monument

Step 5: the maze generator

- The big picture
- GridDirection.java: the six directions
- Cell.java: one unit
- MonumentGrid.java: the grid and its walls
- MazeSettings.java: the maze settings and the bell curve
- GrowingTreeGenerator.java: carving the maze
- RoomType.java and Room.java: rooms
- RoomFitter.java: giving every unit a room
- SpongeRooms.java: guaranteeing a sponge room
- MonumentLayout.java: putting it all together
- LayoutPrinter.java: the console map

Step 6: from layout to blocks

- BuildStep.java: the three kinds of step
- BuildPlanner.java: from layout to steps
- MonumentBuilder.java: doing it in the world

Step 7: the Elder Guardian reward and the Monument Hoe

ElderGuardianListener.java: handing out the hoe
MonumentHoe.java: creating and recognizing the hoe
MonumentHoeListener.java: the hoe rules
Known, accepted limitations

Step 8: how the tests work

What a test is, and why this plugin has them
Where they live
JUnit basics, using the smallest test
Two kinds of tests
What each test file guards
Running them
What a failure looks like
What the tests can't check
Adding a test yourself

About this document

This document explains the ResettingStructures plugin file by file. It brings together the step-by-step walkthrough from our conversation, updated to the code as it is now: some files changed after they were first explained, so their explanations and line numbers have been brought up to date here.

How to read it

- Every grey box is a real excerpt from the project, copied automatically from the source file when this PDF was made. The number in front of each line is the line number in that file, so you can open the file and find the same line.
- A box marked with `⋮` skips some lines in between.
- Explanations refer to those line numbers, for example "**Line 45**" or "**Lines 103–105**".
- Java ideas are explained the first time they appear. Step 2 starts with a short table of the basics.

The steps

Step	Files	What it covers
1	`settings.gradle.kts`, `build.gradle.kts`, `plugin.yml`, `monuments.yml`	How the plugin is built, what Paper needs to know, the settings
2	`ResettingStructures`, `StructureModule`, `Monuments`, `MonumentConfig`, the `info` package	Startup, structure modules, reading the settings, `status.txt` and `commands.txt`
3	`StructureCommand`, `MonumentCommand`	The `/structure` commands
4	`Durations`, `Weighted`, `ResetScheduler`, `MonumentArena`, `StructureLibrary`, `CurrentMonument`, `MonumentResetter`	How a check and a reset happen
5	the `layout` package	Generating a random maze layout
6	`BuildStep`, `BuildPlanner`, `MonumentBuilder`	Turning a layout into blocks
7	`ElderGuardianListener`, `MonumentHoe`, `MonumentHoeListener`	The reward and the Monument Hoe rules
8	`src/test`	How the automated tests work

Where the files are

```
resetting-structures/
├── build.gradle.kts, settings.gradle.kts      the build
├── src/main/resources/                         plugin.yml, monuments/monuments.yml
├── src/main/java/com/nltarzan/structures/      the plugin's code
│   ├── ResettingStructures.java, StructureModule.java
│   ├── command/        the shared /structure command
│   ├── info/           status.txt and commands.txt
│   ├── util/           Durations, Weighted
│   └── monument/       everything Ocean Monument
│       ├── command/  config/  layout/  build/  listener/
└── src/test/java/com/nltarzan/structures/      the tests
```

Step 1: setup files

These four files don't contain game logic. They say how the plugin is built, what Paper needs to know about it, and what you can configure.

settings.gradle.kts

settings.gradle.kts

```
1  rootProject.name = "resetting-structures"
```

- **Line 1:** the project's name. Gradle uses it to name the jar: `resetting-structures-0.1.0.jar`. That's all this file does for now. If the plugin ever grows into separate modules (one per structure, say), they'd be listed here.

build.gradle.kts

The build recipe. Gradle runs it whenever you type `.\gradlew.bat build` or `.\gradlew.bat runServer`. It's written in Kotlin (`.kts`), which reads much like Java.

Lines 1–6: tools for the build

build.gradle.kts

```
1  import xyz.jpenilla.runpaper.task.RunServer
2
3  plugins {
4      java
5      id("xyz.jpenilla.run-paper") version "3.1.0"
6  }
```

- **Line 1:** makes the type `RunServer` available, for the 26.3 test server further down (line 77). It works like `import` in Java.
- **Line 4:** the `java` plugin. Gradle learns to compile Java, run tests and build a jar. This creates the tasks `build`, `test`, `jar` and `clean`.
- **Line 5:** the **run-paper** plugin (version 3.1.0). It adds `runServer`, which downloads Paper, adds your plugin and starts a test server.

Lines 8–13: project details

build.gradle.kts

```
8  group = "com.nltarzan"
9  version = "0.1.0"
10
11 // The Minecraft version we compile against. The plugin.yml api-version is derived from it,
12 // so the plugin loads on this version and every newer one (26.3 included).
13 val minecraftVersion = "26.2"
```

- **Line 8:** who made it, written as a reversed domain name. It matches the Java packages (`com.nltarzan.structures…`). It only really matters if you ever publish the plugin as a library for other developers.
- **Line 9:** the plugin version. It appears in the jar name, in `plugin.yml` (via line 49) and in `/plugins`. Raise it with each release, for example `0.2.0`.

- **Line 13:** a variable used twice: as the `api-version` in `plugin.yml`, and as the version of the test server. Lines 11–12 explain that `26.2` means "26.2 or newer", so 26.3 works too.

Lines 15–18: where libraries come from

build.gradle.kts

```
15  repositories {
16      mavenCentral()
17      maven("https://repo.papermc.io/repository/maven-public/")
18  }
```

- **Line 16:** Maven Central, the main public store for Java libraries. JUnit (for the tests) comes from here.
- **Line 17:** PaperMC's own store. The Paper API comes from here.

Lines 20–29: libraries

build.gradle.kts

```
20  dependencies {
21      compileOnly("io.papermc.paper:paper-api:26.2.build.129-stable")
22      // The tests read YAML and use text colors, so they need the Paper API too (never packed in the
plugin).
23      testImplementation("io.papermc.paper:paper-api:26.2.build.129-stable")
24
25      testImplementation(platform("org.junit:junit-bom:6.1.3"))
26      testImplementation("org.junit.jupiter:junit-jupiter")
27      testImplementation("org.junit.jupiter:junit-jupiter-params")
28      testRuntimeOnly("org.junit.platform:junit-platform-launcher")
29  }
```

- **Line 21:** the **Paper API**: everything in the code starting with `org.bukkit` or `io.papermc`. `compileOnly` means "needed to compile, but don't pack it into the jar", because the server already has Paper. `26.2.build.129-stable` is the exact build.
- **Lines 22–23:** the tests also get the Paper API, because they read YAML files and use text colors. `testImplementation` means it's only used for the tests; it never ends up in the plugin.
- **Lines 25–28: JUnit 6**, for the tests in `src/test`.
  - **Line 25:** a "BOM" that fixes all JUnit parts to version 6.1.3, so lines 26–28 don't need a version of their own.
  - **Line 26:** the basics: `@Test` and `assertEquals`.
  - **Line 27:** tests that run several times with different values (`@ParameterizedTest`), for example once per newest chance.
  - **Line 28:** the part that actually starts the tests. Gradle 9 requires it to be listed separately.

Lines 31–42: Java version and tests

build.gradle.kts

```
31 java {
32     toolchain.languageVersion = JavaLanguageVersion.of(25)
33 }
34
35 tasks.withType<JavaCompile>().configureEach {
36     options.encoding = "UTF-8"
37     options.release = 25
38 }
39
40 tasks.test {
41     useJUnitPlatform()
42 }
```

- **Line 32:** always compile with Java 25, whatever `java` your PATH points to.
- **Line 36:** read source files as UTF-8, so special characters in comments can't be misread.
- **Line 37:** produce code for Java 25 and only allow Java 25 features. Paper 26.x needs Java 25 itself.
- **Line 41:** tells Gradle the tests are JUnit tests. Without it, `test` finds nothing.

Lines 44–61: what goes into the jar besides the code

build.gradle.kts

```
44 // The .nbt structures are taken straight from the datapack, so the datapack stays the single
45 // source of truth while you keep editing structures in-game.
46 val datapackStructures =
layout.projectDirectory.dir("../Structures/data/structures/structure/monument")
47
48 tasks.processResources {
49     val props = mapOf("version" to project.version, "apiVersion" to minecraftVersion)
50     inputs.properties(props)
51     filesMatching("plugin.yml") {
52         expand(props)
53     }
54     from(datapackStructures.dir("pre-gen")) {
55         into("structures/monument/pre-gen")
56     }
57     from(datapackStructures.dir("maze")) {
58         into("structures/monument/maze")
59         exclude("maze_nodes.nbt") // obsolete, the nodes are generated
60     }
61 }
```

`processResources` copies `src/main/resources` (`plugin.yml`, `monuments/monuments.yml`) into the jar. Here it does a bit more:

- **Line 46:** the path to the monument structures in your datapack, one folder up (`../`) from the project. The `.nbt` files aren't copied into the plugin's source; they're read from the datapack on every build. If you save a structure to the datapack in-game, the next build picks it up.
- **Line 49:** the values to fill into `plugin.yml`: the version (line 9) and the API version (line 13).
- **Line 50:** tells Gradle this step depends on those values, so a version change makes it run again instead of reusing an old result.
- **Lines 51–53:** replaces the placeholders `${version}` and `${apiVersion}` in `plugin.yml` with those values.
- **Lines 54–56:** copies the 44 pre-built quadrant files into the jar under `structures/monument/pre-gen/`.

- **Lines 57–60:** copies the 19 maze templates (rooms, shell, wings, penthouse, `roof_open`) under `structures/monument/maze/`. It skips `maze_nodes.nbt`, which your datapack already called obsolete; the plugin works out the node positions itself.

The folder inside the jar is called `structures/monument/`, which fits the plan for more structures: a mansion would go in `structures/mansion/`.

Lines 63–83: test servers

build.gradle.kts

```
63  // ./gradlew runServer      -> local 26.2 test server in run/26.2
64  // ./gradlew runServer26_3  -> local 26.3 test server in run/26.3
65  // Both test servers get LuckPerms, to hand out the structures_admin permission.
66  tasks.withType<RunServer>().configureEach {
67      downloadPlugins {
68          modrinth("luckperms", "v5.5.71-bukkit")
69      }
70  }
71
72  tasks.runServer {
73      minecraftVersion(minecraftVersion)
74      runDirectory = layout.projectDirectory.dir("run/$minecraftVersion")
75  }
76
77  tasks.register<RunServer>("runServer26_3") {
78      group = "run paper"
79      description = "Run a 26.3 test server with the plugin."
80      minecraftVersion("26.3")
81      runDirectory = layout.projectDirectory.dir("run/26.3")
82      pluginJars(tasks.jar.flatMap { it.archiveFile })
83  }
```

- **Lines 66–70:** *every* test server task (`withType<RunServer>`) downloads LuckPerms v5.5.71 from Modrinth into its plugins folder.
- **Lines 72–75:** the standard `runServer` task.
  - **Line 73:** Paper 26.2.
  - **Line 74:** the server lives in `run/26.2`: world, `server.properties`, `eula.txt` and `plugins/ResettingStructures/`.
- **Lines 77–83:** a self-made second task, `runServer26_3`.
  - **Lines 78–79:** how it appears in Gradle's task list.
  - **Line 80:** Paper 26.3.
  - **Line 81:** its own folder, so the two worlds never mix.
  - **Line 82:** adds your plugin jar. `runServer` does this automatically; a self-made task has to be told. `flatMap` means "whatever file the jar task produces", so the jar is built first.

The files Gradle brought along:

- **The Gradle wrapper** (`gradlew`, `gradlew.bat`, the `gradle/wrapper` folder): it downloads exactly Gradle 9.8.0 on first use, so nobody needs to install Gradle.
- **`.gitignore`:** lists what Git should skip if you ever put the project in a repository: `build/`, `run/` and IDE files.

src/main/resources/plugin.yml

Paper reads this file first to learn what the plugin is. If it's missing or wrong, the plugin doesn't load.

plugin.yml

```
1  name: ResettingStructures
2  version: '${version}'
3  main: com.nltarzan.structures.ResettingStructures
4  api-version: '${apiVersion}'
5  description: Custom-built versions of vanilla structures that reset with a new random layout after
a player has cleared them.
6  authors: [nlTarzan]
7
8  permissions:
9    structures_admin:
10     description: Use /structure and receive the structures' broadcast messages. Grant it with a
permissions plugin such as LuckPerms.
11     default: false
```

- **Line 1:** the plugin's name. It shows in `/plugins` and in log lines (`[ResettingStructures] …`), and it names the plugin's folder: `plugins/ResettingStructures/`.
- **Line 2:** the version, filled in by the build (`0.1.0`).
- **Line 3:** the class Paper starts first: `ResettingStructures` in the package `com.nltarzan.structures`. It must be a class that extends `JavaPlugin`. That's the start of step 2.
- **Line 4:** the minimum Minecraft version (`26.2`). An older server refuses to load the plugin.
- **Line 5:** a description, shown in `/version ResettingStructures`.
- **Line 6:** the author.
- **Lines 8–11:** the permissions the plugin defines.
  - **Line 9:** the one permission, `structures_admin`.
  - **Line 10:** what it's for. LuckPerms shows this text, for example in its web editor.
  - **Line 11:** `default: false` means nobody has it unless it's granted, **not even ops**. The alternative, `default: op`, would hand it to every op automatically.

There's no `commands:` section. That's the old way of declaring commands; the plugin registers `/structure` in code instead (step 2).

src/main/resources/monuments/monuments.yml

This is the **default** Monument config. The first time the plugin starts, it copies this file to `plugins/ResettingStructures/monuments/monuments.yml`, next to the monument's saved data. Each structure keeps its config and its data together in its own folder. After that it only reads that copy and never overwrites it, so after an update you have to add new settings to your server's copy yourself. Lines starting with `#` are comments, ignored by the plugin.

Lines 1–6: header

monuments.yml

```
1  # ==================== ResettingStructures: Monuments ====================
2  # Settings for Ocean Monuments ("monuments" for short).
3  # Every interval the monument's area is loaded and checked for Elder Guardians.
4  # When none are left, a new monument is placed: a random pre-built one, or a generated maze.
5  # Ported from the "Structures" datapack.
6  # ========================================================================
```

This file is only for Ocean Monuments ("monuments" for short). It's checked regularly, and replaced by a pre-built or generated monument once it's cleared.

Lines 8–22: location (where the monument is)

monuments.yml

```
8  location:
9    # Center of the monument. This replaces the datapack's marker entity:
10   #   /summon marker 224.0 40.0 -320.0 {Tags:["ResettingStructure","OceanMonument"]}
11   # It is the corner where the 4 center blocks meet, on the Y level the structures are placed from.
12   # Stand on that corner and run /structure monument setcenter, or use setcenter <x> <y> <z>.
13   # On a new server nothing is checked or built until /structure monument active on is run,
14   # so check this location first.
15   world: world
16   x: 224
17   y: 40
18   z: -320
19   # Elder Guardians and players within this many blocks of the center count as "at the monument"
20   # (datapack: 40), and so does anything inside the monument itself, even its far corners.
21   # The chunks covering both are loaded during a check.
22   check-radius: 40
```

- **Lines 9–12:** the center replaces the datapack's marker entity. It's the corner where the 4 center blocks meet. The rooms start at this height; the empty shell and the pre-built quadrants are placed 1 block lower.
- **Lines 13–14:** a reminder that a new server builds nothing until `/structure monument active on` (step 3). That's the moment to check these values.
- **Line 15:** the world, matching the world's folder name. Your normal overworld is `world`.
- **Lines 16–18:** the center's coordinates. `/structure monument setcenter` rewrites them for you, from where you stand or from coordinates you type.
- **Line 22:** a radius in blocks around the center. Elder Guardians and players within it count as "at the monument", and so does anything inside the monument's 58×22×58 box, even its far corners (lines 19–21). The chunks covering both are kept loaded during a check. The hoe rules use only the exact box.

The plugin checks this section strictly. If it's missing or incomplete, the monument is paused instead of being built somewhere else (step 2). And even with a valid location, nothing happens on a new server until an admin turns the monument on.

Lines 24–49: reset (when and how it resets)

monuments.yml

```
24  reset:
25    # What a reset places:
26    #   pre-built  one of the variants below
27    #   generated  a new random maze of rooms (see maze:)
28    # Try a generated monument first with /structure monument reset generated.
29    mode: pre-built
30    # Time between checks. After every check one entry is picked at random, by weight.
31    # Units: t = ticks, s = seconds, d = Minecraft days (24000 ticks). 1 real day = 72d.
32    # For testing use e.g. 10s / 15s / 20s / 25s like the datapack did, or run /structure monument
check.
33    intervals:
34      - { delay: 1d, weight: 30 }
35      - { delay: 2d, weight: 30 }
36      - { delay: 3d, weight: 30 }
37      - { delay: 72d, weight: 10 }
38    # When the monument is cleared but a player is within check-radius, it doesn't reset.
39    # Instead the next check is picked from these intervals, which should be shorter than the ones
above.
40    occupied-intervals:
41      - { delay: 300s, weight: 1 }
42      - { delay: 600s, weight: 1 }
43      - { delay: 900s, weight: 1 }
44    # Remove the Guardians left in the area before a new monument is placed,
45    # so they don't pile up over many resets.
46    clear-guardians: true
47    # Send the reset messages (the datapack's /say) to players with the structures_admin permission.
48    # Toggle in-game with /structure monument broadcast [on|off].
49    announce: true
```

- **Line 29:** what a reset places: `pre-built` (one of the 10 variants) or `generated` (a new maze). Lines 25–28 explain the options.
- **Lines 33–37:** the time between checks. After each check one line is picked at random, using `weight` as its share. Out of 30+30+30+10 = 100 that gives:

Wait	Chance
1 Minecraft day	30%
2 Minecraft days	30%
3 Minecraft days	30%
72 Minecraft days (1 real day)	10%

These are the values that were commented out in your datapack. Line 31 lists the units: `t` ticks, `s` seconds, `d` Minecraft days, where 1 day is 20 real minutes. Line 32 suggests short values like `10s` for testing.

- **Lines 40–43:** used when the monument is cleared but a player is still inside. The next check then comes after 5, 10 or 15 minutes, each equally likely. If these are longer than the normal intervals, the plugin logs a warning.
- **Line 46:** before a new monument is placed, all remaining Guardians in the area are removed, so they don't pile up over many resets.
- **Line 49:** reset messages go to players with `structures_admin`. `/structure monument broadcast` toggles this line and saves the file.

Lines 51–65: variants (pre-built monuments)

monuments.yml

```
51  # Pre-built monuments, with their chance weight.
52  # Each name needs <name>_ne, _nw, _se and _sw .nbt files in structures/monument/pre-gen.
53  # monument_00 is the empty monument. It could never be rolled in the datapack, so its weight is 0.
54  variants:
55    monument_00: 0
56    monument_01: 10
57    monument_02: 10
58    monument_03: 10
59    monument_04: 10
60    monument_05: 10
61    monument_06: 10
62    monument_07: 10
63    monument_08: 10
64    monument_09: 10
65    monument_10: 10
```

- **Lines 54–65:** each name stands for four files, `<name>_ne`, `_nw`, `_se` and `_sw` `.nbt`. The number is that variant's weight.
  - `monument_00` (the empty monument) stays at 0, as in your datapack, where it could never be rolled.
  - The other ten each get an equal 10% chance.
  - Set one to 0 to switch it off, or raise it to make it more common.

Lines 67–78: elder-guardian and reward

monuments.yml

```
67  # Elder Guardian stats. Pre-built monuments get one extra in the top room (the quadrant structures
68  # contain the other two); generated monuments get all three, in the penthouse and both wings.
69  elder-guardian:
70    # Pre-built only: height above location.y. The pre-built quadrants are placed 1 block below
71    # location.y; at 11.5 the Elder Guardian is in the same spot of the top room as in the datapack.
72    y-offset: 11.5
73    health: 100
74    attack-damage: 10
75
76  reward:
77    # Give the player who kills the last Elder Guardian a Monument Hoe for mining the sponges.
78    enabled: true
```

- **Line 72:** pre-built only. It's how high above `location.y` the third Elder Guardian spawns: 11.5, because the pre-built quadrants are placed one block below the center (lines 70–71).
- **Lines 73–74:** health and attack damage for the Elder Guardians the plugin spawns: one with pre-built monuments, all three with generated ones. Vanilla is 80 and 8; these are your datapack's tougher values.
- **Line 78:** whether the player who kills the last Elder Guardian gets the Monument Hoe.

Lines 80–97: maze (generated monuments)

monuments.yml

```
  80  # Generated monuments (reset.mode: generated). Preview layouts with /structure monument layout
[seed] [newest-chance],
  81  # or show the one that's built right now with /structure monument layout current. Both print in the
server console.
  82  maze:
  83    # The rooms are laid out with a Growing Tree maze. At every step it continues from the newest
  84    # room with this chance, otherwise from a random room already in the maze:
  85    #   1.0  always the newest: long winding corridors with few dead ends (depth-first)
  86    #   0.0  always random: lots of short branches and dead ends (like Prim's)
  87    # Every monument draws its own chance from a bell curve: most land close to the peak, about
  88    # 2 in 3 within peak +/- spread, and never outside min..max.
  89    newest-chance:
  90      peak: 0.65
  91      spread: 0.15
  92      min: 0.0
  93      max: 1.0
  94    # Every monument gets at least this many sponge rooms (only open at the floor, like vanilla).
  95    # When a layout has too few, a dead end as far from the entrance as possible is turned into one;
  96    # only when there's no suitable dead end, another unit is used and the maze is reconnected around
it.
  97    min-sponge-rooms: 1
```

- **Lines 83–88:** what the newest chance does: 1.0 gives long winding corridors (depth-first), 0.0 gives lots of short branches (like Prim's).
- **Line 90:** the most common value, the top of the bell curve.
- **Line 91:** how wide the curve is. About 2 in 3 monuments get a value between 0.50 and 0.80.
- **Lines 92–93:** hard limits. A value outside them is drawn again.
- **Line 97:** each generated monument gets at least this many sponge rooms. Lines 95–96 describe how: first a dead end far from the entrance is used; only if there isn't one does another unit get used, with the maze reconnected around it.

Step 2: startup, structure modules and settings

A few Java basics first

These come back in every file, so they're explained once here:

What you see	Meaning
`package com.nltarzan.structures;`	The folder the class lives in. It must match the folder path under `src/main/java`.
`import org.bukkit.World;`	Lets the file use a class from another package by its short name (`World` instead of `org.bukkit.World`).
`public final class X`	`public`: other packages can use it. `final`: no other class can extend it.
`extends JavaPlugin`	"X is a kind of JavaPlugin": it inherits everything JavaPlugin can do, like `getServer()`, `getLogger()` and `getDataFolder()`.
`implements Something`	"X promises to have the methods listed in the interface `Something`."
`private final Foo foo;`	A **field**: data the object keeps. `private`: only this class can see it. `final`: set once, then never changed.
`static`	Belongs to the class itself, not to one object, e.g. a constant like `ADMIN_PERMISSION`.
`@Override`	"This replaces a method from the parent class or interface." The compiler checks that the method really exists there.
`/** ... */` and `//`	Comments. `/** */` is documentation that editors show when you hover over a name.
`record`	A short way to write a class that only holds values. Java generates the constructor and getters for you.
`List<Weighted<Long>>`	Generics: "a list of `Weighted`s, each holding a `Long`". The type in `< >` says what's inside.
`null`	"Nothing". A variable that holds no object.

ResettingStructures.java: where the plugin starts

This is the class `plugin.yml` names under `main:`. Paper creates exactly one of it and calls `onEnable()` when the server starts and `onDisable()` when it stops.

ResettingStructures.java

```
1  package com.nltarzan.structures;
2
3  import com.nltarzan.structures.command.StructureCommand;
4  import com.nltarzan.structures.info.InfoFiles;
5  import com.nltarzan.structures.monument.Monuments;
6  import org.bukkit.World;
7  import org.bukkit.plugin.java.JavaPlugin;
8
9  import java.util.List;
10
11 /**
12  * ResettingStructures: custom-built versions of vanilla structures that reset with a new random
13  * layout once a player has cleared them. Each structure is a {@link StructureModule} with its own
14  * config file and /structure subcommand; so far that's the Ocean Monument ({@link Monuments}).
15  */
16 public final class ResettingStructures extends JavaPlugin {
17
18     /** Needed for /structure and to receive the broadcast messages. Not given to anyone by
default. */
19     public static final String ADMIN_PERMISSION = "structures_admin";
20
21     private List<StructureModule> modules = List.of();
```

- **Line 1:** the package. It matches the folder `src/main/java/com/nltarzan/structures/`.
- **Lines 3–9:** what this file uses: the `/structure` command, the info files, the `Monuments` module, Bukkit's `World` and `JavaPlugin`, and Java's `List`.
- **Lines 11–15:** documentation. `{@link …}` turns into a clickable link in editors.
- **Line 16:** the class. `extends JavaPlugin` is what makes it a Paper plugin.
- **Line 19:** the permission name, stored once as a constant. The command and the broadcasts both use `ResettingStructures.ADMIN_PERMISSION`, so renaming it means changing only this line (and `plugin.yml`).
- **Line 21:** the list of structure modules. It starts as an empty list (`List.of()`), so `onDisable` works safely even if `onEnable` failed early.

ResettingStructures.java

```
23     @Override
24     public void onEnable() {
25         getDataFolder().mkdirs();
26         InfoFiles infoFiles = new InfoFiles(this);
27
28         // Every structure the plugin manages. A new structure is one more entry here.
29         modules = List.of(new Monuments(this, infoFiles::writeStatus));
30
31         StructureCommand structureCommand = new StructureCommand();
32         for (StructureModule module : modules) {
33             module.enable();
34             infoFiles.add(module);
35             structureCommand.register(module.commandName(), module.command());
36         }
37         registerCommand("structure", "Manage the resetting structures, e.g. /structure monument
status.", List.of(),
38                 structureCommand);
39
40         infoFiles.writeCommands();
41         infoFiles.writeStatus();
42     }
```

- **Line 24:** called by Paper once, when the plugin starts.

- **Line 25:** makes sure the plugin's folder `plugins/ResettingStructures/` exists.
- **Line 26:** creates the writer for `status.txt` and `commands.txt`.
- **Line 29:** the structures the plugin manages. Right now that's only the Ocean Monument. It gets the plugin itself (`this`) and a way to say "my status changed": `infoFiles::writeStatus` is a **method reference**, short for "call `writeStatus` on `infoFiles`". A future mansion is one more entry in this list.
- **Line 31:** the root command `/structure`.
- **Lines 32–36:** for every structure:
  - **Line 33:** start it: load its config and structure files, start its reset timer, register its listeners.
  - **Line 34:** add it to the info files.
  - **Line 35:** hook up its subcommand, e.g. `/structure monument`.
- **Lines 37–38:** register `/structure` with Paper: its name, a description shown in `/help`, no aliases (`List.of()`), and the command object.
- **Lines 40–41:** write `commands.txt` and `status.txt` for the first time.

ResettingStructures.java

```
44      @Override
45      public void onDisable() {
46          for (StructureModule module : modules) {
47              module.disable();
48          }
49          for (World world : getServer().getWorlds()) {
50              world.removePluginChunkTickets(this);
51          }
52      }
53  }
```

- **Line 45:** called by Paper once, when the server stops or the plugin is turned off.
- **Lines 46–48:** stops every structure, for the monument that's its reset timer.
- **Lines 49–51:** for every world, releases any chunks this plugin was keeping loaded. During a check, the monument's chunks are held loaded with "chunk tickets"; this makes sure none are left behind.

StructureModule.java: what every structure provides

StructureModule.java

```
 9  /**
10   * One kind of structure the plugin manages (so far the Ocean Monument). The main class starts
every
11   * module, gives it its /structure subcommand and collects its status and command help for
12   * status.txt and commands.txt.
13   */
14  public interface StructureModule {
15
16      /** Shown as the heading in /structure ... status, status.txt and commands.txt, e.g. "Ocean
Monument". */
17      String name();
18
19      /** The word after /structure, e.g. "monument". */
20      String commandName();
21
22      StructureCommand.Subcommand command();
23
24      /** The structure's current status, one line per fact. */
25      List<StatusLine> status();
26
27      /** Every command of this structure, for commands.txt. */
28      List<CommandHelp> commandHelp();
29
30      void enable();
31
32      void disable();
33  }
```

An **interface** is a list of methods that a class promises to have. Every structure (now the Ocean Monument, later mansions and end cities) implements this one, so the main class can handle them all the same way:

- **Line 17:** `name()`: the heading in the status, `status.txt` and `commands.txt`, e.g. "Ocean Monument".
- **Line 20:** `commandName()`: the word after `/structure`, e.g. `monument`.
- **Line 22:** `command()`: the object that handles `/structure monument …`.
- **Line 25:** `status()`: the current status, one `StatusLine` per fact.
- **Line 28:** `commandHelp()`: every command with an explanation, for `commands.txt`.
- **Lines 30 and 32:** `enable()` and `disable()`: start and stop.

Monuments.java: everything Ocean Monument

This class is the monument's own "mini plugin". It owns its config file, its saved data and all the parts that do the work. A mansion would later get a `Mansions` class like it.

Lines 34–72: the class and its fields

Monuments.java

```
34  /**
35   * Everything for the Ocean Monument ("monuments" for short): its config file and saved state (both
36   * in the monuments/ folder), the reset timer, the hoe rules and /structure monument.
37   * Other structures get a module like this of their own.
38   *
39   * <p>A monuments.yml that can't be read or has no valid location: section never leads to a
40   * monument being built somewhere else: on startup the monument is paused (no checks, no resets, no
41   * hoe rules) until the file is fixed and reloaded; on a reload the previous settings stay in use.
```

```
 *
 * <p>The same goes while /structure monument active is off, which it is on a new server: whoever
 * installs the plugin first checks the location instead of getting a monument at the default one.
 */
public final class Monuments implements StructureModule {

    private static final String CONFIG_FILE = "monuments.yml";
    private static final DateTimeFormatter CLOCK = DateTimeFormatter.ofPattern("HH:mm");
    private static final DateTimeFormatter DATE_CLOCK = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm");

    /** The outcome of the last check, for the status. */
    private record LastCheck(LocalDateTime time, Result result) {
    }

    private final JavaPlugin plugin;
    private final Runnable statusChanged;
    private final File configFile;
    private final File folder;
    private YamlConfiguration config;

    private MonumentConfig settings;
    /** What's wrong with monuments.yml, or null when it's fine. */
    private String configProblem;
    /** True when no valid monuments.yml has been loaded since the server started. */
    private boolean paused;
    private LastCheck lastCheck;

    private StructureLibrary structures;
    private MonumentResetter resetter;
    private ResetScheduler scheduler;
    private CurrentMonument currentMonument;
```

- **Lines 39–41:** the safety rule: a broken `monuments.yml` never leads to a monument being built somewhere else.
- **Lines 43–44:** the same holds while the monument is switched off with `/structure monument active off`, and a new server starts switched off. So whoever installs the plugin first checks the location, instead of getting a monument at the default one.
- **Line 46:** `implements StructureModule`, so it promises the methods above.
- **Line 48:** the config file's name as a constant.
- **Lines 49–50:** two clock formats for the status: `14:03`, or with the date when it isn't today.
- **Lines 53–54:** a small private record for the last check: when it happened and how it ended.
- **Lines 56–60:**
  - `plugin`: the plugin, needed for Bukkit calls.
  - `statusChanged`: what to call when the status changes (rewrites `status.txt`).
  - `configFile`: `plugins/ResettingStructures/monuments/monuments.yml`.
  - `folder`: `plugins/ResettingStructures/monuments/`, which holds everything of the monument: `monuments.yml`, `state.yml` and `current.yml`.
  - `config`: the file's contents as loaded, which can also be edited and saved.
- **Lines 62–67:**
  - `settings`: the config turned into ready-to-use values (`MonumentConfig`, below).
  - `configProblem`: what's wrong with `monuments.yml`, or `null` when it's fine.
  - `paused`: true when no valid `monuments.yml` was loaded since the server started.
  - `lastCheck`: the outcome of the last check.

- **Lines 69–72:** the parts, each explained in step 4:
  - `structures`: all `.nbt` templates, loaded into memory.
  - `resetter`: does one check, and places a new monument when it's cleared.
  - `scheduler`: the timer that decides *when* checks happen.
  - `currentMonument`: remembers what's built right now, its sponge count and its ID.

Lines 74–141: constructor, name, start and stop

Monuments.java

```
74      /**
75       * @param statusChanged called whenever the monument's status changes, to rewrite status.txt
76       */
77      public Monuments(JavaPlugin plugin, Runnable statusChanged) {
78          this.plugin = plugin;
79          this.statusChanged = statusChanged;
80          this.folder = new File(plugin.getDataFolder(), "monuments");
81          this.configFile = new File(folder, CONFIG_FILE);
82      }
83
84      @Override
85      public String name() {
86          return "Ocean Monument";
87      }
88
89      @Override
90      public String commandName() {
91          return "monument";
92      }
93
94      @Override
95      public void enable() {
96          folder.mkdirs();
97          File oldConfig = new File(plugin.getDataFolder(), CONFIG_FILE);
98          if (oldConfig.exists()) {
99              moveOldConfig(oldConfig);
100         }
101         // Never copy the default over a config that couldn't be moved: without a config the
monument
102         // is paused, while the default's location could be the wrong place.
103         if (!configFile.exists() && !oldConfig.exists()) {
104             plugin.saveResource(folder.getName() + "/" + CONFIG_FILE, false);
105         }
106         loadConfig();
107         structures = new StructureLibrary(plugin);
108         structures.load(settings.variants());
109         currentMonument = new CurrentMonument(plugin.getLogger(), folder);
110         resetter = new MonumentResetter(this);
111         scheduler = new ResetScheduler(this);
112         if (!paused) {
113             scheduler.start();
114             if (!scheduler.isActive()) {
115                 getLogger().warning("The monument is inactive, so nothing is checked or built. On a
new server,"
116                         + " check the location in " + CONFIG_FILE + " (or stand on the center and
run /structure"
117                         + " monument setcenter), then run /structure monument active on.");
118             }
119         }
120
121         MonumentHoe hoe = new MonumentHoe(plugin);
```

```
122          plugin.getServer().getPluginManager().registerEvents(new ElderGuardianListener(this, hoe),
plugin);
123          plugin.getServer().getPluginManager().registerEvents(new MonumentHoeListener(this, hoe),
plugin);
124      }
125
126      @Override
127      public void disable() {
128          if (scheduler != null) {
129              scheduler.stop();
130          }
131      }
132
133      @Override
134      public MonumentCommand command() {
135          return new MonumentCommand(this);
136      }
137
138      @Override
139      public List<CommandHelp> commandHelp() {
140          return MonumentCommand.HELP;
141      }
```

- **Lines 77–82:** the **constructor**, which runs once when `new Monuments(...)` is called. It stores the plugin and the callback (`this.plugin` is the field; `plugin` alone is the parameter) and works out the two paths: first the `monuments/` folder, then the config file inside it. A `File` is only a path; nothing is created yet.
- **Lines 84–92:** the name and command word from `StructureModule`.
- **Lines 95–124, `enable()`:**
  - **Line 96:** create the `monuments/` folder if it doesn't exist.
  - **Lines 97–100:** older versions kept `monuments.yml` directly in `plugins/ResettingStructures/`. If it's still there, move it into `monuments/` (`moveOldConfig`, below).
  - **Lines 101–105:** if there's no `monuments.yml` at all, copy the default one out of the jar: `monuments/monuments.yml` inside the jar ends up as `monuments/monuments.yml` in the plugin's folder. Not when an old one couldn't be moved, though: the monument is then paused for lack of a config, which is safer than a fresh default with a location that may be wrong.
  - **Line 106:** read `monuments.yml` strictly (see `loadConfig` below).
  - **Lines 107–108:** load all structure templates, including the configured pre-built variants.
  - **Line 109:** load `current.yml`.
  - **Lines 110–111:** create the resetter and the timer. They get `this` (the `Monuments` object), so they can reach the settings, the structures and so on.
  - **Lines 112–113:** start the timer, but **not** when paused. The timer itself also does nothing while the monument is inactive (step 4).
  - **Lines 114–118:** inactive (always the case on a new server)? Then log a warning on every startup, explaining how to check the location and turn the monument on.
  - **Line 121:** create the hoe helper, which knows how to make and recognize a Monument Hoe.
  - **Lines 122–123:** register the two **listeners**, classes whose methods Paper calls when something happens in the game (step 7). Both get the same `hoe` object.
- **Lines 127–131:** stop the timer when the plugin shuts down, so no check fires in the middle of shutting down.
- **Lines 134–136:** creates the `/structure monument` command.
- **Lines 139–141:** the command help for `commands.txt`, which lives in `MonumentCommand`.

Lines 143–211: reload, broadcast, setcenter, active, last check

Monuments.java

```
149      public boolean reloadSettings() {
150          boolean ok = loadConfig();
151          if (ok) {
152              structures.load(settings.variants());
153              scheduler.start();
154          }
155          statusChanged.run();
156          return ok;
157      }
158
159      /**
160       * Turns the broadcast messages on or off and saves it in monuments.yml (reset.announce).
161       *
162       * @return false when monuments.yml has a problem; it's then not saved, so it can't overwrite
the file
163       */
164      public boolean setAnnounce(boolean announce) {
165          if (configProblem != null) {
166              return false;
167          }
168          config.set("reset.announce", announce);
169          saveConfig();
170          settings = MonumentConfig.load(config, getLogger());
171          statusChanged.run();
172          return true;
173      }
174
175      /**
176       * Moves the monument's center and saves it in monuments.yml.
177       *
178       * @return false when monuments.yml has a problem; it's then not saved, so it can't overwrite
the file
179       */
180      public boolean setCenter(World world, int x, int y, int z) {
181          if (configProblem != null) {
182              return false;
183          }
184          config.set("location.world", world.getName());
185          config.set("location.x", x);
186          config.set("location.y", y);
187          config.set("location.z", z);
188          saveConfig();
189          reloadSettings();
190          return true;
191      }
192
193      /**
194       * Turns the checks and resets on or off. Saved in monuments/state.yml, so it lasts through
restarts.
195       *
196       * @return false when turning on while monuments.yml has a problem; the location isn't known
then
197       */
198      public boolean setActive(boolean active) {
199          if (active && paused) {
200              return false;
201          }
202          scheduler.setActive(active);
203          statusChanged.run();
```

```
204          return true;
205      }
206
207      /** Remembers the outcome of a check for the status. Called by the scheduler after every check.
*/
208      void recordCheck(Result result) {
209          lastCheck = new LastCheck(LocalDateTime.now(), result);
210          statusChanged.run();
211      }
```

- **Lines 149–157,** `/structure monument reload`**:** re-read the file. Only if it's fine, reload the templates (in case the variants changed) and restart the timer; the timer keeps the saved countdown, so a reload doesn't reset the wait. Then update `status.txt`, and return whether it worked.
- **Lines 164–173,** `/structure monument broadcast`**:**
  - **Lines 165–167:** refuse while `monuments.yml` has a problem. Saving would overwrite the file you're fixing.
  - **Lines 168–172:** change `reset.announce`, save the file, rebuild the settings so it takes effect immediately, and update the status.
- **Lines 180–191,** `/structure monument setcenter`**:** the same safety check, then write the new position into the `location:` section, save, and reload so the timer and checks use the new spot.
- **Lines 198–205,** `/structure monument active on|off`**:**
  - **Lines 199–201:** turning **on** is refused while `monuments.yml` has a problem, because the location isn't known then. Turning off always works.
  - **Line 202:** the timer does the actual work and saves the choice in `state.yml` (step 4).
  - **Line 203:** update `status.txt`.
- **Lines 208–211:** called by the timer after every check: remember the time and outcome for the status, and update `status.txt`.

Lines 213–224: the monument area

Monuments.java

```
213      /** The monument area, or {@code null} when its world isn't loaded or the monument isn't
active. */
214      public MonumentArena arena() {
215          if (!isActive()) {
216              return null;
217          }
218          World world = plugin.getServer().getWorld(settings.worldName());
219          if (world == null) {
220              return null;
221          }
222          return new MonumentArena(plugin, world, settings.centerX(), settings.centerY(),
settings.centerZ(),
223                  settings.checkRadius());
224      }
```

- **Lines 215–217:** no area while the monument isn't active (`isActive()`, at the end of the file): when paused there's no known location, and when switched off the default location may well be someone's build. Because the hoe rules and the Elder Guardian reward both ask for the area, they're off too.
- **Line 218:** look up the world by name.
- **Lines 219–221:** if that world isn't loaded (or the name is wrong), return `null`. Every caller checks for that.
- **Lines 222–223:** otherwise create a `MonumentArena`: the center, the radius and the 58×22×58 box, with helpers like "which entities are at the monument" and "count the sponges" (step 4). A new one each time is

cheap, and it means a changed center takes effect immediately.

Lines 226–303: the status

Monuments.java

```
226      @Override
227      public List<StatusLine> status() {
228          List<StatusLine> lines = new ArrayList<>();
229          if (configProblem != null) {
230              lines.add(new StatusLine("Problem", "monuments.yml " + configProblem + (paused
231                      ? ". Paused until it's fixed and reloaded."
232                      : ". Still using the previous settings."), NamedTextColor.RED));
233          }
234          lines.add(new StatusLine("Location", paused ? "Unknown" : settings.worldName() + " "
235                  + settings.centerX() + " " + settings.centerY() + " " + settings.centerZ(),
236                  paused ? NamedTextColor.DARK_GRAY : NamedTextColor.WHITE));
237          lines.add(currentMonument.get()
238                  .map(built -> new StatusLine("Monument", switch (built) {
239                      case Built.PreBuilt(String variant) -> "Pre-built " + variant;
240                      case Built.Generated generated -> "Generated, seed " + generated.seed();
241                  }, NamedTextColor.AQUA))
242                  .orElse(new StatusLine("Monument", "Not placed by the plugin yet",
NamedTextColor.DARK_GRAY)));
243          currentMonument.sponges().ifPresent(sponges ->
244                  lines.add(new StatusLine("Sponges", String.valueOf(sponges),
NamedTextColor.YELLOW)));
245
246          MonumentArena arena = arena();
247          if (arena != null && arena.world().isChunkLoaded(arena.center().getBlockX() >> 4,
arena.center().getBlockZ() >> 4)) {
248              int alive = arena.entities(ElderGuardian.class).size();
249              lines.add(new StatusLine("Elder Guardians", alive + " alive",
250                      alive > 0 ? NamedTextColor.RED : NamedTextColor.GREEN));
251          }
252          if (lastCheck != null) {
253              lines.add(lastCheckLine());
254          }
255
256          String mode = settings.mode() == ResetMode.GENERATED ? "Generated" : "Pre-built";
257          lines.add(structures.mazeReady()
258                  ? new StatusLine("Reset mode", mode, NamedTextColor.AQUA)
259                  : new StatusLine("Reset mode", mode + " (generated templates missing)",
NamedTextColor.RED));
260
261          long ticks = scheduler.ticksUntilNextCheck();
262          lines.add(scheduler.isActive()
263                  ? new StatusLine("Active", "On", NamedTextColor.GREEN)
264                  : new StatusLine("Active", "Off: nothing is checked or built. Check the location,
then run"
265                          + " /structure monument active on", NamedTextColor.GOLD));
266          // While inactive there's no next check; the Active line above already says why.
267          if (paused) {
268              lines.add(new StatusLine("Next check", "Paused", NamedTextColor.RED));
269          } else if (scheduler.isActive()) {
270              lines.add(ticks < 0
271                      ? new StatusLine("Next check", "Not scheduled", NamedTextColor.RED)
272                      : new StatusLine("Next check", "in " + Durations.format(ticks) + " (around "
273                              + clockIn(ticks / Durations.TICKS_PER_SECOND) + ")",
NamedTextColor.WHITE));
274          }
275          lines.add(settings.announce()
276                  ? new StatusLine("Broadcasts", "On", NamedTextColor.GREEN)
```

```
277                 : new StatusLine("Broadcasts", "Off", NamedTextColor.RED));
278         return lines;
279     }
```

This builds the list of status lines used by both `/structure monument status` (in color) and `status.txt` (plain text). Each `StatusLine` has a label, a value and a color.

- **Lines 229–233:** a red **Problem** line first, when `monuments.yml` has a mistake.
- **Lines 234–236:** the location, or "Unknown" while paused.
- **Lines 237–242:** what's built: "Pre-built monument_03", "Generated, seed 123", or "Not placed by the plugin yet". `get()` returns an `Optional` (a box that may be empty); `.map(...)` turns its content into a line, `.orElse(...)` is used when it's empty.
- **Lines 243–244:** the sponge count, only when known.
- **Lines 246–251:** the number of Elder Guardians alive, red when above 0 and green at 0. Only shown when the area is loaded, because the count isn't reliable otherwise. `>> 4` turns a block coordinate into a chunk coordinate (divide by 16).
- **Lines 252–254:** the last check (below).
- **Lines 256–259:** the reset mode, with a red note when the generated templates didn't load. `a ? b : c` means "if a then b, otherwise c".
- **Lines 262–265: Active**: "On" in green, or "Off" in gold with what to do about it.
- **Lines 266–274:** the next check: "Paused", "Not scheduled", or "in 1h 4m 30s (around 15:08)". While the monument is switched off there's no next check, so this line is left out (the Active line already explains why).
- **Lines 275–277:** broadcasts on (green) or off (red).

Monuments.java

```
281     private StatusLine lastCheckLine() {
282         String time = clock(lastCheck.time());
283         return switch (lastCheck.result()) {
284             case Result.NotCleared(int alive) -> new StatusLine("Last check",
285                     time + ", not cleared: " + alive + " Elder Guardian(s) alive",
NamedTextColor.RED);
286             case Result.PlayersInside(int players) -> new StatusLine("Last check",
287                     time + ", cleared but " + players + " player(s) inside, reset postponed",
NamedTextColor.YELLOW);
288             case Result.Placed(String placed) -> new StatusLine("Last check",
289                     time + ", reset: " + placed, NamedTextColor.GREEN);
290             case Result.Failed(String reason) -> new StatusLine("Last check",
291                     time + ", failed: " + reason, NamedTextColor.RED);
292         };
293     }
294
295     /** A clock time, with the date when it isn't today. */
296     private static String clock(LocalDateTime time) {
297         return time.toLocalDate().equals(LocalDate.now()) ? CLOCK.format(time) :
DATE_CLOCK.format(time);
298     }
299
300     /** The clock time a number of seconds from now (an estimate: game time only runs while the
server does). */
301     private static String clockIn(long seconds) {
302         return clock(LocalDateTime.now().plusSeconds(seconds));
303     }
```

- **Lines 281–293:** the last check's line, by outcome: not cleared (red), cleared but players inside (yellow), reset (green) or failed (red). The `switch` with `case Result.NotCleared(int alive)` takes the number out of the result directly; this is called a **record pattern**.

- **Lines 296–298:** a time as `14:03`, or with the date when it isn't today.
- **Lines 301–303:** the clock time some seconds from now. It's an estimate, because game time only runs while the server does.

Lines 305–380: reading, moving and saving monuments.yml safely

Monuments.java

```
310      private boolean loadConfig() {
311          YamlConfiguration yaml = new YamlConfiguration();
312          String problem;
313          String details = null;
314          try {
315              yaml.load(configFile);
316              problem = MonumentConfig.locationProblem(yaml);
317          } catch (IOException | InvalidConfigurationException e) {
318              problem = "can't be read (" + firstLine(e.getMessage()) + ")";
319              details = e.getMessage();
320          }
321
322          if (problem == null) {
323              config = yaml;
324              settings = MonumentConfig.load(yaml, getLogger());
325              configProblem = null;
326              paused = false;
327              return true;
328          }
329
330          configProblem = problem;
331          if (settings == null) {
332              // Startup: no earlier settings to fall back on. Use the defaults for everything except
the
333              // location, and pause so nothing is checked or built until the file is fixed.
334              config = yaml;
335              settings = MonumentConfig.load(new YamlConfiguration(), Logger.getAnonymousLogger());
336              paused = true;
337              getLogger().severe(CONFIG_FILE + " " + problem + ". The monument is paused: no checks,
resets or hoe"
338                      + " rules until it's fixed and /structure monument reload is run.");
339          } else {
340              getLogger().severe(CONFIG_FILE + " " + problem + ". Still using the previous settings;
fix it and"
341                      + " run /structure monument reload again.");
342          }
343          if (details != null) {
344              getLogger().severe(details);
345          }
346          return false;
347      }
```

- **Lines 311–320:** read the file strictly. `yaml.load` throws an error on a YAML mistake (for example a missing space after a colon), instead of quietly returning an empty file. If it reads fine, check the `location:` section (`MonumentConfig.locationProblem`, below). `try` / `catch` catches the error and turns it into a problem description; `details` keeps the full error text for the console.
- **Lines 322–328:** no problem: use it, clear any old problem and un-pause.
- **Lines 330–346:** a problem:
  - **Lines 331–338, at startup** (no earlier settings): use the defaults for everything except the location, **pause** the monument, and log a clear error.
  - **Lines 339–342, on a reload:** keep the previous working settings, and log the error.

- **Lines 343–345:** log the full error details too.

Without this, a broken file would make every coordinate 0, and the next check would build a monument at 0, 0, 0.

Monuments.java

```
 349      /** Moves monuments.yml from the plugin's folder (where older versions kept it) into
monuments/. */
 350      private void moveOldConfig(File oldConfig) {
 351          if (configFile.exists()) {
 352              getLogger().warning("There's a " + CONFIG_FILE + " in both " +
plugin.getDataFolder().getName() + "/ and "
 353                      + folder.getName() + "/. Using " + folder.getName() + "/" + CONFIG_FILE + ";
the other one is"
 354                      + " ignored and can be deleted.");
 355              return;
 356          }
 357          try {
 358              Files.move(oldConfig.toPath(), configFile.toPath());
 359              getLogger().info("Moved " + CONFIG_FILE + " into the " + folder.getName() + "/
folder.");
 360          } catch (IOException e) {
 361              getLogger().log(Level.SEVERE, "Could not move " + CONFIG_FILE + " into the " +
folder.getName()
 362                      + "/ folder. Move it there by hand and run /structure monument reload.", e);
 363          }
 364      }
```

- **Moving an old `monuments.yml` into `monuments/`**, for servers that ran an older version of the plugin.
- **Lines 351–356:** there's already one in `monuments/`? Then that one wins, and a warning says the old one can be deleted.
- **Lines 357–359:** otherwise move the file and log it. Your settings stay exactly as they were.
- **Lines 360–363:** if moving fails (for example because another program has the file open), log an error saying to move it by hand. The monument then stays paused until that's done and `reload` is run.

Monuments.java

```
 366      private static String firstLine(String message) {
 367          if (message == null) {
 368              return "unknown error";
 369          }
 370          String line = message.strip().lines().findFirst().orElse(message);
 371          return line.length() > 120 ? line.substring(0, 120) + "..." : line;
 372      }
 373
 374      private void saveConfig() {
 375          try {
 376              config.save(configFile);
 377          } catch (IOException e) {
 378              getLogger().log(Level.WARNING, "Could not save " + CONFIG_FILE, e);
 379          }
 380      }
```

- **Lines 366–372:** the first line of an error message, shortened to 120 characters, for the status.
- **Lines 374–380:** write the YAML back to disk. Paper keeps the comments when it does this. If saving fails, a warning is logged instead of crashing.

Lines 382–433: getters

The rest of the file are **getters**, small methods that hand out a field so the other parts can reach it: `configProblem()`, `isPaused()`, `isActive()`, `plugin()`, `folder()`, `getLogger()`, `getServer()`, `isEnabled()`, `settings()`, `structures()`, `resetter()`, `scheduler()` and `currentMonument()`. `getLogger()`, `getServer()` and `isEnabled()` pass straight through to the plugin, so the other monument classes can write `monuments.getLogger()` without knowing about the plugin.

One of them is more than a getter:

Monuments.java

```
392     /** Whether checks, resets and the hoe rules run: monuments.yml is fine and active is on. */
393     public boolean isActive() {
394         return !paused && scheduler.isActive();
395     }
```

- **Line 394:** the monument only does anything (checks, resets, hoe rules, the reward) when `monuments.yml` is fine **and** it's switched on. Everything else asks this one method, so the rule lives in a single place.

MonumentConfig.java: reading monuments.yml

This turns the raw YAML into a fixed set of checked values. Bad values get a warning in the console and a safe default, so a mistake in the config can't crash the plugin.

MonumentConfig.java

```
16  public record MonumentConfig(
17          String worldName,
18          int centerX,
19          int centerY,
20          int centerZ,
21          double checkRadius,
22          List<Weighted<Long>> intervals,
23          List<Weighted<Long>> occupiedIntervals,
24          ResetMode mode,
25          boolean clearGuardians,
26          boolean announce,
27          List<Weighted<String>> variants,
28          double elderYOffset,
29          double elderHealth,
30          double elderAttackDamage,
31          boolean rewardEnabled,
32          MazeSettings maze) {
33
34      private static final long FALLBACK_INTERVAL = Durations.TICKS_PER_DAY;
35      private static final long FALLBACK_OCCUPIED_INTERVAL = 300 * Durations.TICKS_PER_SECOND;
```

- **Line 16:** a **record**: Java creates a getter for each value automatically (e.g. `settings.worldName()`). It can't be changed after it's made; a reload builds a new one.
- **Lines 17–32:** one value per setting in `monuments.yml`:
  - `location:` → `worldName`, `centerX/Y/Z`, `checkRadius`
  - `reset:` → the two interval lists (each entry a delay in ticks with its weight), `mode`, `clearGuardians`, `announce`
  - `variants:` → a list of names with weights
  - `elder-guardian:` → three numbers
  - `reward:` → one on/off value

- `maze:` → a `MazeSettings` object (step 5)

- **Lines 34–35:** what to use when an interval list is unusable: 1 Minecraft day for normal checks, 300 seconds when players are inside.

MonumentConfig.java

```
37      public static MonumentConfig load(FileConfiguration config, Logger logger) {
38          List<Weighted<Long>> intervals = loadIntervals(config, "reset.intervals",
FALLBACK_INTERVAL, logger);
39          List<Weighted<Long>> occupied = loadIntervals(config, "reset.occupied-intervals",
40                  FALLBACK_OCCUPIED_INTERVAL, logger);
41          if (longest(occupied) >= shortest(intervals)) {
42              logger.warning("reset.occupied-intervals should be shorter than reset.intervals, "
43                      + "otherwise a postponed reset waits as long as a normal one.");
44          }
45
46          return new MonumentConfig(
47                  config.getString("location.world", "world"),
48                  config.getInt("location.x"),
49                  config.getInt("location.y"),
50                  config.getInt("location.z"),
51                  Math.max(1, config.getDouble("location.check-radius", 40)),
52                  intervals,
53                  occupied,
54                  loadMode(config, logger),
55                  config.getBoolean("reset.clear-guardians", true),
56                  config.getBoolean("reset.announce", true),
57                  loadVariants(config, logger),
58                  config.getDouble("elder-guardian.y-offset", 11.5),
59                  config.getDouble("elder-guardian.health", 100),
60                  config.getDouble("elder-guardian.attack-damage", 10),
61                  config.getBoolean("reward.enabled", true),
62                  loadMaze(config, logger));
63      }
```

- **Line 37:** `static` because it's called as `MonumentConfig.load(...)`, without an object. It takes the YAML and a logger for warnings.
- **Lines 38–40:** read both interval lists (`loadIntervals`, below).
- **Lines 41–44:** if the longest "players inside" wait isn't shorter than the shortest normal wait, log a warning. The values are still used.
- **Lines 46–62:** builds the record, in the same order as lines 17–32. A YAML path like `"location.x"` means "the `x` under `location:`". The second argument is the default if a setting is missing. `Math.max(1, …)` stops the radius from being 0 or negative. Settings that need more checking go through their own helper methods.

MonumentConfig.java

```
65      /**
66       * Checks the location: section, which says where the monument is. Without a valid one every
67       * coordinate would quietly become 0 and a reset would build a monument at 0, 0, 0.
68       *
69       * @return what's wrong, or {@code null} when it's fine
70       */
71      public static String locationProblem(ConfigurationSection config) {
72          if (!config.isConfigurationSection("location")) {
73              return "has no location: section";
74          }
75          if (!config.isString("location.world") || config.getString("location.world").isBlank()) {
76              return "has no location.world";
77          }
78          for (String axis : List.of("x", "y", "z")) {
79              if (!config.isInt("location." + axis)) {
80                  return "has no whole number for location." + axis;
81              }
82          }
83          return null;
84      }
```

- **The strict check of `location:`**, used by `Monuments.loadConfig`.
- **Lines 72–74:** the section must exist.
- **Lines 75–77:** `world` must be text and not empty.
- **Lines 78–82:** `x`, `y` and `z` must be whole numbers.
- **Line 83:** `null` means "no problem".

MonumentConfig.java

```
86      /** What a reset places. */
87      public enum ResetMode {
88          PRE_BUILT,
89          GENERATED
90      }
91
92      private static ResetMode loadMode(FileConfiguration config, Logger logger) {
93          String mode = config.getString("reset.mode", "pre-built");
94          return switch (mode.trim().toLowerCase(Locale.ROOT)) {
95              case "pre-built" -> ResetMode.PRE_BUILT;
96              case "generated" -> ResetMode.GENERATED;
97              default -> {
98                  logger.warning("Unknown reset.mode '" + mode + "', use pre-built or generated.
Using pre-built.");
99                  yield ResetMode.PRE_BUILT;
100             }
101         };
102     }
```

- **Lines 87–90:** an **enum** is a fixed list of choices. A reset mode can only be one of these two, which is safer than passing text around.
- **Line 93:** reads the text, `pre-built` by default.
- **Line 94:** `trim()` removes stray spaces; `toLowerCase` makes `Generated` work too; `Locale.ROOT` makes that work the same on any computer's language settings. `switch` picks a result based on the text.
- **Lines 97–100:** anything else (a typo like `generatd`) logs a warning and uses pre-built. `yield` is how a `{ }` block inside a switch hands back its result.

MonumentConfig.java

```
104      private static MazeSettings loadMaze(FileConfiguration config, Logger logger) {
105          MazeSettings defaults = MazeSettings.DEFAULT;
106          double min = clamp(config.getDouble("maze.newest-chance.min", defaults.min()));
107          double max = clamp(config.getDouble("maze.newest-chance.max", defaults.max()));
108          if (max < min) {
109              logger.warning("maze.newest-chance.max is below maze.newest-chance.min, using " + min +
" for both.");
110              max = min;
111          }
112          double peak = config.getDouble("maze.newest-chance.peak", defaults.peak());
113          if (peak < min || peak > max) {
114              logger.warning("maze.newest-chance.peak " + peak + " is outside min..max, moving it
inside.");
115              peak = Math.clamp(peak, min, max);
116          }
117          double spread = Math.max(0, config.getDouble("maze.newest-chance.spread",
defaults.spread()));
118          int sponges = Math.max(0, config.getInt("maze.min-sponge-rooms",
defaults.minSpongeRooms()));
119          return new MazeSettings(peak, spread, min, max, sponges);
120      }
121
122      private static double clamp(double chance) {
123          return Math.clamp(chance, 0.0, 1.0);
124      }
```

- **Line 105:** the built-in defaults (peak 0.65, spread 0.15, 0–1, at least 1 sponge room), used for any missing setting.
- **Lines 106–107:** read min and max and force them between 0 and 1 (`clamp`, lines 122–124).
- **Lines 108–111:** if max is below min, use min for both.
- **Lines 112–116:** if the peak lies outside min..max, move it to the nearest edge.
- **Lines 117–118:** spread and the sponge minimum can't be negative.
- **Line 119:** builds the maze settings. `MazeSettings` checks the values once more itself.

MonumentConfig.java

```
126     private static List<Weighted<Long>> loadIntervals(FileConfiguration config, String path, long
fallback,
127                                                       Logger logger) {
128         List<Weighted<Long>> intervals = new ArrayList<>();
129         for (Map<?, ?> entry : config.getMapList(path)) {
130             Object delay = entry.get("delay");
131             Object weight = entry.get("weight");
132             try {
133                 long ticks = Durations.parseTicks(String.valueOf(delay));
134                 int w = weight == null ? 1 : Integer.parseInt(String.valueOf(weight));
135                 intervals.add(new Weighted<>(ticks, w));
136             } catch (IllegalArgumentException e) {
137                 logger.warning("Skipping " + path + " entry " + entry + ": " + e.getMessage());
138             }
139         }
140         if (intervals.stream().noneMatch(w -> w.weight() > 0)) {
141             logger.warning("No usable " + path + ", falling back to " + Durations.format(fallback)
+ ".");
142             intervals = List.of(new Weighted<>(fallback, 1));
143         }
144         return List.copyOf(intervals);
145     }
146
147     private static long shortest(List<Weighted<Long>> intervals) {
148         return intervals.stream().filter(w -> w.weight() >
0).mapToLong(Weighted::value).min().orElse(0);
149     }
150
151     private static long longest(List<Weighted<Long>> intervals) {
152         return intervals.stream().filter(w -> w.weight() >
0).mapToLong(Weighted::value).max().orElse(0);
153     }
```

- **Used for both interval lists.**
- **Line 128:** start with an empty list.
- **Line 129:** go through each `- { delay: ..., weight: ... }` line. `getMapList` gives each one as a map of names to values.
- **Lines 130–131:** take out `delay` and `weight`.
- **Line 133:** turn `1d`, `10s` or `5t` into ticks (`Durations.parseTicks`, step 4).
- **Line 134:** the weight, or 1 if it's left out.
- **Lines 136–138:** a bad entry is skipped with a warning naming it; the others still work.
- **Lines 140–143:** if nothing usable is left, use the fallback. `stream().noneMatch(...)` means "none of the entries has a weight above 0". `w -> w.weight() > 0` is a **lambda**, a small inline function.
- **Line 144:** `List.copyOf` makes the list unchangeable.
- **Lines 147–153:** the shortest and longest delay with a weight above 0, for the warning on lines 41–44.

MonumentConfig.java

```
155     private static List<Weighted<String>> loadVariants(FileConfiguration config, Logger logger) {
156         ConfigurationSection section = config.getConfigurationSection("variants");
157         if (section == null) {
158             logger.warning("No variants configured.");
159             return List.of();
160         }
161         List<Weighted<String>> variants = new ArrayList<>();
162         for (String name : section.getKeys(false)) {
163             int weight = section.getInt(name);
164             if (weight < 0) {
165                 logger.warning("Variant " + name + " has a negative weight, using 0.");
166                 weight = 0;
167             }
168             variants.add(new Weighted<>(name, weight));
169         }
170         return List.copyOf(variants);
171     }
```

- **Line 156:** the `variants:` section.
- **Lines 157–160:** if it's missing, warn and return an empty list. Pre-built resets then fail with a clear message instead of crashing.
- **Line 162:** each name under `variants:`. `false` means "only the names directly under it, not deeper".
- **Lines 163–167:** read the weight; a negative weight becomes 0, with a warning.

The info package: status.txt and commands.txt

StatusLine.java and CommandHelp.java

StatusLine.java

```
 9  public record StatusLine(String label, String value, NamedTextColor color) {
10  }
```

- **One fact in a status**, e.g. "Sponges: 87", with the color the status command uses. `status.txt` uses only the label and value.

CommandHelp.java

```
12  public record CommandHelp(String usage, String description, List<String> examples) {
13
14      public CommandHelp {
15          examples = List.copyOf(examples);
16      }
17
18      public CommandHelp(String usage, String description) {
19          this(usage, description, List.of());
20      }
21  }
```

- **One command for `commands.txt`:** how it's typed, what it does, and example uses.
- **Lines 14–16:** stores an unchangeable copy of the examples.
- **Lines 18–20:** a second constructor for commands without examples; it calls the first one with an empty list.

InfoFiles.java

InfoFiles.java

```
23  public final class InfoFiles {
24
25      public static final String STATUS_FILE = "status.txt";
26      public static final String COMMANDS_FILE = "commands.txt";
27
28      private static final int WIDTH = 90;
29      private static final DateTimeFormatter TIME = DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss");
30
31      /** One structure's section in status.txt. */
32      public record StatusSection(String name, List<StatusLine> lines) {
33      }
34
35      /** One structure's group in commands.txt. */
36      public record CommandGroup(String name, String commandName, List<CommandHelp> commands) {
37      }
38
39      private final JavaPlugin plugin;
40      private final List<StructureModule> modules = new ArrayList<>();
41
42      public InfoFiles(JavaPlugin plugin) {
43          this.plugin = plugin;
44      }
45
46      public void add(StructureModule module) {
47          modules.add(module);
48      }
49
50      public void writeStatus() {
51          List<StatusSection> sections = modules.stream().map(m -> new StatusSection(m.name(),
m.status())).toList();
52          write(STATUS_FILE, statusText(sections, LocalDateTime.now()));
53      }
54
55      public void writeCommands() {
56          List<CommandGroup> groups = modules.stream()
57                  .map(m -> new CommandGroup(m.name(), m.commandName(), m.commandHelp())).toList();
58          write(COMMANDS_FILE, commandsText(groups));
59      }
```

- **Lines 25–26:** the two file names.
- **Line 28:** the width of a line in the files (90 characters).
- **Line 29:** the date format for "Updated 2026-10-06 14:03:12".
- **Lines 32–37:** two small records: one structure's section in `status.txt`, and one structure's group in `commands.txt`.
- **Lines 39–48:** the plugin, and the list of structures added by the main class.
- **Lines 50–53:** collect every structure's status and write `status.txt`.
- **Lines 55–59:** collect every structure's command help and write `commands.txt`.

InfoFiles.java

```
61      /** The text of status.txt. */
62      public static String statusText(List<StatusSection> sections, LocalDateTime updated) {
63          StringBuilder out = new StringBuilder();
64          header(out, "ResettingStructures - status",
65                  "Updated " + TIME.format(updated) + ". Rewritten by the plugin whenever something
changes.");
66          for (StatusSection section : sections) {
67              out.append('\n').append(section.name()).append('\n');
68              int labelWidth = section.lines().stream().mapToInt(l ->
```

```
l.label().length()).max().orElse(0) + 2;
  69              for (StatusLine line : section.lines()) {
  70                  out.append("  ").append(pad(line.label() + ":",
labelWidth)).append(line.value()).append('\n');
  71              }
  72          }
  73          return out.toString();
  74      }
  75
  76      /** The text of commands.txt. */
  77      public static String commandsText(List<CommandGroup> groups) {
  78          StringBuilder out = new StringBuilder();
  79          header(out, "ResettingStructures - commands",
  80                  "Rewritten by the plugin on every start, so it always matches the installed
version.");
  81          out.append('\n')
  82                  .append("Every command needs the
").append(ResettingStructures.ADMIN_PERMISSION).append(" permission.\n")
  83                  .append("How to read them:  <value> = required   [value] = optional   a|b = one of
these\n")
  84                  .append('\n')
  85                  .append("/structure\n");
  86          wrap(out, "Lists the structures this plugin manages. Every structure has its own commands,
below.", "    ");
  87
  88          for (CommandGroup group : groups) {
  89              out.append('\n').append("-".repeat(WIDTH)).append('\n')
  90                      .append(group.name()).append("   (/structure
").append(group.commandName()).append(" ...)\n")
  91                      .append("-".repeat(WIDTH)).append('\n');
  92              for (CommandHelp command : group.commands()) {
  93                  out.append('\n').append(command.usage()).append('\n');
  94                  wrap(out, command.description(), "    ");
  95                  for (String example : command.examples()) {
  96                      out.append("    Example: ").append(example).append('\n');
  97                  }
  98              }
  99          }
 100          return out.toString();
 101      }
```

- **Lines 62–74, the text of `status.txt`:** the header, then per structure its name and its lines. Line 68 finds the longest label, so `pad` can line all values up in one column.
- **Lines 77–101, the text of `commands.txt`:** the header, the permission note, how to read `<required>` / `[optional]` / `a|b`, the `/structure` command itself, and then per structure a ruled heading followed by each command, its wrapped explanation and its examples.

InfoFiles.java

```
 103      private static void header(StringBuilder out, String title, String note) {
 104          out.append("=".repeat(WIDTH)).append('\n')
 105                  .append(title).append('\n')
 106                  .append("READ-ONLY: generated by the plugin, any changes you make to this file are
lost.").append('\n')
 107                  .append(note).append('\n')
 108                  .append("=".repeat(WIDTH)).append('\n');
 109      }
 110
 111      /** Word-wraps a text to the file width, every line starting with the indent. */
 112      private static void wrap(StringBuilder out, String text, String indent) {
 113          StringBuilder line = new StringBuilder(indent);
 114          for (String word : text.split(" ")) {
```

```
115             if (line.length() > indent.length() && line.length() + 1 + word.length() > WIDTH) {
116                 out.append(line).append('\n');
117                 line = new StringBuilder(indent);
118             }
119             if (line.length() > indent.length()) {
120                 line.append(' ');
121             }
122             line.append(word);
123         }
124         out.append(line).append('\n');
125     }
126 
127     private static String pad(String text, int width) {
128         return text.length() >= width ? text + " " : text + " ".repeat(width - text.length());
129     }
130 
131     private void write(String name, String text) {
132         File file = new File(plugin.getDataFolder(), name);
133         try {
134             if (file.exists()) {
135                 file.setWritable(true);
136             }
137             Files.writeString(file.toPath(), text, StandardCharsets.UTF_8);
138             file.setReadOnly();
139         } catch (IOException e) {
140             plugin.getLogger().log(Level.WARNING, "Could not write " + name, e);
141         }
142     }
```

- **Lines 103–109:** the header both files share, with the READ-ONLY warning.
- **Lines 112–125:** word-wraps a text to 90 characters: words are added to a line until the next one wouldn't fit, then a new line starts with the indent.
- **Lines 127–129:** pads a text with spaces to a fixed width.
- **Lines 131–142, writing a file:** if it exists, make it writable again (line 135), write the text as UTF-8 (line 137), then mark it read-only (line 138), so an editor warns before anyone saves over it. A failure is logged as a warning.

Step 3: the commands

`StructureCommand.java` is the shared `/structure` root. `MonumentCommand.java` holds everything after `/structure monument`.

StructureCommand.java: the /structure root

This class only works out *which structure* is meant, then hands the rest of the command to that structure's own handler.

StructureCommand.java

```
1  package com.nltarzan.structures.command;
2
3  import com.nltarzan.structures.ResettingStructures;
4  import io.papermc.paper.command.brigadier.BasicCommand;
5  import io.papermc.paper.command.brigadier.CommandSourceStack;
6  import net.kyori.adventure.text.Component;
7  import net.kyori.adventure.text.format.NamedTextColor;
8
9  import java.util.Arrays;
10 import java.util.Collection;
11 import java.util.LinkedHashMap;
12 import java.util.List;
13 import java.util.Locale;
14 import java.util.Map;
15
16 /**
17  * /structure &lt;structure&gt; ...: the root command. Everything after the structure name goes to
18  * that structure's own command, e.g. /structure monument status. Other structures (mansions, end
19  * cities, ...) are added with {@link #register}.
20  */
21 public final class StructureCommand implements BasicCommand {
```

- **Line 1:** the package. It sits outside `monument`, because it's shared by every structure.
- **Line 3:** for the permission constant.
- **Lines 4–5:** Paper's command system. `BasicCommand` is Paper's simple way to make a command: you get the typed words as a list of text. `CommandSourceStack` is "who ran it, and where": a player, the console or a command block.
- **Lines 6–7:** Adventure, the text library Paper uses for messages. `Component` is a piece of chat text; `NamedTextColor` is a color like `RED`.
- **Line 21:** `implements BasicCommand` means "this class *is* a Paper command". `BasicCommand` demands `execute`, and optionally `suggest` and `permission`.

StructureCommand.java

```
23      /** The commands of one structure. Gets the arguments after the structure name. */
24      public interface Subcommand {
25          void execute(CommandSourceStack source, String[] args);
26
27          Collection<String> suggest(CommandSourceStack source, String[] args);
28      }
29
30      private final Map<String, Subcommand> structures = new LinkedHashMap<>();
31
32      public StructureCommand register(String name, Subcommand command) {
33          structures.put(name.toLowerCase(Locale.ROOT), command);
34          return this;
35      }
36
37      @Override
38      public String permission() {
39          return ResettingStructures.ADMIN_PERMISSION;
40      }
```

- **Lines 24–28:** an interface of our own: what every structure's command must offer. `execute` runs the command; `suggest` returns the tab-completion options. Both receive `String[] args`, the words typed **after** the structure name, so for `/structure monument reset generated` the monument's command receives `["reset", "generated"]`. `String[]` is an **array**: a fixed-size list.
- **Line 30:** a **map** is a lookup table from a key to a value; here from a structure name to its command. `LinkedHashMap` remembers the order things were added, so the usage message and tab completion list structures in a fixed order.
- **Lines 32–35:** adds a structure, stored under its lowercase name so `/structure MONUMENT` works too. `return this` hands the same object back, so calls can be chained.
- **Lines 37–40:** tells Paper that `/structure` needs `structures_admin`. Paper then does all the checking itself: players without it can't run it, don't get it in tab completion, and their client doesn't even see that the command exists.

StructureCommand.java

```
42      @Override
43      public void execute(CommandSourceStack source, String[] args) {
44          Subcommand command = args.length == 0 ? null :
structures.get(args[0].toLowerCase(Locale.ROOT));
45          if (command == null) {
46              source.getSender().sendMessage(Component.text("Usage: /structure <" + String.join("|",
structures.keySet())
47                      + "> ... (every command is explained in
plugins/ResettingStructures/commands.txt)", NamedTextColor.RED));
48              return;
49          }
50          command.execute(source, Arrays.copyOfRange(args, 1, args.length));
51      }
52
53      @Override
54      public Collection<String> suggest(CommandSourceStack source, String[] args) {
55          if (args.length <= 1) {
56              String prefix = args.length == 0 ? "" : args[0].toLowerCase(Locale.ROOT);
57              return structures.keySet().stream().filter(name -> name.startsWith(prefix)).toList();
58          }
59          Subcommand command = structures.get(args[0].toLowerCase(Locale.ROOT));
60          return command == null ? List.of() : command.suggest(source, Arrays.copyOfRange(args, 1,
args.length));
61      }
```

- **Line 43:** Paper calls this when someone runs `/structure ...`.
- **Line 44:** looks up the structure named by the first word (`args[0]`; arrays count from 0). If nothing was typed, or the name is unknown, `command` becomes `null`.
- **Lines 45–49:** no valid structure, so send a red usage message and stop. `String.join("|", structures.keySet())` builds `monument` from the registered names; with a mansion later it would show `monument|mansion`.
- **Line 50:** pass everything *after* the structure name to that structure's command. `Arrays.copyOfRange(args, 1, args.length)` drops the first word: `["monument", "status"]` becomes `["status"]`.
- **Lines 54–61:** tab completion. On the first word, suggest structure names starting with what's typed so far (`stream()` walks through them, `filter` keeps the matches, `toList` collects them). After that, ask that structure's own `suggest` for the rest.

MonumentCommand.java: /structure monument ...

Lines 29–94: the class, its constants and HELP

MonumentCommand.java

```
 34  public final class MonumentCommand implements StructureCommand.Subcommand {
 35
 36      static final String COMMAND = "/structure monument";
 37      /** The y-level an Ocean Monument is normally built at; setcenter reminds you when it differs.
*/
 38      private static final int USUAL_Y = 40;
 39      /** Every subcommand. MonumentCommandTest checks that each one is explained in HELP. */
 40      static final List<String> SUBCOMMANDS =
 41              List.of("status", "active", "check", "reset", "setcenter", "layout", "broadcast",
"reload");
 42
 43      /**
 44       * Every command with what it does, for commands.txt. Keep this in step with execute() below;
 45       * MonumentCommandTest fails when a subcommand is missing here.
 46       */
 47      public static final List<CommandHelp> HELP = List.of(
 48              new CommandHelp(COMMAND + " [status]",
 49                      "Shows the monument's location, what's built right now, its sponges, the Elder
Guardians"
 50                              + " (when the area is loaded), the last check, the reset mode, when the
next check"
 51                              + " is and whether broadcasts are on. The same information is in
status.txt."),
 52              new CommandHelp(COMMAND + " active [on|off]",
 53                      "Turns the checks and resets on or off; without on or off it switches. It's off
on a new"
 54                              + " server, so nothing is built until you've checked the location in
monuments.yml"
 55                              + " (or used setcenter) and turned it on. Saved, so it stays on or off
after restarts."
 56                              + " While off, the Monument Hoe rules are off too.",
 57                      List.of(COMMAND + " active on", COMMAND + " active off")),
```

- **Line 34:** `implements StructureCommand.Subcommand`, so the root command can hand work to it.
- **Line 36:** the command's beginning as a constant, used in every usage message and help text.
- **Line 38:** the y-level an Ocean Monument is normally built at. `setcenter` reminds you to double check when you pick a different one.
- **Lines 40–41:** the eight subcommands, for tab completion and the usage message. They're package-private (no `private`), so the test `MonumentCommandTest` can read them.
- **Lines 47–94 (`HELP`):** one `CommandHelp` per way of using the command: the usage as typed, an explanation, and sometimes examples (line 57). `Monuments` hands this list to `InfoFiles`, which writes it into `commands.txt`. The test `MonumentCommandTest` fails if a subcommand is missing from `HELP`, or explained without existing (lines 43–46).

Lines 96–126: picking the subcommand

MonumentCommand.java

```
 96      private final Monuments monuments;
 97
 98      public MonumentCommand(Monuments monuments) {
 99          this.monuments = monuments;
100      }
```

```
101
102     @Override
103     public void execute(CommandSourceStack source, String[] args) {
104         CommandSender sender = source.getSender();
105         String sub = args.length == 0 ? "status" : args[0].toLowerCase(Locale.ROOT);
106         switch (sub) {
107             case "status" -> status(sender);
108             case "active" -> active(sender, args);
109             case "check" -> run(sender, false, null, null);
110             case "reset" -> reset(sender, args);
111             case "setcenter" -> setCenter(sender, source.getLocation(), args);
112             case "layout" -> layout(sender, args);
113             case "broadcast" -> broadcast(sender, args);
114             case "reload" -> {
115                 if (monuments.reloadSettings()) {
116                     sender.sendMessage(Component.text("Reloaded monuments.yml and the structures.",
NamedTextColor.GREEN));
117                 } else {
118                     sender.sendMessage(Component.text("monuments.yml " + monuments.configProblem()
+ ". "
119                             + (monuments.isPaused() ? "The monument stays paused." : "Still using
the previous settings.")
120                             + " Details are in the console.", NamedTextColor.RED));
121                 }
122             }
123             default -> sender.sendMessage(Component.text("Usage: " + COMMAND + " <" +
String.join("|", SUBCOMMANDS)
124                     + ">. Every command is explained in plugins/ResettingStructures/commands.txt.",
NamedTextColor.RED));
125         }
126     }
```

- **Lines 98–100:** the constructor stores the `Monuments` object, so every subcommand can reach the settings, the timer, the current monument and so on.
- **Line 104:** who ran it, a player or the console. `CommandSender` covers both.
- **Line 105:** the first word, or `status` when nothing was typed. That's why `/structure monument` on its own shows the status.
- **Lines 106–125:** a `switch` that runs a different method per word:
  - **Line 108:** `active` needs the next word (`on`, `off` or nothing), so it gets `args`.
  - **Line 109:** `check` is a normal check, not forced; the two `null`s mean no variant and no seed.
  - **Line 110:** `reset` needs the remaining words for a variant or seed, so it gets `args`.
  - **Line 111:** `setcenter` gets the player's position (`source.getLocation()`, where the command was run) and the remaining words, which may be coordinates.
  - **Lines 114–122:** `reload`. If `reloadSettings()` succeeds, the reply is green. If not, it's red, names the problem, and says whether the monument stays paused or still uses the previous settings.
  - **Lines 123–124:** any other word gets a short usage message pointing to `commands.txt`.

Lines 128–155: tab completion

MonumentCommand.java

```
128     @Override
129     public Collection<String> suggest(CommandSourceStack source, String[] args) {
130         if (args.length <= 1) {
131             String prefix = args.length == 0 ? "" : args[0].toLowerCase(Locale.ROOT);
132             return SUBCOMMANDS.stream().filter(s -> s.startsWith(prefix)).toList();
133         }
134         if (args.length == 2 && args[0].equalsIgnoreCase("reset")) {
135             return Stream.concat(Stream.of(MonumentResetter.GENERATED),
monuments.structures().names().stream())
136                     .filter(s -> s.startsWith(args[1])).toList();
137         }
138         if (args.length == 2 && (args[0].equalsIgnoreCase("broadcast") ||
args[0].equalsIgnoreCase("active"))) {
139             return Stream.of("on", "off").filter(s ->
s.startsWith(args[1].toLowerCase(Locale.ROOT))).toList();
140         }
141         if (args.length == 2 && args[0].equalsIgnoreCase("layout")) {
142             return Stream.of("current", "random").filter(s ->
s.startsWith(args[1].toLowerCase(Locale.ROOT))).toList();
143         }
144         if (args.length >= 2 && args.length <= 4 && args[0].equalsIgnoreCase("setcenter")
145                 && source.getSender() instanceof Player) {
146             // Your own position (the nearest block corner), like vanilla commands suggest.
147             Location here = source.getLocation();
148             long[] xyz = {Math.round(here.getX()), here.getBlockY(), Math.round(here.getZ())};
149             return List.of(String.valueOf(xyz[args.length - 2]));
150         }
151         if (args.length == 3 && args[0].equalsIgnoreCase("layout")) {
152             return Stream.of("0", "0.5", "1").filter(s -> s.startsWith(args[2])).toList();
153         }
154         return List.of();
155     }
```

- **Lines 130–133:** first word: the eight subcommands, filtered on what's typed so far.
- **Lines 134–137:** second word after `reset`: `generated` plus every pre-built variant name. `Stream.concat` joins the two lists; `equalsIgnoreCase` compares text without caring about upper or lower case.
- **Lines 138–143:** second word after `broadcast` or `active`: `on` or `off`. After `layout`: `current` or `random`. `||` means "or".
- **Lines 144–150:** the coordinates after `setcenter`, for a player: your own position, rounded to the nearest block corner just like `setcenter` without coordinates would. The second word gets your x, the third your y and the fourth your z (`xyz[args.length - 2]` picks the right one), like vanilla commands such as `/tp` suggest.
- **Lines 151–153:** third word after `layout`: example newest chances.
- **Line 154:** nothing to suggest anywhere else, for example a seed, which can be any number.

Lines 157–176: broadcast

MonumentCommand.java

```
 157      /** /structure monument broadcast [on|off]: without an argument it toggles. Saved in
monuments.yml. */
 158      private void broadcast(CommandSender sender, String[] args) {
 159          boolean on;
 160          if (args.length < 2) {
 161              on = !monuments.settings().announce();
 162          } else if (args[1].equalsIgnoreCase("on")) {
 163              on = true;
 164          } else if (args[1].equalsIgnoreCase("off")) {
 165              on = false;
 166          } else {
 167              sender.sendMessage(Component.text("Usage: " + COMMAND + " broadcast [on|off]",
NamedTextColor.RED));
 168              return;
 169          }
 170          if (!monuments.setAnnounce(on)) {
 171              configProblem(sender);
 172              return;
 173          }
 174          sender.sendMessage(Component.text("Monument broadcasts are now " + (on ? "on" : "off") +
".",
 175                  on ? NamedTextColor.GREEN : NamedTextColor.YELLOW));
 176      }
```

- **Line 159:** a `boolean` holds `true` or `false`.
- **Lines 160–161:** no word after `broadcast`, so flip the current setting. `!` means "the opposite of".
- **Lines 162–165:** `on` or `off` sets it explicitly.
- **Lines 166–169:** anything else gets a usage message.
- **Lines 170–173:** save it. `setAnnounce` returns `false` when `monuments.yml` has a problem, because it then refuses to save over the broken file; the reply explains that (`configProblem`, lines 326–330).
- **Lines 174–175:** confirm, in green when on and yellow when off.

Lines 178–224: layout

MonumentCommand.java

```
 183      private void layout(CommandSender sender, String[] args) {
 184          if (args.length > 1 && args[1].equalsIgnoreCase("current")) {
 185              Optional<Built> built = monuments.currentMonument().get();
 186              if (built.isEmpty()) {
 187                  sender.sendMessage(Component.text("The plugin hasn't placed a monument yet.",
NamedTextColor.YELLOW));
 188              } else if (built.get() instanceof Built.PreBuilt(String variant)) {
 189                  sender.sendMessage(Component.text("The current monument is pre-built variant " +
variant
 190                          + ", it has no generated layout.", NamedTextColor.YELLOW));
 191              } else if (built.get() instanceof Built.Generated generated) {
 192                  printLayout(sender, generated.layout(), "Current layout");
 193              }
 194              return;
 195          }
```

- **`layout current`:**
  - **Line 185:** asks what's built. The answer is an `Optional`, which is empty when the plugin hasn't placed a monument yet (line 186).

- **Line 188:** `instanceof Built.PreBuilt(String variant)` checks "is it a pre-built one?" and takes its variant name out in one go (a record pattern).
- **Line 191:** for a generated one, `generated.layout()` rebuilds the layout from the saved seed and settings, and prints it.

MonumentCommand.java

```
197         long seed = ThreadLocalRandom.current().nextLong();
198         MazeSettings settings = monuments.settings().maze();
199         try {
200             if (args.length > 1 && !args[1].equalsIgnoreCase("random")) {
201                 seed = Long.parseLong(args[1]);
202             }
203             if (args.length > 2) {
204                 double chance = Double.parseDouble(args[2]);
205                 if (chance < 0 || chance > 1) {
206                     throw new NumberFormatException();
207                 }
208                 settings = settings.withNewestChance(chance);
209             }
210         } catch (NumberFormatException e) {
211             sender.sendMessage(Component.text("Usage: " + COMMAND + " layout [current|seed|random]
[newest-chance 0-1]",
212                     NamedTextColor.RED));
213             return;
214         }
215         printLayout(sender, MonumentLayout.generate(seed, settings), "Layout");
216     }
217
218     private void printLayout(CommandSender sender, MonumentLayout layout, String what) {
219         LayoutPrinter.print(layout).forEach(monuments.getLogger()::info);
220         if (!(sender instanceof ConsoleCommandSender)) {
221             sender.sendMessage(Component.text(what + " " + layout.seed() + " (" +
Math.round(layout.newestChance() * 100)
222                     + "% newest) printed in the server console.", NamedTextColor.GREEN));
223         }
224     }
```

- **Line 197:** start with a random seed. A `long` is a very large whole number, which is what seeds are.
- **Line 198:** start with the maze settings from `monuments.yml`.
- **Lines 200–202:** a second word that isn't `random` is read as the seed. `Long.parseLong` turns text into a number and throws a `NumberFormatException` if it isn't one.
- **Lines 203–209:** a third word is the newest chance, which must be between 0 and 1. A number outside that range deliberately throws the same error (line 206), so both mistakes end up in one place.
- **Lines 210–214:** `catch` handles that error: show the usage message and stop. This is how `try` / `catch` turns a crash into a friendly message.
- **Line 215:** generate the layout and print it.
- **Lines 218–224:** writes the map to the console. `monuments.getLogger()::info` means "for each line, call `info(line)` on the logger". A player also gets a short chat note with the seed and newest chance.

Lines 226–248: reset

MonumentCommand.java

```
226      /** /structure monument reset [variant|generated] [seed]: places a new monument right away. */
227      private void reset(CommandSender sender, String[] args) {
228          String variant = args.length > 1 ? args[1] : null;
229          if (MonumentResetter.GENERATED.equalsIgnoreCase(variant)) {
230              variant = MonumentResetter.GENERATED; // so "Generated" works too
231          }
232          Long seed = null;
233          if (args.length > 2) {
234              if (!MonumentResetter.GENERATED.equalsIgnoreCase(variant)) {
235                  // A seed only picks a generated layout; a pre-built variant has nothing to seed.
236                  sender.sendMessage(Component.text("A seed only works with " + COMMAND + " reset
generated <seed>."
237                          + " Nothing was reset.", NamedTextColor.RED));
238                  return;
239              }
240              try {
241                  seed = Long.parseLong(args[2]);
242              } catch (NumberFormatException e) {
243                  sender.sendMessage(Component.text("Usage: " + COMMAND + " reset generated [seed]",
NamedTextColor.RED));
244                  return;
245              }
246          }
247          run(sender, true, variant, seed);
248      }
```

- **Line 228:** the second word is a variant name or `generated`. Without one it's `null`, which means "follow `reset.mode`".
- **Lines 229–231:** `Generated` with a capital also means `generated`.
- **Line 232:** `Long` with a capital L can also be `null`, meaning "no seed given".
- **Lines 233–246:** a third word is a seed. Seeds only work with `generated`; for a pre-built variant the reply is red and nothing is reset (lines 234–239). A seed that isn't a number gets a usage message.
- **Line 247:** run a **forced** reset (`true`).

Lines 250–257: status

MonumentCommand.java

```
250      /** The status in color: a gray label and a colored value per line (the same lines as
status.txt). */
251      private void status(CommandSender sender) {
252          sender.sendMessage(Component.text(monuments.name(), NamedTextColor.DARK_AQUA,
TextDecoration.BOLD));
253          for (StatusLine line : monuments.status()) {
254              sender.sendMessage(Component.text(" " + line.label() + ": ", NamedTextColor.GRAY)
255                      .append(Component.text(line.value(), line.color())));
256          }
257      }
```

- **Lines 251–257:** the heading "Ocean Monument" in dark aqua, bold, then for each status line from `Monuments` (the same ones as in `status.txt`) the label in gray with the value in its own color. `.append` glues two pieces of chat text together.

Lines 259–292: active

This switches the monument on or off. It's built the same way as `broadcast`, so it works the same way: `active on`, `active off`, or just `active` to switch.

MonumentCommand.java

```
 259      /** /structure monument active [on|off]: without an argument it toggles. Saved in
monuments/state.yml. */
 260      private void active(CommandSender sender, String[] args) {
 261          boolean wasOn = monuments.scheduler().isActive();
 262          boolean on;
 263          if (args.length < 2) {
 264              on = !wasOn;
 265          } else if (args[1].equalsIgnoreCase("on")) {
 266              on = true;
 267          } else if (args[1].equalsIgnoreCase("off")) {
 268              on = false;
 269          } else {
 270              sender.sendMessage(Component.text("Usage: " + COMMAND + " active [on|off]",
NamedTextColor.RED));
 271              return;
 272          }
 273          if (on == wasOn) {
 274              sender.sendMessage(Component.text("The monument is already " + (on ? "active" :
"inactive") + ".",
 275                      NamedTextColor.YELLOW));
 276              return;
 277          }
 278          if (!monuments.setActive(on)) {
 279              sender.sendMessage(Component.text("Not turned on: monuments.yml " +
monuments.configProblem()
 280                      + ". Fix it and run " + COMMAND + " reload first.", NamedTextColor.RED));
 281              return;
 282          }
 283          if (on) {
 284              var settings = monuments.settings();
 285              sender.sendMessage(Component.text("The monument is now active at " +
settings.worldName() + " "
 286                      + settings.centerX() + " " + settings.centerY() + " " + settings.centerZ() + ".
Use " + COMMAND
 287                      + " status to see when the first check is.", NamedTextColor.GREEN));
 288          } else {
 289              sender.sendMessage(Component.text("The monument is now inactive: nothing is checked or
built until "
 290                      + COMMAND + " active on.", NamedTextColor.YELLOW));
 291          }
 292      }
```

- **Line 261:** whether it's switched on right now. This asks the timer directly, not `isActive()`: that one is also false while paused, and switching should follow what the admin last chose.
- **Lines 262–272:** work out the wanted state, exactly like `broadcast`: flip it when nothing follows, otherwise `on` or `off`, and a usage message for anything else.
- **Lines 273–277:** already in the wanted state? Just say so, in yellow, and change nothing. Without this, `active on` on an active monument would restart the timer for no reason.
- **Lines 278–282:** switch it. `setActive` refuses to turn **on** while `monuments.yml` is broken; the reply says why.
- **Lines 283–287, on:** a green reply naming the exact location it will build at, so a wrong location is noticed right away. The first check comes after a fresh random wait; `status` shows when.
- **Lines 288–291, off:** a yellow reply: nothing is checked or built until it's turned on again.

Lines 294–371: running a check, setcenter

MonumentCommand.java

```
294      private void run(CommandSender sender, boolean force, String variant, Long seed) {
295          info(sender, force ? "Resetting the monument..." : "Checking the monument...");
296          monuments.scheduler().runNow(force, variant, seed).thenAccept(result ->
sender.sendMessage(switch (result) {
297              case Result.NotCleared(int alive) -> Component.text(
298                      "Not cleared: " + alive + " Elder Guardian(s) alive. Use " + COMMAND + " reset
to force it.",
299                      NamedTextColor.YELLOW);
300              case Result.PlayersInside(int players) -> Component.text(
301                      "Cleared, but " + players + " player(s) are inside. Postponed; use " + COMMAND
+ " reset to force it.",
302                      NamedTextColor.YELLOW);
303              case Result.Placed(String placed) -> Component.text("Placed " + placed + ".",
NamedTextColor.GREEN);
304              case Result.Failed(String reason) -> Component.text("Failed: " + reason,
NamedTextColor.RED);
305          }));
306      }
```

- **Lines 294–306, check or reset:**
  - **Line 295:** an immediate gray "Checking..." or "Resetting...", because the actual work can take a while.
  - **Line 296:** asks the timer to run a check now, so it can start a fresh random wait afterwards. The result isn't ready immediately (loading chunks and building take several ticks), so `runNow` returns a **`CompletableFuture`**, a "promise" of a result later. `.thenAccept(result -> ...)` says what to do once it arrives: send the reply.
  - **Lines 297–304:** one message per outcome. The compiler checks that all four outcomes are handled, because `Result` is a sealed type with exactly these four kinds.

MonumentCommand.java

```
308      /**
309       * /structure monument setcenter [x y z]: without coordinates the player's own position, with
310       * coordinates exactly those (in the player's world, or from the console in the configured
world).
311       */
312      private void setCenter(CommandSender sender, Location location, String[] args) {
313          World world;
314          int x;
315          int y;
316          int z;
317          if (args.length == 1) {
318              if (!(sender instanceof Player)) {
319                  sender.sendMessage(Component.text("From the console, give coordinates: " + COMMAND
320                          + " setcenter <x> <y> <z>", NamedTextColor.RED));
321                  return;
322              }
323              // The datapack's marker stood on the corner where the 4 center blocks meet, so round
to the nearest corner.
324              world = location.getWorld();
325              x = (int) Math.round(location.getX());
326              y = location.getBlockY();
327              z = (int) Math.round(location.getZ());
328          } else if (args.length == 4) {
329              try {
330                  x = Integer.parseInt(args[1]);
331                  y = Integer.parseInt(args[2]);
332                  z = Integer.parseInt(args[3]);
```

```
333             } catch (NumberFormatException e) {
334                 sender.sendMessage(Component.text("Coordinates must be whole numbers: " + COMMAND
335                         + " setcenter <x> <y> <z>", NamedTextColor.RED));
336                 return;
337             }
338             world = sender instanceof Player
339                     ? location.getWorld()
340                     : monuments.getServer().getWorld(monuments.settings().worldName());
341             if (world == null) {
342                 sender.sendMessage(Component.text("World '" + monuments.settings().worldName() + "'
is not loaded.",
343                         NamedTextColor.RED));
344                 return;
345             }
346         } else {
347             sender.sendMessage(Component.text("Usage: " + COMMAND + " setcenter [<x> <y> <z>]",
NamedTextColor.RED));
348             return;
349         }
350
351         if (!monuments.setCenter(world, x, y, z)) {
352             configProblem(sender);
353             return;
354         }
355         sender.sendMessage(Component.text("Monument center set to " + x + " " + y + " " + z + " in
"
356                 + world.getName() + ".", NamedTextColor.GREEN));
357         if (y != USUAL_Y) {
358             sender.sendMessage(Component.text("Double check your y-level. An Ocean Monument is
usually set at y="
359                     + USUAL_Y + ".", NamedTextColor.GOLD));
360         }
361     }
```

`setcenter` works in two ways: on its own it uses where you're standing, and with three numbers it uses exactly those.

- **Lines 313–316:** the four values to fill in, by whichever way is used.
- **Lines 317–327, without coordinates** (`args` only holds `setcenter`):
  - **Lines 318–322:** only a player has a position to use, so the console is told to give coordinates.
  - **Lines 324–327:** the player's world, with x and z rounded to the **nearest block corner**, matching the datapack's marker, which stood exactly on the corner where the 4 center blocks meet. `(int)` turns the rounded number into a whole number. y is the height of the block the player's feet are in, which is the level the rooms start from.
- **Lines 328–345, with coordinates** (`setcenter` plus three words):
  - **Lines 329–337:** read the three numbers. Something that isn't a whole number (like `abc` or `40.5`) gets a red message instead. Whole numbers are what you want: the center is a block corner.
  - **Lines 338–340:** a player sets it in the world they're in; the console has no world of its own, so it keeps the world from `monuments.yml`.
  - **Lines 341–344:** that world must be loaded.
- **Lines 346–348:** any other number of words (one or two coordinates, or four) gets the usage message.
- **Lines 351–354:** save it. If `monuments.yml` is broken, nothing is saved, and the reply says why.
- **Lines 355–356:** confirm, including the world, so a center set in the wrong world is noticed right away.
- **Lines 357–360:** a y-level other than 40 is allowed, but gets a gold reminder: "Double check your y-level. An Ocean Monument is usually set at y=40." This catches the most likely mistake: standing one block too high

or too low.

MonumentCommand.java

```
363      /** monuments.yml has a mistake, so nothing is saved: saving would overwrite the file being
fixed. */
364      private void configProblem(CommandSender sender) {
365          sender.sendMessage(Component.text("Not saved: monuments.yml " + monuments.configProblem()
366                  + ". Fix it and run " + COMMAND + " reload first.", NamedTextColor.RED));
367      }
368
369      private static void info(CommandSender sender, String message) {
370          sender.sendMessage(Component.text(message, NamedTextColor.GRAY));
371      }
```

- **Lines 363–367:** the shared red "Not saved…" message, used by `broadcast` and `setcenter`.
- **Lines 369–371:** sends a gray message.

Step 4: how a reset happens

The files in this step work together. Here's the whole journey of one check:

```
ResetScheduler        only while active: waits a random interval (saved in state.yml)
      │  time's up
      ▼
MonumentResetter      ── MonumentArena.loadChunks()   keep the area loaded, wait for mobs to load
      │                 Elder Guardians alive?  → "not cleared", stop
      │                 players inside?          → "postponed", stop
      │                 otherwise: place a new monument
      │                   pre-built → StructureLibrary (the 4 quadrant templates)
      │                   generated → MonumentBuilder (step 6)
      │                 CurrentMonument.set(…)   remember what's built + count sponges
      │               ── MonumentArena.releaseChunks()
      ▼
ResetScheduler        picks the next interval, saves state.yml, updates status.txt
```

Two small helpers in `util` are used along the way: `Durations` (times like `1d`, `10s`, `5t`) and `Weighted` (weighted random picks).

util/Durations.java: times like 1d, 10s, 5t

Durations.java

```
 9  public final class Durations {
10
11      public static final long TICKS_PER_SECOND = 20;
12      public static final long TICKS_PER_DAY = 24_000;
13
14      private Durations() {
15      }
16
17      public static long parseTicks(String text) {
18          String value = text.trim().toLowerCase(Locale.ROOT);
19          if (value.isEmpty()) {
20              throw new IllegalArgumentException("Empty duration");
21          }
22
23          char unit = value.charAt(value.length() - 1);
24          long multiplier = 1;
25          String number = value;
26          if (Character.isLetter(unit)) {
27              multiplier = switch (unit) {
28                  case 't' -> 1;
29                  case 's' -> TICKS_PER_SECOND;
30                  case 'd' -> TICKS_PER_DAY;
31                  default -> throw new IllegalArgumentException(
32                          "Unknown unit '" + unit + "' in '" + text + "', use t, s or d");
33              };
34              number = value.substring(0, value.length() - 1);
35          }
36
37          double amount;
38          try {
39              amount = Double.parseDouble(number);
40          } catch (NumberFormatException e) {
41              throw new IllegalArgumentException("Invalid duration '" + text + "', use e.g. 20t, 10s
or 2d");
42          }
```

```
43          if (amount <= 0) {
44              throw new IllegalArgumentException("Duration must be positive: '" + text + "'");
45          }
46          return Math.max(1, Math.round(amount * multiplier));
47      }
```

- **Lines 11–12:** Minecraft runs at 20 ticks per second, and a Minecraft day is 24,000 ticks. The `_` in `24_000` only makes it easier to read.
- **Lines 14–15:** a `private` constructor means nobody can make a `Durations` object. It's only a toolbox of `static` methods.
- **Line 18:** removes stray spaces and makes `2D` work like `2d`.
- **Lines 19–21:** empty text is an error. "Throwing" an `IllegalArgumentException` reports it to whoever called this method; `MonumentConfig` catches it and logs "Skipping ... entry".
- **Line 23:** the last character: the unit. A `char` is a single character.
- **Lines 26–35:** if that last character is a letter, look up how many ticks the unit is worth and cut the letter off. An unknown letter, such as `m`, is an error with a hint. Without a letter, the number is in ticks.
- **Lines 37–42:** turn the number part into a decimal, so `1.5d` also works.
- **Lines 43–45:** zero or negative isn't allowed.
- **Line 46:** multiply and round to whole ticks, never less than 1.

Durations.java

```
49      /** Human readable real time for a number of ticks, e.g. {@code 1h 4m 30s}. */
50      public static String format(long ticks) {
51          long seconds = Math.max(0, ticks) / TICKS_PER_SECOND;
52          long hours = seconds / 3600;
53          long minutes = (seconds % 3600) / 60;
54          long secs = seconds % 60;
55
56          StringBuilder out = new StringBuilder();
57          if (hours > 0) {
58              out.append(hours).append("h ");
59          }
60          if (hours > 0 || minutes > 0) {
61              out.append(minutes).append("m ");
62          }
63          return out.append(secs).append('s').toString();
64      }
```

- **The reverse direction:** ticks become readable text like `1h 4m 30s`, used in the logs and the status.
- **Lines 51–54:** split into hours, minutes and seconds. `/` is whole-number division and `%` is the remainder: 3,725 seconds is 1 hour, `3725 % 3600 = 125` seconds left over, which is 2 minutes and 5 seconds.
- **Lines 56–63:** a `StringBuilder` builds up text piece by piece. Hours only appear when there are any, so you get `45s` or `2m 5s` instead of `0h 0m 45s`.

util/Weighted.java: weighted random picks

Weighted.java

```
 8  public record Weighted<T>(T value, int weight) {
 9
10      public Weighted {
11          if (weight < 0) {
12              throw new IllegalArgumentException("Weight can't be negative: " + weight);
13          }
14      }
15
16      /** Picks one value proportional to its weight. Empty when nothing has a weight above 0. */
17      public static <T> Optional<T> pick(List<Weighted<T>> entries, RandomGenerator random) {
18          int total = entries.stream().mapToInt(Weighted::weight).sum();
19          if (total <= 0) {
20              return Optional.empty();
21          }
22
23          int roll = random.nextInt(total);
24          for (Weighted<T> entry : entries) {
25              roll -= entry.weight();
26              if (roll < 0) {
27                  return Optional.of(entry.value());
28              }
29          }
30          throw new IllegalStateException("unreachable");
31      }
```

- **Line 8:** a record holding a value and its weight. `<T>` makes it **generic**: `T` can be any type. That's how the same class serves the intervals (`Weighted<Long>`) and the variants (`Weighted<String>`).
- **Lines 10–14:** a check that runs every time one is created: no negative weights.
- **Line 18:** adds up all the weights. For the reset intervals that's 30+30+30+10 = 100.
- **Lines 19–21:** if everything weighs 0, there's nothing to pick, so the result is an empty `Optional`.
- **Line 23:** a random number from 0 up to (not including) the total, so 0–99.
- **Lines 24–29:** walk through the entries, subtracting each weight, until the number drops below 0. With 30/30/30/10, a roll of 0–29 lands on 1 day, 30–59 on 2 days, 60–89 on 3 days and 90–99 on 72 days. This is exactly your datapack's `random value 1..100` with its `matches 1..30` ranges, but for any list.
- **Line 30:** the loop always returns before it ends; this line is only there because Java requires every path to end with a `return` or a `throw`.

ResetScheduler.java: the timer

ResetScheduler.java

```
 17  /**
 18   * The randomized check loop from schedule.mcfunction. Unlike the datapack, the next check time is
 19   * saved (in world game ticks), so a server restart doesn't restart the 1-3 day wait from zero.
 20   *
 21   * <p>On a new server nothing runs until /structure monument active on, so a monument is never
built
 22   * at a location nobody checked. Whether it's on is saved in state.yml as well.
 23   */
 24  public final class ResetScheduler {
 25
 26      /** Retry delay after a failed check, e.g. when the area's entities didn't load in time. */
 27      private static final long RETRY_TICKS = 60 * Durations.TICKS_PER_SECOND;
 28      /** Delay for a check that became due while the server was offline. */
 29      private static final long OVERDUE_TICKS = 10 * Durations.TICKS_PER_SECOND;
 30
 31      private final Monuments monuments;
 32      private final File stateFile;
 33      private final Random random = new Random();
 34
 35      private BukkitTask task;
 36      /** Set with /structure monument active on|off. While off there are no checks or resets. */
 37      private boolean active;
 38      private String stateWorld;
 39      private long nextCheckAt = -1;
 40
 41      public ResetScheduler(Monuments monuments) {
 42          this.monuments = monuments;
 43          this.stateFile = new File(monuments.folder(), "state.yml");
 44          loadState();
 45      }
```

- **Lines 21–22:** the timer is also where "active" lives: on a new server it doesn't run until `/structure monument active on`.
- **Line 27:** after a failed check, try again in 1 minute.
- **Line 29:** a check that came due while the server was off runs 10 seconds after startup.
- **Lines 31–33:** the monument, the `state.yml` file, and a random number generator for picking intervals.
- **Line 35:** the pending timer. A `BukkitTask` is something Paper will run later, and it can be cancelled. When nothing is planned it's `null`.
- **Lines 36–39:** what's saved in `state.yml`: whether the monument is switched on, the world, and the **game tick** at which the next check happens. `-1` means "not known yet".
- **Lines 41–45:** the constructor sets the file to `monuments/state.yml` and reads it straight away.

ResetScheduler.java

```
 47      /** (Re)starts the timer, continuing the saved countdown when it belongs to the current world.
*/
 48      public void start() {
 49          stop();
 50          if (!active) {
 51              return;
 52          }
 53          World world = world();
 54          if (world == null) {
 55              monuments.getLogger().warning("World '" + monuments.settings().worldName()
 56                      + "' is not loaded, the monument won't reset until it is (then run /structure
monument reload).");
```

```
57              return;
58          }
59
60          long now = world.getGameTime();
61          if (!world.getName().equals(stateWorld) || nextCheckAt < 0) {
62              schedule(pick(monuments.settings().intervals()));
63          } else if (nextCheckAt > now) {
64              schedule(nextCheckAt - now);
65          } else {
66              schedule(OVERDUE_TICKS);
67          }
68      }
69
70      public void stop() {
71          if (task != null) {
72              task.cancel();
73              task = null;
74          }
75      }
76
77      public boolean isActive() {
78          return active;
79      }
80
81      /**
82       * Turns the checks on or off (saved in state.yml, so it lasts through restarts). Turning off
83       * forgets the countdown, so turning on again starts a fresh random wait.
84       */
85      public void setActive(boolean active) {
86          this.active = active;
87          if (active) {
88              saveState();
89              start();
90          } else {
91              stop();
92              nextCheckAt = -1;
93              saveState();
94          }
95      }
96
97      /** Runs a check right away (from a command) and then starts a fresh random interval. */
98      public CompletableFuture<Result> runNow(boolean force, String variant, Long seed) {
99          if (monuments.resetter().isRunning()) {
100             return CompletableFuture.completedFuture(new Result.Failed("A check is already
running."));
101         }
102         stop();
103         return runCycle(force, variant, seed);
104     }
105
106     /** Ticks until the next check, or -1 when nothing is scheduled. */
107     public long ticksUntilNextCheck() {
108         World world = world();
109         if (task == null || world == null) {
110             return -1;
111         }
112         return Math.max(0, nextCheckAt - world.getGameTime());
113     }
```

- **Lines 48–68,** `start()`**:** called on startup and on every reload.
  - **Line 49:** first cancel any running timer, so there are never two at once.

- **Lines 50–52:** switched off? Then stop here: nothing gets planned. Because this check sits inside `start()` itself, every caller (startup, `reload`, `setcenter`) automatically respects it.
- **Lines 53–58:** if the monument's world isn't loaded, warn and stop.
- **Line 60:** the world's **game time**, a tick counter that only goes up and only counts while the server runs. That's why the wait "pauses" while the server is off, just like your datapack's `/schedule`.
- **Lines 61–67:** three cases. No saved time (or it belongs to another world): pick a fresh random interval. The saved time is still ahead: wait the remaining ticks, which keeps the countdown going across restarts. The saved time has passed: check in 10 seconds.

- **Lines 70–75:** cancels the pending timer, if there is one.
- **Lines 77–95, switched on or off** (`/structure monument active`):
  - **Lines 87–89, on:** save it, then start the timer, which picks a fresh random wait.
  - **Lines 90–94, off:** stop the timer and forget the countdown (`-1`) before saving. Otherwise turning it on again later would find an old, long-overdue countdown and check just 10 seconds later.
- **Lines 97–104, `runNow`:** for `check` and `reset`. If a check is already busy, reply "already running". Otherwise cancel the waiting timer and check now. `completedFuture` is a promise that's already fulfilled.
- **Lines 106–113:** for the status: how many ticks are left, or `-1` for "nothing planned".

ResetScheduler.java

```
115     private CompletableFuture<Result> runCycle(boolean force, String variant, Long seed) {
116         return monuments.resetter().run(force, variant, seed).whenComplete((result, error) -> {
117             if (!monuments.isEnabled() || !monuments.isActive()) {
118                 return;
119             }
120             if (result instanceof Result.Failed(String reason)) {
121                 monuments.getLogger().warning("Monument check failed: " + reason + " Retrying in "
122                         + Durations.format(RETRY_TICKS) + ".");
123                 schedule(RETRY_TICKS);
124             } else if (result instanceof Result.PlayersInside(int players)) {
125                 monuments.getLogger().info("Monument is cleared but " + players
126                         + " player(s) are inside, postponing the reset.");
127                 schedule(pick(monuments.settings().occupiedIntervals()));
128             } else {
129                 schedule(pick(monuments.settings().intervals()));
130             }
131             // After scheduling, so the status shows the new next check too.
132             monuments.recordCheck(result);
133         });
134     }
135 
136     private void schedule(long delayTicks) {
137         stop();
138         World world = world();
139         if (world == null) {
140             return;
141         }
142         nextCheckAt = world.getGameTime() + delayTicks;
143         stateWorld = world.getName();
144         saveState();
145         task = monuments.getServer().getScheduler().runTaskLater(monuments.plugin(), () -> {
146             task = null;
147             runCycle(false, null, null);
148         }, delayTicks);
149         monuments.getLogger().info("Next monument check in " + Durations.format(delayTicks)
150                 + " (" + delayTicks + " ticks).");
151     }
```

- **Lines 115–134, one check plus planning the next one:**
  - **Line 116:** let the resetter do the check. `whenComplete` runs once it's finished, which can be many ticks later for a generated build.
  - **Lines 117–119:** don't plan anything while the server is shutting down, or the monument is paused or was switched off in the meantime.
  - **Lines 120–130:** choose the next wait based on the outcome: failed → retry in a minute; players inside → the short `occupied-intervals`; anything else → a normal interval.
  - **Line 132:** report the outcome to `Monuments` for the status's "Last check" line and `status.txt`. This happens after scheduling, so the status also shows the new next check.
- **Lines 136–151, `schedule`:**
  - **Lines 142–144:** work out the game tick at which the next check happens, and save it right away, so a crash doesn't lose the countdown.
  - **Lines 145–148:** ask Paper to run a check after `delayTicks`. The part between `() -> {` and `}` is the code that runs later: forget the old task, then run a normal, unforced check.
  - **Line 149:** log it, e.g. "Next monument check in 1h 4m 30s".

ResetScheduler.java

```
153      private long pick(List<Weighted<Long>> intervals) {
154          return Weighted.pick(intervals, random).orElse(Durations.TICKS_PER_DAY);
155      }
156
157      private World world() {
158          return monuments.getServer().getWorld(monuments.settings().worldName());
159      }
160
161      private void loadState() {
162          if (!stateFile.exists()) {
163              return;
164          }
165          YamlConfiguration state = YamlConfiguration.loadConfiguration(stateFile);
166          stateWorld = state.getString("world");
167          nextCheckAt = state.getLong("next-check-at", -1);
168          // A state.yml from before "active" existed already had a running countdown: keep it
running.
169          active = state.getBoolean("active", nextCheckAt >= 0);
170      }
171
172      private void saveState() {
173          YamlConfiguration state = new YamlConfiguration();
174          state.set("active", active);
175          state.set("world", stateWorld);
176          state.set("next-check-at", nextCheckAt);
177          try {
178              state.save(stateFile);
179          } catch (IOException e) {
180              monuments.getLogger().log(Level.WARNING, "Could not save " + stateFile.getName(), e);
181          }
182      }
```

- **Lines 153–155:** a weighted random interval. 1 day is a safety net that can't actually happen, because `MonumentConfig` always provides a usable list.
- **Lines 157–159:** the monument's world, looked up by name.
- **Lines 161–182:** read and write `state.yml`, which holds three values: `active`, `world` and `next-check-at`.

- **Line 162:** no `state.yml` (a new server): everything keeps its starting value, so the monument is switched off.
- **Line 169:** an older `state.yml` without `active` but with a countdown came from a server where the monument was already running, so it counts as on. Your own server keeps running after the update without you doing anything.

MonumentArena.java: the area around the monument

MonumentArena.java

```
19  public final class MonumentArena {
20
21      /** How long to wait for the area's entities to load before giving up on a check. */
22      private static final int ENTITY_LOAD_TIMEOUT_TICKS = 200;
23
24      /** The monument box, in blocks from the center: its four 29-wide quadrants and 22 levels. */
25      private static final int BOX_WEST = -29;
26      private static final int BOX_EAST = 28;
27      private static final int BOX_BOTTOM = -1;
28      private static final int BOX_TOP = 20;
29
30      private final Plugin plugin;
31      private final World world;
32      private final Location center;
33      private final double radius;
34      private final List<ChunkPos> chunks = new ArrayList<>();
35
36      private record ChunkPos(int x, int z) {
37      }
38
39      public MonumentArena(Plugin plugin, World world, int x, int y, int z, double radius) {
40          this.plugin = plugin;
41          this.world = world;
42          this.center = new Location(world, x, y, z);
43          this.radius = radius;
44
45          // The chunks covering the radius and the whole monument box, whichever reaches further.
46          int r = (int) Math.ceil(radius);
47          int west = Math.min(x - r, x + BOX_WEST);
48          int east = Math.max(x + r, x + BOX_EAST);
49          int north = Math.min(z - r, z + BOX_WEST);
50          int south = Math.max(z + r, z + BOX_EAST);
51          for (int cx = west >> 4; cx <= east >> 4; cx++) {
52              for (int cz = north >> 4; cz <= south >> 4; cz++) {
53                  chunks.add(new ChunkPos(cx, cz));
54              }
55          }
56      }
```

- **Line 22:** wait at most 200 ticks (10 seconds) for the area's mobs to load.
- **Lines 25–28:** the monument box, in blocks from the center: 29 west/north to 28 east/south, 1 below to 20 above. That's the 58×22×58 blocks of the four quadrants.
- **Lines 30–34:** the world, the center, the radius from `check-radius`, and the list of chunks to keep loaded.
- **Lines 36–37:** a tiny record for one chunk's position.
- **Lines 45–55:** work out every chunk covering the radius **and** the whole monument box, whichever reaches further. `>> 4` divides by 16 (rounding down, correct for negative coordinates too), turning a block coordinate into a chunk coordinate. This replaces your datapack's hard-coded `forceload add 192 -289 255 -352`.

MonumentArena.java

```
58      public World world() {
59          return world;
60      }
61
62      /** The block corner the four quadrants are placed around. Returns a copy. */
63      public Location center() {
64          return center.clone();
65      }
66
67      /**
68       * Whether a location counts as "at the monument": within check-radius of the center, or
anywhere
69       * in the monument box. The box matters for its far corners, which are more than 40 blocks
away.
70       * Used for counting Elder Guardians and players.
71       */
72      public boolean contains(Location location) {
73          return location.getWorld() == world && isAround(location.getX() - center.getX(),
74                  location.getY() - center.getY(), location.getZ() - center.getZ(), radius);
75      }
76
77      /** {@link #contains}, by the offset from the center. */
78      static boolean isAround(double dx, double dy, double dz, double radius) {
79          return dx * dx + dy * dy + dz * dz <= radius * radius
80                  || inBox((int) Math.floor(dx), (int) Math.floor(dy), (int) Math.floor(dz));
81      }
82
83      /** Living entities of the given type at the monument (see {@link #contains}). Only sees loaded
chunks. */
84      public <T extends Entity> List<T> entities(Class<T> type) {
85          // Search a box big enough for both the radius and the monument box, then keep the ones
that count.
86          double horizontal = Math.max(radius, -BOX_WEST + 1);
87          double vertical = Math.max(radius, BOX_TOP + 1);
88          return world.getNearbyEntitiesByType(type, center, horizontal, vertical,
horizontal).stream()
89                  .filter(e -> !e.isDead() && contains(e.getLocation()))
90                  .toList();
91      }
92
93      /**
94       * Whether a location is inside the monument itself: the 58x22x58 box of its four quadrants,
95       * which also holds the wings and the top room.
96       */
97      public boolean inMonument(Location location) {
98          return location.getWorld() == world && inBox(location.getBlockX() - center.getBlockX(),
99                  location.getBlockY() - center.getBlockY(), location.getBlockZ() -
center.getBlockZ());
100     }
101
102     /** Whether a block offset from the center is in the monument box. */
103     static boolean inBox(int dx, int dy, int dz) {
104         return dx >= BOX_WEST && dx <= BOX_EAST && dy >= BOX_BOTTOM && dy <= BOX_TOP
105                 && dz >= BOX_WEST && dz <= BOX_EAST;
106     }
```

- **Lines 63–65:** the center. It hands out a *copy* (`clone()`), because a `Location` can be changed, and the arena's own center must stay put.
- **Lines 72–81, "at the monument":** within the radius (a sphere around the center) **or** anywhere in the monument box. The box matters for the far corners, which are more than 40 blocks from the center (about

40.3 at the bottom corners, 45.6 at the top). Comparing the squared distance with `radius * radius` avoids a slow square root and gives the same answer.

- **Lines 84–91, all living entities of one kind at the monument**, e.g. `entities(ElderGuardian.class)` or `entities(Player.class)`. This replaces the datapack's `@e[type=elder_guardian,distance=..40]`. `<T extends Entity>` means it works for any entity type. Line 88 asks Paper for everything in a box big enough for both the radius and the monument box; line 89 keeps only the living ones that really count.
- **Lines 97–106, inside the monument itself:** the exact box, used for the hoe rules.

MonumentArena.java

```
108     /** Counts the sponges (wet and dry) in the monument (see {@link #inMonument}). Its chunks must
be loaded. */
109     public int countSponges() {
110         int count = 0;
111         int cx = center.getBlockX();
112         int cy = center.getBlockY();
113         int cz = center.getBlockZ();
114         for (int x = cx + BOX_WEST; x <= cx + BOX_EAST; x++) {
115             for (int y = cy + BOX_BOTTOM; y <= cy + BOX_TOP; y++) {
116                 for (int z = cz + BOX_WEST; z <= cz + BOX_EAST; z++) {
117                     Material type = world.getType(x, y, z);
118                     if (type == Material.WET_SPONGE || type == Material.SPONGE) {
119                         count++;
120                     }
121                 }
122             }
123         }
124         return count;
125     }
```

- **Counts the sponges for the hoe's durability:** every block in the box (about 74,000) is checked for wet or dry sponge. `count++` adds one. The chunks have to be loaded, which they are during a reset.

MonumentArena.java

```
127     /**
128      * Keeps the area's chunks loaded with plugin tickets (the datapack's {@code forceload add}) and
129      * completes once their entities are loaded too, so the Elder Guardian count can be trusted.
130      */
131     public CompletableFuture<Void> loadChunks() {
132         for (ChunkPos c : chunks) {
133             world.addPluginChunkTicket(c.x(), c.z(), plugin);
134         }
135
136         CompletableFuture<Void> loaded = new CompletableFuture<>();
137         BukkitTask[] task = new BukkitTask[1];
138         int[] waited = {0};
139         task[0] = plugin.getServer().getScheduler().runTaskTimer(plugin, () -> {
140             if (allEntitiesLoaded()) {
141                 task[0].cancel();
142                 loaded.complete(null);
143             } else if (++waited[0] >= ENTITY_LOAD_TIMEOUT_TICKS) {
144                 task[0].cancel();
145                 loaded.completeExceptionally(new TimeoutException(
146                         "Monument chunks did not finish loading entities in time"));
147             }
148         }, 1L, 1L);
149         return loaded;
150     }
151
```

```
152     /** Removes the plugin chunk tickets again (the datapack's {@code forceload remove}). */
153     public void releaseChunks() {
154         for (ChunkPos c : chunks) {
155             world.removePluginChunkTicket(c.x(), c.z(), plugin);
156         }
157     }
158 
159     private boolean allEntitiesLoaded() {
160         for (ChunkPos c : chunks) {
161             if (!world.isChunkLoaded(c.x(), c.z()) || !world.getChunkAt(c.x(),
c.z()).isEntitiesLoaded()) {
162                 return false;
163             }
164         }
165         return true;
166     }
```

- **Lines 131–150, loading the area** (replaces the datapack's `forceload add` followed by a 5-tick wait):
  - **Lines 132–134:** a **plugin chunk ticket** keeps a chunk loaded for as long as the plugin holds it, the same idea as `forceload`.
  - **Line 136:** a promise that's fulfilled once everything is loaded.
  - **Lines 137–138:** small arrays of one element. It's a Java workaround: the code inside the timer can't change ordinary local variables, but it can change what's *inside* an array.
  - **Lines 139–148:** a timer that runs every tick. Once every chunk *and its mobs* are loaded, stop and fulfil the promise. This matters because mobs load slightly after blocks; counting too early would find "0 Elder Guardians" and reset an uncleared monument. After 200 ticks without success, give up with an error; the check then fails safely and is retried a minute later.
- **Lines 153–157:** let go of the chunks again (the datapack's `forceload remove`).
- **Lines 159–166:** true only when every chunk and its mobs are loaded.

StructureLibrary.java: the .nbt templates

StructureLibrary.java

```
24  public final class StructureLibrary {
25
26      private static final String PRE_GEN = "structures/monument/pre-gen/";
27      private static final String MAZE = "structures/monument/maze/";
28
29      /** Every maze template, by its path below structures/monument/maze/. */
30      public static final List<String> MAZE_TEMPLATES = Stream.concat(
31              Stream.of("monument_empty_nw", "monument_empty_ne", "monument_empty_sw",
"monument_empty_se",
32                      "roof_open", "room/wing_open", "room/wing_closed", "room/penthouse"),
33              Arrays.stream(RoomType.values()).map(RoomType::template)).toList();
34
35      private final Map<String, Structure> mazeTemplates = new LinkedHashMap<>();
36
37      /** One pre-built monument: four 29x29 quadrants placed around the center. */
38      public record MonumentVariant(String name, Structure se, Structure sw, Structure ne, Structure
nw) {
39      }
40
41      private final JavaPlugin plugin;
42      private final Map<String, MonumentVariant> variants = new LinkedHashMap<>();
43      private List<Weighted<MonumentVariant>> weighted = List.of();
44
45      public StructureLibrary(JavaPlugin plugin) {
46          this.plugin = plugin;
47      }
```

- **Lines 26–27:** where the templates are inside the jar (from `build.gradle.kts`).
- **Lines 30–33:** the full list of 19 maze templates: the 8 fixed pieces, plus one per room type from `RoomType` (step 5), so a new room type is loaded automatically.
- **Line 35:** the loaded maze templates, by name. A `Structure` is Bukkit's in-memory version of an `.nbt` file, ready to place.
- **Lines 38–39:** one pre-built monument: its name and its four quadrants.
- **Lines 42–43:** the pre-built monuments, by name (for `reset monument_03`) and with their weights (for random picks).

StructureLibrary.java

```
49      /** (Re)loads every configured variant. Variants with missing files are skipped with a warning.
*/
50      public void load(List<Weighted<String>> configured) {
51          variants.clear();
52          List<Weighted<MonumentVariant>> loaded = new ArrayList<>();
53          for (Weighted<String> entry : configured) {
54              String name = entry.value();
55              try {
56                  MonumentVariant variant = new MonumentVariant(name,
57                          read(name + "_se"), read(name + "_sw"), read(name + "_ne"), read(name +
"_nw"));
58                  variants.put(name, variant);
59                  loaded.add(new Weighted<>(variant, entry.weight()));
60              } catch (IOException e) {
61                  plugin.getLogger().warning("Skipping variant " + name + ": " + e.getMessage());
62              }
63          }
64          weighted = List.copyOf(loaded);
65          plugin.getLogger().info("Loaded " + variants.size() + " monument variants.");
66
67          mazeTemplates.clear();
68          for (String template : MAZE_TEMPLATES) {
69              try {
70                  mazeTemplates.put(template, readFile(MAZE + template + ".nbt"));
71              } catch (IOException e) {
72                  plugin.getLogger().warning("Generated monuments are unavailable: " +
e.getMessage());
73                  mazeTemplates.clear();
74                  return;
75              }
76          }
77          plugin.getLogger().info("Loaded " + mazeTemplates.size() + " templates for generated
monuments.");
78      }
```

- **Called on startup and on every reload.** It loads all files once, so placing them later is instant.
- **Lines 53–63:** for each variant in `monuments.yml`, load its four quadrant files. If one is missing, that variant is skipped with a warning and the others still work.
- **Lines 67–77:** load all 19 maze templates. If even one is missing, generated monuments are switched off as a whole; a half-built monument with holes would be worse. The status then shows "(generated templates missing)".

StructureLibrary.java

```
80      /** Whether every template for generated monuments is loaded. */
81      public boolean mazeReady() {
82          return mazeTemplates.size() == MAZE_TEMPLATES.size();
83      }
84
85      /** A template for generated monuments, by its path below structures/monument/maze/. */
86      public Structure maze(String template) {
87          Structure structure = mazeTemplates.get(template);
88          if (structure == null) {
89              throw new IllegalStateException("Template " + template + " is not loaded");
90          }
91          return structure;
92      }
93
94      public Optional<MonumentVariant> get(String name) {
95          return Optional.ofNullable(variants.get(name));
```

```
    }

    public Set<String> names() {
        return variants.keySet();
    }

    public List<Weighted<MonumentVariant>> weighted() {
        return weighted;
    }

    private Structure read(String file) throws IOException {
        return readFile(PRE_GEN + file + ".nbt");
    }

    private Structure readFile(String path) throws IOException {
        try (InputStream in = plugin.getResource(path)) {
            if (in == null) {
                throw new IOException("missing " + path);
            }
            return Bukkit.getStructureManager().loadStructure(in);
        }
    }
```

- **Lines 80–104:** ways to ask for what's loaded: `mazeReady()` (all 19 maze templates loaded), `maze(name)` (one maze template), `get(name)` (one variant), `names()` (all variant names, for tab completion) and `weighted()` (the variants with their weights).
- **Lines 110–117:** opens a file inside the jar and lets Bukkit turn it into a `Structure`. `try (…)` is "try-with-resources": the file is closed automatically afterwards, even if something goes wrong. `throws IOException` warns callers that this can fail.

CurrentMonument.java: what's built right now

CurrentMonument.java

```
public final class CurrentMonument {

    /** What's built: a pre-built variant name, or the seed and settings of a generated layout. */
    public sealed interface Built {
        record PreBuilt(String variant) implements Built {
        }

        record Generated(long seed, MazeSettings settings) implements Built {
            public MonumentLayout layout() {
                return MonumentLayout.generate(seed, settings);
            }
        }
    }

    private final Logger logger;
    private final File file;
    private Built built;
    private int sponges = -1;
    private long id;

    /** @param folder where current.yml is kept */
    public CurrentMonument(Logger logger, File folder) {
        this.logger = logger;
        this.file = new File(folder, "current.yml");
        load();
    }
```

```
46
47      /**
48       * A random number that changes every time a new monument is placed. Monument Hoes carry the id
49       * of the monument they came from, so a hoe kept from an earlier monument can be recognised.
50       */
51      public long id() {
52          return id;
53      }
54
55      /** Empty until the plugin has placed its first monument. */
56      public Optional<Built> get() {
57          return Optional.ofNullable(built);
58      }
59
60      /** How many sponges the monument had when it was placed, or empty when unknown. */
61      public OptionalInt sponges() {
62          return sponges < 0 ? OptionalInt.empty() : OptionalInt.of(sponges);
63      }
64
65      /**
66       * @param sponges the number of sponges in the monument right after placing it
67       */
68      public void set(Built built, int sponges) {
69          this.built = built;
70          this.sponges = sponges;
71          this.id = newId();
72          save();
73      }
```

- **Lines 23–32:** a **sealed interface** allows only the listed kinds. A monument is either pre-built (with its variant name) or generated (with its seed and maze settings). A generated one doesn't store its whole layout: the seed plus the settings are enough to regenerate exactly the same layout, which is what `layout current` uses (lines 28–30).
- **Lines 34–38:** the logger, `monuments/current.yml`, what's built, the sponge count (`-1` means unknown), and the monument's random ID.
- **Lines 51–53:** the monument's ID. The Monument Hoe stores it, so a hoe from an earlier monument is recognized and deleted.
- **Lines 56–63:** what's built and the sponge count, as `Optional`s that are empty when unknown.
- **Lines 68–73:** called by the resetter after placing a monument: store what was built and the sponge count, give the new monument a fresh random ID (which makes all older hoes invalid), and save.

CurrentMonument.java

```
75      private void save() {
76          YamlConfiguration yaml = new YamlConfiguration();
77          yaml.set("id", id);
78          yaml.set("sponges", sponges);
79          switch (built) {
80              case null -> {
81              }
82              case Built.PreBuilt(String variant) -> {
83                  yaml.set("type", "pre-built");
84                  yaml.set("variant", variant);
85              }
86              case Built.Generated(long seed, MazeSettings s) -> {
87                  yaml.set("type", "generated");
88                  yaml.set("seed", seed);
89                  yaml.set("peak", s.peak());
90                  yaml.set("spread", s.spread());
91                  yaml.set("min", s.min());
```

```
 92                 yaml.set("max", s.max());
 93                 yaml.set("min-sponge-rooms", s.minSpongeRooms());
 94             }
 95         }
 96         try {
 97             yaml.save(file);
 98         } catch (IOException e) {
 99             logger.log(Level.WARNING, "Could not save " + file.getName(), e);
100         }
101     }
102
103     private static long newId() {
104         return ThreadLocalRandom.current().nextLong();
105     }
106
107     private void load() {
108         if (!file.exists()) {
109             // No monument placed by the plugin yet; give the one in the world an id that survives
restarts.
110             id = newId();
111             save();
112             return;
113         }
114         YamlConfiguration yaml = YamlConfiguration.loadConfiguration(file);
115         sponges = yaml.getInt("sponges", -1);
116         id = yaml.contains("id") ? yaml.getLong("id") : newId();
117         try {
118             built = switch (yaml.getString("type", "")) {
119                 case "pre-built" -> new Built.PreBuilt(yaml.getString("variant", "?"));
120                 case "generated" -> new Built.Generated(yaml.getLong("seed"), new MazeSettings(
121                         yaml.getDouble("peak"), yaml.getDouble("spread"), yaml.getDouble("min"),
122                         yaml.getDouble("max"), yaml.getInt("min-sponge-rooms")));
123                 default -> null;
124             };
125         } catch (IllegalArgumentException e) {
126             logger.warning("Ignoring " + file.getName() + ": " + e.getMessage());
127         }
128         if (!yaml.contains("id")) {
129             save();
130         }
131     }
```

- **Lines 75–101, writing `current.yml`:** the ID, the sponge count, and either the variant or the seed with all the maze settings. All the settings are saved because you might change `monuments.yml` later, but the layout that's built depends on the settings at the time. `case null` handles "nothing built yet".
- **Lines 107–131, reading it at startup:**
  - **Lines 108–113:** no file yet: give the monument in the world an ID and save it straight away, so the ID survives restarts and hoes stay valid.
  - **Lines 117–127:** rebuild "what's built" from the file. If the saved maze settings are invalid, `MazeSettings` throws an error, which is caught: a warning is logged, and what's built becomes unknown.
  - **Lines 128–130:** a file from before IDs existed gets one added and saved.

MonumentResetter.java: one check, and placing the new monument

MonumentResetter.java

```
37 public final class MonumentResetter {
38
39     /** Use as the variant to build a generated monument, whatever reset.mode says. */
```

```
40      public static final String GENERATED = "generated";
41
42      /** Width of one quadrant structure; the other three are placed this far to the west/north. */
43      private static final int QUADRANT_SIZE = 29;
44
45      public sealed interface Result {
46          record NotCleared(int eldersAlive) implements Result {
47          }
48
49          /** Cleared, but the reset was postponed because players are inside the area. */
50          record PlayersInside(int players) implements Result {
51          }
52
53          /** @param description what was placed, e.g. "monument_03" or "generated (seed 123)" */
54          record Placed(String description) implements Result {
55          }
56
57          record Failed(String reason) implements Result {
58          }
59      }
60
61      private final Monuments monuments;
62      private final MonumentBuilder builder;
63      private final Random random = new Random();
64      private boolean running;
65
66      public MonumentResetter(Monuments monuments) {
67          this.monuments = monuments;
68          this.builder = new MonumentBuilder(monuments);
69      }
70
71      public boolean isRunning() {
72          return running;
73      }
```

- **Line 40:** the special word that means "a generated monument", used by `reset generated`.
- **Line 43:** a pre-built quadrant is 29 blocks wide; the other three quadrants are placed 29 blocks further west and/or north.
- **Lines 45–59:** the four possible outcomes of a check, each a small record carrying the details: how many Elder Guardians are alive, how many players are inside, what was placed, or why it failed.
- **Line 62:** the builder for generated monuments (step 6).
- **Line 64:** whether a check is busy, so two can never overlap.

MonumentResetter.java

```
80      public CompletableFuture<Result> run(boolean force, String variant, Long seed) {
81          if (running) {
82              return CompletableFuture.completedFuture(new Result.Failed("A check is already
running."));
83          }
84          if (monuments.isPaused()) {
85              return CompletableFuture.completedFuture(new Result.Failed("Paused: monuments.yml "
86                      + monuments.configProblem() + ". Fix it and run /structure monument reload."));
87          }
88          if (!monuments.isActive()) {
89              return CompletableFuture.completedFuture(new Result.Failed("The monument is inactive.
Check the location,"
90                      + " then run /structure monument active on."));
91          }
92          MonumentArena arena = monuments.arena();
93          if (arena == null) {
```

```
 94              return CompletableFuture.completedFuture(new Result.Failed(
 95                      "World '" + monuments.settings().worldName() + "' is not loaded."));
 96          }
 97
 98          running = true;
 99          return arena.loadChunks()
100                  .thenCompose(ignored -> evaluate(arena, force, variant, seed))
101                  .exceptionally(e -> {
102                      Throwable cause = e instanceof CompletionException && e.getCause() != null ?
e.getCause() : e;
103                      monuments.getLogger().log(Level.WARNING, "Monument check failed", cause);
104                      return new Result.Failed(String.valueOf(cause.getMessage()));
105                  })
106                  .whenComplete((result, error) -> {
107                      arena.releaseChunks();
108                      running = false;
109                  });
110      }
```

- **Lines 81–96:** reasons to stop before starting: a check is already running, the monument is paused, it's switched off (so `check` and `reset` are refused too, with a hint to run `active on`), or the world isn't loaded.
- **Line 98:** mark it as busy.
- **Lines 99–109:** a chain of steps that run one after another, each once the previous one is finished:
  - **Line 99:** load the area and wait for its mobs.
  - **Line 100:** then look at the monument and, if needed, place a new one (`evaluate`). `thenCompose` is used because `evaluate` itself returns a promise: a generated build takes several ticks.
  - **Lines 101–105:** if *anything* in the chain fails, turn it into a `Failed` result with the reason, and log the full error. Line 102 unwraps the actual error from the "wrapper" error Java puts around it.
  - **Lines 106–109: always**, whether it succeeded or failed, release the chunks and clear the busy flag.

MonumentResetter.java

```
112      private CompletableFuture<Result> evaluate(MonumentArena arena, boolean force, String
requested, Long seed) {
113          MonumentConfig settings = monuments.settings();
114
115          int alive = arena.entities(ElderGuardian.class).size();
116          if (alive > 0 && !force) {
117              announce(Component.text("The Monument has not been cleared.", NamedTextColor.RED));
118              return done(new Result.NotCleared(alive));
119          }
120          if (!force) {
121              int players = arena.entities(Player.class).size();
122              if (players > 0) {
123                  announce(Component.text("The Monument has been cleared, it will reset once everyone
has left.",
124                          NamedTextColor.YELLOW));
125                  return done(new Result.PlayersInside(players));
126              }
127              announce(Component.text("The Monument has been cleared.", NamedTextColor.GREEN));
128          }
129
130          boolean generated = requested == null ? settings.mode() == ResetMode.GENERATED :
requested.equals(GENERATED);
131          return generated ? placeGenerated(arena, seed) : placePreBuilt(arena, requested);
132      }
```

- **The heart of your datapack's `monument_checks`.**

- **Lines 115–119:** any Elder Guardians alive (and not forced)? Then broadcast "not cleared" to admins and stop.
- **Lines 120–128:** not forced and no Elder Guardians: are players inside? Then postpone. Otherwise, broadcast "cleared". A forced `reset` skips both checks.
- **Line 130:** what to place: no specific request → follow `reset.mode`; `generated` → a generated monument; anything else → that pre-built variant.
- **Line 131:** place it.

MonumentResetter.java

```
134     private CompletableFuture<Result> placePreBuilt(MonumentArena arena, String requested) {
135         MonumentConfig settings = monuments.settings();
136         Optional<MonumentVariant> variant = requested != null
137                 ? monuments.structures().get(requested)
138                 : Weighted.pick(monuments.structures().weighted(), random);
139         if (variant.isEmpty()) {
140             return done(new Result.Failed(requested != null
141                     ? "Unknown variant '" + requested + "'."
142                     : "No monument variants with a weight above 0 are loaded."));
143         }
144 
145         clearGuardians(arena);
146         // One block below the center, like the generated monument's empty shell
(maze/reset.mcfunction
147         // places it at ~-1). The datapack placed these at ~, which put them one block too high.
148         Location center = arena.center().add(0, -1, 0);
149         placeQuadrant(variant.get().se(), center);
150         placeQuadrant(variant.get().sw(), center.clone().add(-QUADRANT_SIZE, 0, 0));
151         placeQuadrant(variant.get().ne(), center.clone().add(0, 0, -QUADRANT_SIZE));
152         placeQuadrant(variant.get().nw(), center.clone().add(-QUADRANT_SIZE, 0, -QUADRANT_SIZE));
153         // The quadrant structures hold the two wing Elder Guardians; the third one in the top room
is
154         // summoned separately, because saving it in a quadrant gave monuments with 2, 3 or 4 of
them.
155         spawnElderGuardian(arena.center().add(0, settings.elderYOffset(), 0));
156 
157         monuments.currentMonument().set(new Built.PreBuilt(variant.get().name()),
arena.countSponges());
158         monuments.getLogger().info("Placed monument variant " + variant.get().name() + ".");
159         announce(Component.text("The Ocean Monument has been reset.", NamedTextColor.GREEN));
160         return done(new Result.Placed(variant.get().name()));
161     }
```

- **Your datapack's `monument_place`.**
- **Lines 136–138:** the requested variant, or a weighted random one.
- **Line 145:** remove leftover Guardians, if `clear-guardians` is on.
- **Line 148:** the quadrants go 1 block below the center, like the generated monument's empty shell.
- **Lines 149–152:** the four quadrants, at the same offsets as your datapack's `~`, `~-29 ~ ~`, `~ ~ ~-29` and `~-29 ~ ~-29`.
- **Line 155:** the third Elder Guardian in the top room. The two wing ones come with the quadrant files.
- **Line 157:** remember what's built, and count the sponges for the hoe.
- **Lines 158–160:** log it, tell admins, and report "placed".

MonumentResetter.java

```
163      private CompletableFuture<Result> placeGenerated(MonumentArena arena, Long requestedSeed) {
164          if (!monuments.structures().mazeReady()) {
165              return done(new Result.Failed("The templates for generated monuments didn't load, see
the console."));
166          }
167          long seed = requestedSeed != null ? requestedSeed : ThreadLocalRandom.current().nextLong();
168          var mazeSettings = monuments.settings().maze();
169          MonumentLayout layout = MonumentLayout.generate(seed, mazeSettings);
170
171          clearGuardians(arena);
172          return builder.build(arena, BuildPlanner.plan(layout), seed,
this::spawnElderGuardian).thenApply(ignored -> {
173              monuments.currentMonument().set(new Built.Generated(seed, mazeSettings),
arena.countSponges());
174              String description = "generated monument (seed " + seed + ", "
175                      + Math.round(layout.newestChance() * 100) + "% newest)";
176              monuments.getLogger().info("Built " + description + ".");
177              announce(Component.text("The Ocean Monument has been reset.", NamedTextColor.GREEN));
178              return new Result.Placed(description);
179          });
180      }
```

- **Line 164:** without all 19 templates, refuse.
- **Lines 167–169:** the requested seed or a random one, then generate the layout (step 5). `var` lets Java work out the type itself.
- **Line 172:** turn the layout into build steps (`BuildPlanner`) and let the builder place them over several ticks. It also gets a way to spawn the three Elder Guardians: `this::spawnElderGuardian` passes this class's own spawn method along. `thenApply` runs once building has finished.
- **Lines 173–178:** the same as for pre-built, plus the seed and newest chance in the description.

MonumentResetter.java

```
182      private void clearGuardians(MonumentArena arena) {
183          if (monuments.settings().clearGuardians()) {
184              // ElderGuardian extends Guardian, so this also clears any Elder Guardians left by a
forced reset.
185              arena.entities(Guardian.class).forEach(Entity::remove);
186          }
187      }
188
189      private void placeQuadrant(Structure structure, Location origin) {
190          // Same as /place template: entities included, no rotation, random palette, full integrity.
191          structure.place(origin, true, StructureRotation.NONE, Mirror.NONE, -1, 1f, random);
192      }
193
194      /** Spawns an Elder Guardian with the configured health and attack damage. */
195      private void spawnElderGuardian(Location location) {
196          MonumentConfig settings = monuments.settings();
197          location.getWorld().spawn(location, ElderGuardian.class, guardian -> {
198              setBase(guardian, Attribute.MAX_HEALTH, settings.elderHealth());
199              setBase(guardian, Attribute.ATTACK_DAMAGE, settings.elderAttackDamage());
200              guardian.setHealth(settings.elderHealth());
201              guardian.setRemoveWhenFarAway(false);
202          });
203      }
204
205      private static void setBase(LivingEntity entity, Attribute attribute, double value) {
206          AttributeInstance instance = entity.getAttribute(attribute);
207          if (instance != null) {
```

```
208             instance.setBaseValue(value);
209         }
210     }
211
212     private static CompletableFuture<Result> done(Result result) {
213         return CompletableFuture.completedFuture(result);
214     }
215
216     /** Sends a message to everyone with the admin permission (and the console), when broadcasts
are on. */
217     private void announce(Component message) {
218         if (monuments.settings().announce()) {
219             monuments.getServer().broadcast(message, ResettingStructures.ADMIN_PERMISSION);
220         }
221     }
```

- **Lines 182–187:** remove every Guardian at the monument. Elder Guardians count as a kind of Guardian, so leftovers from a forced reset go too.
- **Lines 189–192:** place one template, like `/place template`: entities included, no rotation or mirroring, a random palette (`-1`), and every block (`1f`).
- **Lines 195–203:** spawns an Elder Guardian with your stats. The part after `guardian ->` runs *before* it appears in the world, so it never shows up with vanilla stats for a moment. Max health and attack damage are set, its health is filled to the new max, and it never despawns.
- **Lines 205–210:** set an attribute's base value, skipping it safely if the mob somehow doesn't have that attribute.
- **Lines 212–214:** wraps an immediate answer in an already-fulfilled promise.
- **Lines 217–221:** the admin broadcast, only when `announce` is on, and only to players with `structures_admin` (and the console).

Step 5: the maze generator

These files only work out a **layout**: which walls are open and which room goes where. They never touch the world, which is why they can be tested so thoroughly (thousands of layouts per test run). Step 6 turns a layout into blocks.

The big picture

The grid, seen from above, entrance at the bottom:

```
Floors 0 and 1 (5 x 4 units)        Floor 2 (3 x 2 units)
row 3  [0][1][2][3][4]   ← back
row 2  [0][1][2][3][4]
row 1  [0][1][2][3][4]                row 1     [1][2][3]
row 0  [0][1][2][3][4]   ← front      row 0     [1][2][3]
              ↑ entrance (x 2, row 0, floor 0)
```

- **Units:** each `[ ]` is one 8×8 unit, your datapack's "node". That's 20 + 20 + 6 = **46 units**.
- **Dummy units:** stand-ins that join the maze but get no room. The **wings** sit in front of the front corners (row −1 at x 0 and x 4 on floor 1, and also floor 0 for an open wing). The **penthouse** sits above the top floor's front center (floor 3).

How a layout is made (`MonumentLayout.generate`):

```
1. draw this monument's newest chance from the bell curve          MazeSettings
2. roll the wings (open/closed) and the core's position            MonumentLayout
3. close every wall, then carve a maze by opening walls            GrowingTreeGenerator
4. give every unit a room: core, entrance, big, hall, tall, ...    RoomFitter
5. too few sponge rooms? rewire a unit into one, fit rooms again   SpongeRooms
```

GridDirection.java: the six directions

GridDirection.java

```
 9  public enum GridDirection {
10      EAST(1, 0, 0),
11      WEST(-1, 0, 0),
12      BACK(0, 0, 1),
13      FRONT(0, 0, -1),
14      UP(0, 1, 0),
15      DOWN(0, -1, 0);
16
17      public static final List<GridDirection> HORIZONTAL = List.of(EAST, WEST, BACK, FRONT);
18
19      private final int dx;
20      private final int dFloor;
21      private final int dRow;
22
23      GridDirection(int dx, int dFloor, int dRow) {
24          this.dx = dx;
25          this.dFloor = dFloor;
26          this.dRow = dRow;
27      }
28
29      public int dx() {
30          return dx;
31      }
```

```
32
33      public int dFloor() {
34          return dFloor;
35      }
36
37      public int dRow() {
38          return dRow;
39      }
40
41      public GridDirection opposite() {
42          return switch (this) {
43              case EAST -> WEST;
44              case WEST -> EAST;
45              case BACK -> FRONT;
46              case FRONT -> BACK;
47              case UP -> DOWN;
48              case DOWN -> UP;
49          };
50      }
```

- **Lines 9–15:** an `enum` whose choices carry data. Each direction stores how a step that way changes x, the floor and the row. `BACK` means one row further from the entrance: north in your datapack.
- **Line 17:** the four sideways directions.
- **Lines 19–39:** where the three numbers are stored (an enum constructor runs once per choice, when Java loads the enum), plus getters.
- **Lines 41–50:** the other side of a wall: opening EAST from one unit is the same wall as WEST from its neighbor.

Cell.java: one unit

Cell.java

```
 8  public final class Cell {
 9
10      public enum Kind {
11          ROOM,
12          LEFT_WING,
13          RIGHT_WING,
14          PENTHOUSE
15      }
16
17      private final int id;
18      private final int x;
19      private final int floor;
20      private final int row;
21      private final Kind kind;
22
23      final Cell[] neighbors = new Cell[GridDirection.values().length];
24      final boolean[] open = new boolean[GridDirection.values().length];
25      final boolean[] blocked = new boolean[GridDirection.values().length];
26
27      Cell(int id, int x, int floor, int row, Kind kind) {
28          this.id = id;
29          this.x = x;
30          this.floor = floor;
31          this.row = row;
32          this.kind = kind;
33      }
```

- **Lines 10–15:** what a cell is: a real unit, or a dummy for a wing or the penthouse.

- **Line 17:** a number from 0 upward, unique within the grid. Other code uses it as an index into arrays like "visited" (`visited[cell.id()]`), which is very fast.
- **Lines 18–21:** its position and kind.
- **Lines 23–25:** three arrays with one slot per direction (6), in the enum's order. `direction.ordinal()` is a direction's position in that order (EAST = 0 … DOWN = 5).
  - `neighbors`: the cell next to it in each direction, or `null` at the edge.
  - `open`: whether the wall that way is open.
  - `blocked`: walls that may never open. Only one is blocked: the entrance's ceiling, as in your datapack.

These have no `private` or `public`: that's **package-private**. Only classes in the same package (`layout`) can change them directly, which forces everyone else to change walls through `MonumentGrid.setOpen`, which always updates both sides together.

- **Line 27:** also package-private, so only `MonumentGrid` can create cells.

Cell.java

```
58      public boolean isDummy() {
59          return kind != Kind.ROOM;
60      }
61
62      public Cell neighbor(GridDirection direction) {
63          return neighbors[direction.ordinal()];
64      }
65
66      public boolean isOpen(GridDirection direction) {
67          return open[direction.ordinal()];
68      }
69
70      public int openCount() {
71          int count = 0;
72          for (boolean o : open) {
73              if (o) {
74                  count++;
75              }
76          }
77          return count;
78      }
79
80      @Override
81      public String toString() {
82          return isDummy() ? kind.name() : "(" + x + "," + floor + "," + row + ")";
83      }
```

- **Lines 58–78:** small questions: is it a dummy, who's next to it, is that wall open, and how many open walls it has. A cell with one open wall is a **dead end**.
- **Lines 80–83:** how a cell looks in error messages, e.g. `(2,0,0)` or `PENTHOUSE`.

(The comment at the top of this file, line 4, still says "three dummy units"; since open wings were added there can be up to five.)

MonumentGrid.java: the grid and its walls

MonumentGrid.java

```
21  public final class MonumentGrid {
22
23      public static final int WIDTH = 5;
24      public static final int ROWS = 4;
25      public static final int FLOORS = 3;
26
27      /** A wall between {@code cell} and its neighbor in {@code direction}. */
28      public record Wall(Cell cell, GridDirection direction) {
29          public Cell other() {
30              return cell.neighbor(direction);
31          }
32      }
33
34      private final List<Cell> cells = new ArrayList<>();
35      private final List<Cell> roomCells = new ArrayList<>();
36      private final Map<Long, Cell> byPosition = new HashMap<>();
37      private final Cell entry;
38      private final boolean leftWingOpen;
39      private final boolean rightWingOpen;
```

- **Lines 23–25:** the grid's size.
- **Lines 28–32:** a wall, described as "this cell's wall in that direction". `other()` is the cell on the far side.
- **Lines 34–39:** all cells, the 46 real ones only, a lookup from position to cell, the entrance, and the wing designs.

MonumentGrid.java

```
46      public MonumentGrid(boolean leftWingOpen, boolean rightWingOpen) {
47          this.leftWingOpen = leftWingOpen;
48          this.rightWingOpen = rightWingOpen;
49          for (int floor = 0; floor < FLOORS; floor++) {
50              for (int row = 0; row < ROWS; row++) {
51                  for (int x = 0; x < WIDTH; x++) {
52                      if (isRoomPosition(x, floor, row)) {
53                          roomCells.add(add(x, floor, row, Cell.Kind.ROOM));
54                      }
55                  }
56              }
57          }
58          // The wings sit in front of the front corners, the penthouse above the top floor's front
center.
59          // At these positions each dummy only touches its own unit (and an open wing's other half).
60          add(0, 1, -1, Cell.Kind.LEFT_WING);
61          add(4, 1, -1, Cell.Kind.RIGHT_WING);
62          if (leftWingOpen) {
63              add(0, 0, -1, Cell.Kind.LEFT_WING);
64          }
65          if (rightWingOpen) {
66              add(4, 0, -1, Cell.Kind.RIGHT_WING);
67          }
68          add(2, 3, 0, Cell.Kind.PENTHOUSE);
69
70          for (Cell cell : cells) {
71              for (GridDirection d : GridDirection.values()) {
72                  cell.neighbors[d.ordinal()] = at(cell.x() + d.dx(), cell.floor() + d.dFloor(),
cell.row() + d.dRow());
73              }
74          }
```

```
75
76          entry = at(2, 0, 0);
77          // Like the datapack: the entry room has no opening in its ceiling.
78          block(entry, GridDirection.UP);
79      }
80
81      public static boolean isRoomPosition(int x, int floor, int row) {
82          if (floor == 0 || floor == 1) {
83              return x >= 0 && x < WIDTH && row >= 0 && row < ROWS;
84          }
85          return floor == 2 && x >= 1 && x <= 3 && row >= 0 && row <= 1;
86      }
```

- **Lines 49–57:** create the 46 real units, skipping positions that don't exist (the top floor is smaller).
- **Lines 60–68:** the dummies. Each wing has a unit in front of a front corner on floor 1; an **open** wing gets a second unit under it, on floor 0, so it can connect on both floors (your datapack's wing nodes at `~` and `~4`). The penthouse sits above the top floor's middle front unit. These positions are chosen so each dummy only touches its own unit.
- **Lines 70–74:** for every cell and every direction, look up the neighbor by position. A missing one becomes `null`.
- **Lines 76–78:** remember the entrance, and permanently block its ceiling.
- **Lines 81–86:** which positions are real units: all 5×4 on floors 0 and 1, and x 1–3, rows 0–1 on floor 2. `||` means "or", `&&` means "and".

MonumentGrid.java

```
112     /** The cell at a grid position (dummies included), or {@code null}. */
113     public Cell at(int x, int floor, int row) {
114         return byPosition.get(key(x, floor, row));
115     }
116
117     /** Whether the wall can ever be open: a neighbor exists and the wall isn't blocked. */
118     public boolean canOpen(Cell cell, GridDirection direction) {
119         return cell.neighbor(direction) != null && !cell.blocked[direction.ordinal()];
120     }
121
122     /** Opens or closes a wall on both sides. Returns false when the wall can't be opened. */
123     public boolean setOpen(Cell cell, GridDirection direction, boolean open) {
124         if (open && !canOpen(cell, direction)) {
125             return false;
126         }
127         Cell other = cell.neighbor(direction);
128         if (other == null) {
129             return !open;
130         }
131         cell.open[direction.ordinal()] = open;
132         other.open[direction.opposite().ordinal()] = open;
133         return true;
134     }
135
136     /** Opens (or closes) every wall that can be opened. */
137     public void setAllOpen(boolean open) {
138         for (Cell cell : cells) {
139             for (GridDirection d : GridDirection.values()) {
140                 if (canOpen(cell, d)) {
141                     setOpen(cell, d, open);
142                 }
143             }
144         }
145     }
```

```
146
147     /** Every wall between two cells, each listed once, open or closed. */
148     public List<Wall> walls() {
149         List<Wall> walls = new ArrayList<>();
150         for (Cell cell : cells) {
151             for (GridDirection d : List.of(GridDirection.EAST, GridDirection.BACK,
GridDirection.UP)) {
152                 if (cell.neighbor(d) != null) {
153                     walls.add(new Wall(cell, d));
154                 }
155             }
156         }
157         return walls;
158     }
```

- **Line 113:** the cell at a position, or `null`.
- **Lines 118–120:** a wall can ever open if there's a neighbor and it isn't blocked.
- **Lines 123–134: the only way walls change.** Line 131 sets this side and line 132 the neighbor's side (the opposite direction), so both always agree.
- **Lines 137–145:** open or close everything at once. The maze generator starts with "all closed".
- **Lines 148–158:** every wall exactly once. It only looks EAST, BACK and UP from each cell: the WEST, FRONT and DOWN walls are the same walls seen from the other side.

MonumentGrid.java

```
160     /** Whether a path through open walls leads from {@code from} to the entry. */
161     public boolean reachesEntry(Cell from) {
162         return reachableFrom(from)[entry.id()];
163     }
164
165     /** Whether every cell, dummies included, can reach the entry through open walls. */
166     public boolean allReachEntry() {
167         boolean[] reached = reachableFrom(entry);
168         for (Cell cell : cells) {
169             if (!reached[cell.id()]) {
170                 return false;
171             }
172         }
173         return true;
174     }
175
176     /** Which cells (by id) can reach the entry through open walls. */
177     public boolean[] reachableFromEntry() {
178         return reachableFrom(entry);
179     }
180
181     /** Number of steps through open walls from the entry to every cell (by id), -1 when
unreachable. */
182     public int[] distancesFromEntry() {
183         int[] distance = new int[cells.size()];
184         Arrays.fill(distance, -1);
185         Deque<Cell> queue = new ArrayDeque<>();
186         distance[entry.id()] = 0;
187         queue.add(entry);
188         while (!queue.isEmpty()) {
189             Cell cell = queue.poll();
190             for (GridDirection d : GridDirection.values()) {
191                 Cell n = cell.neighbor(d);
192                 if (n != null && cell.isOpen(d) && distance[n.id()] < 0) {
193                     distance[n.id()] = distance[cell.id()] + 1;
194                     queue.add(n);
```

```
                }
            }
        }
        return distance;
    }
```

- **Lines 160–179:** "can everything still be reached from the entrance?" The tests use this on every layout, and so does the sponge guarantee.
- **Lines 181–199:** how many steps each unit is from the entrance, walking through open walls. This is a **breadth-first search**, like water spreading out from the entrance one step at a time:
  - **Line 184:** everything starts at "not reached" (`-1`).
  - **Lines 186–187:** the entrance is 0 steps. Put it in a **queue**, a waiting line: first in, first out.
  - **Lines 188–197:** take the next cell from the queue. Every neighbor through an open wall that hasn't been reached gets "one step further", and joins the queue.

MonumentGrid.java

```
    private void block(Cell cell, GridDirection direction) {
        Cell other = cell.neighbor(direction);
        setOpen(cell, direction, false);
        cell.blocked[direction.ordinal()] = true;
        if (other != null) {
            other.blocked[direction.opposite().ordinal()] = true;
        }
    }

    private Cell add(int x, int floor, int row, Cell.Kind kind) {
        Cell cell = new Cell(cells.size(), x, floor, row, kind);
        cells.add(cell);
        byPosition.put(key(x, floor, row), cell);
        return cell;
    }

    private static long key(int x, int floor, int row) {
        return ((long) (x + 16) << 32) | ((long) (floor + 16) << 16) | (row + 16);
    }
```

- **Lines 219–226:** close a wall and mark it blocked on both sides.
- **Lines 228–233:** create a cell, using the current number of cells as its ID (0, 1, 2…).
- **Lines 235–237:** turns three small numbers into one number, to use as a map key. `+ 16` keeps them positive (the wings are at row −1); `<<` shifts each into its own part of the big number and `|` combines them. Think of writing a date as 20261006: three numbers in one.

MazeSettings.java: the maze settings and the bell curve

MazeSettings.java

```
public record MazeSettings(double peak, double spread, double min, double max, int minSpongeRooms)
{

    public static final MazeSettings DEFAULT = new MazeSettings(0.65, 0.15, 0.0, 1.0, 1);

    private static final int MAX_DRAWS = 100;

    public MazeSettings {
        if (min < 0 || max > 1 || max < min) {
            throw new IllegalArgumentException("Newest chance must satisfy 0 <= min <= max <= 1,
got " + min + ".." + max);
        }
```

```
28          if (peak < min || peak > max) {
29              throw new IllegalArgumentException("Peak " + peak + " is outside " + min + ".." + max);
30          }
31          if (spread < 0) {
32              throw new IllegalArgumentException("Spread can't be negative: " + spread);
33          }
34          if (minSpongeRooms < 0) {
35              throw new IllegalArgumentException("Minimum sponge rooms can't be negative: " +
minSpongeRooms);
36          }
37      }
38
39      /** Settings that always use this newest chance, for previews and tests. */
40      public MazeSettings withNewestChance(double chance) {
41          return new MazeSettings(chance, 0, chance, chance, minSpongeRooms);
42      }
43
44      public MazeSettings withMinSpongeRooms(int min) {
45          return new MazeSettings(peak, spread, this.min, max, min);
46      }
47
48      /** Draws a newest chance: normal distribution around the peak, redrawn when it falls outside
min..max. */
49      double drawNewestChance(RandomGenerator random) {
50          // Always draw once, so a fixed chance (spread 0) uses the random numbers the same way as a
drawn one.
51          double value = peak + spread * random.nextGaussian();
52          for (int draw = 1; (value < min || value > max) && draw < MAX_DRAWS; draw++) {
53              value = peak + spread * random.nextGaussian();
54          }
55          return Math.clamp(value, min, max);
56      }
```

- **Line 18:** the settings from `maze:` in `monuments.yml`.
- **Line 20:** the defaults.
- **Lines 24–37:** checks every time one is created; impossible values are an error.
- **Lines 40–46:** copies with one thing changed (a record can't be changed, so you make a new one). Line 40 makes a fixed newest chance (the curve squeezed to one value), used by `/structure monument layout <seed> 0.9` and the tests.
- **Lines 49–56, drawing one monument's newest chance:**
  - **Line 51:** `nextGaussian()` gives a random number on a standard bell curve: usually near 0, about 2 in 3 between −1 and +1. Multiplying by the spread and adding the peak moves the curve to 0.65 ± 0.15.
  - **Lines 52–54:** outside min..max? Draw again, up to 100 times.
  - **Line 55:** in the extremely unlikely case all 100 draws missed, force the value inside. `Math.clamp(v, min, max)` means "v, but never below min or above max".

GrowingTreeGenerator.java: carving the maze

GrowingTreeGenerator.java

```
25      static void carve(MonumentGrid grid, RandomGenerator random, double newestChance) {
26          grid.setAllOpen(false);
27          boolean[] visited = new boolean[grid.cells().size()];
28          List<Cell> active = new ArrayList<>();
29
30          List<Cell> starts = grid.roomCells();
31          Cell start = starts.get(random.nextInt(starts.size()));
32          visited[start.id()] = true;
```

```
33          active.add(start);
34
35          List<GridDirection> options = new ArrayList<>(6);
36          while (!active.isEmpty()) {
37              int index = random.nextDouble() < newestChance ? active.size() - 1 :
random.nextInt(active.size());
38              Cell current = active.get(index);
39
40              options.clear();
41              for (GridDirection d : GridDirection.values()) {
42                  if (grid.canOpen(current, d) && !visited[current.neighbor(d).id()]) {
43                      options.add(d);
44                  }
45              }
46              if (options.isEmpty()) {
47                  active.remove(index);
48                  continue;
49              }
50
51              GridDirection step = options.get(random.nextInt(options.size()));
52              Cell next = current.neighbor(step);
53              grid.setOpen(current, step, true);
54              visited[next.id()] = true;
55              active.add(next);
56          }
57      }
```

- **Line 26:** start with every wall closed.
- **Lines 27–28:** `visited` marks the cells already in the maze; `active` lists the cells in the maze that might still have unvisited neighbors.
- **Lines 30–33:** begin at a random real unit.
- **Line 36:** keep going until no active cells are left; by then every cell, dummies included, is in the maze.
- **Line 37, the one line that gives the maze its character:** `nextDouble()` is a random number from 0 to 1. Below the newest chance, continue from the **newest** cell, which makes long corridors; otherwise from a **random** cell in the list, which makes branches.
- **Lines 40–45:** which directions lead to a cell that isn't in the maze yet?
- **Lines 46–49:** none, so this cell is finished; take it off the list. `continue` jumps back to the start of the loop.
- **Lines 51–55:** pick one of those directions at random, open that wall, and add the new cell to the maze and the list.

Because each new cell is reached by exactly one opened wall, the result is a perfect maze: exactly one route between any two units.

RoomType.java and Room.java: rooms

RoomType.java

```
 8  public enum RoomType {
 9      CORE('C', "room/2x2x2_core"),
10      ENTRY('E', "room/1x1x1_start"),
11      /** Vanilla DXY: two hollow pillars. */
12      BIG_PIPES('P', "room/2x1x2_pipes"),
13      /** Vanilla DYZ: 2x2 dark prismarine pillar. */
14      BIG_DARK('D', "room/2x1x2_dark"),
15      /** Vanilla DX: U-shaped. */
16      HALL_U('U', "room/2x1x1_u"),
17      /** Vanilla DZ: open center with sea lanterns. */
18      HALL_LANTERN('L', "room/2x1x1_lantern"),
19      /** Vanilla DY. */
20      TALL('T', "room/1x1x2"),
21      SPONGE('S', "room/1x1x1_sponge"),
22      /** Plus-shaped small room. */
23      SMALL_LANTERNS('l', "room/1x1x1_lanterns"),
24      /** Four-pillar small room, can get a center pillar. */
25      SMALL_PILLARS('i', "room/1x1x1_pillars"),
26      /** Dark prismarine small room, can get a center pillar. */
27      SMALL_DARK('d', "room/1x1x1_dark");
```

- **Every room type,** with its letter for the console map and its template file. The comments give the vanilla name (DXY, DYZ, DX, DZ, DY). Adding a room design means adding a line here; `StructureLibrary` then loads it automatically.

Room.java

```
12  public record Room(RoomType type, List<Cell> cells, GridDirection axis, boolean centerPillar) {
13
14      public Room {
15          cells = List.copyOf(cells);
16      }
17
18      public Cell anchor() {
19          return cells.getFirst();
20      }
```

- **One placed room:** its type, the units it covers, the axis (EAST = along X, BACK = along Z, for 2-wide rooms; otherwise `null`), and whether a small room gets the 2×2 center pillar.
- **Line 15:** stores an unchangeable copy of the list.
- **Lines 18–20:** the **anchor**, the first unit, which is always the lowest-x/floor/row one. The builder measures from there.

RoomFitter.java: giving every unit a room

RoomFitter.java

```
16      private static final List<GridDirection> AXES = List.of(GridDirection.EAST,
GridDirection.BACK);
17
18      private final MonumentGrid grid;
19      private final RandomGenerator random;
20      private final boolean[] claimed;
21      private final List<Room> rooms = new ArrayList<>();
22
23      private RoomFitter(MonumentGrid grid, RandomGenerator random) {
24          this.grid = grid;
```

```
25          this.random = random;
26          this.claimed = new boolean[grid.cells().size()];
27          for (Cell cell : grid.cells()) {
28              claimed[cell.id()] = cell.isDummy();
29          }
30      }
31
32      /**
33       * @param coreAnchor the core's front-left bottom unit; the core takes up 2x2x2 units from
there
34       * @param sponges    units that must become sponge rooms (see {@link SpongeRooms}), claimed
before
35       *                   the other rooms so a tall room from below can't take them
36       */
37      static List<Room> fit(MonumentGrid grid, Cell coreAnchor, Set<Cell> sponges, RandomGenerator
random) {
38          RoomFitter fitter = new RoomFitter(grid, random);
39          fitter.claim(new Room(RoomType.CORE, coreCells(grid, coreAnchor), null, false));
40          fitter.claim(new Room(RoomType.ENTRY, List.of(grid.entry()), null, false));
41          for (Cell sponge : sponges) {
42              fitter.claim(new Room(RoomType.SPONGE, List.of(sponge), null, false));
43          }
44
45          List<Cell> order = new ArrayList<>(grid.roomCells());
46          Collections.shuffle(order, random);
47          for (Cell cell : order) {
48              if (!fitter.claimed[cell.id()]) {
49                  fitter.fitRoom(cell);
50              }
51          }
52          return List.copyOf(fitter.rooms);
53      }
54
55      /** The 8 units of the core, anchor first. */
56      static List<Cell> coreCells(MonumentGrid grid, Cell anchor) {
57          List<Cell> cells = new ArrayList<>(8);
58          for (int floor = 0; floor < 2; floor++) {
59              for (int row = 0; row < 2; row++) {
60                  for (int x = 0; x < 2; x++) {
61                      cells.add(grid.at(anchor.x() + x, anchor.floor() + floor, anchor.row() + row));
62                  }
63              }
64          }
65          return cells;
66      }
```

- **Line 16:** the two directions a 2-wide room can run.
- **Line 20:** which units already have a room. The dummies start as "claimed" (lines 27–29), because they never get one.
- **Lines 39–43:** first the fixed rooms: the **core** (8 units), the **entrance**, and any sponge rooms the guarantee asked for (claimed first so a tall room from below can't take them).
- **Lines 45–51:** go through all units in a **random order** (`shuffle`) and give every unit that's still free the first room that fits. This is vanilla's method.
- **Lines 56–66:** the core's 8 units: 2 wide, 2 deep, 2 high, starting at the anchor.

RoomFitter.java

```
68      private void fitRoom(Cell cell) {
69          Cell up = free(cell, GridDirection.UP);
70
71          // 1-2. Vanilla DXY / DYZ: 2 wide and 2 tall.
```

```
 72          for (GridDirection axis : shuffledAxes()) {
 73              Cell side = free(cell, axis);
 74              Cell sideUp = side == null ? null : free(side, GridDirection.UP);
 75              if (side != null && up != null && sideUp != null) {
 76                  claim(new Room(pick(RoomType.BIG_PIPES, RoomType.BIG_DARK), List.of(cell, side, up,
sideUp), axis, false));
 77                  return;
 78              }
 79          }
 80          // 3-4. Vanilla DZ / DX: 2 wide.
 81          for (GridDirection axis : shuffledAxes()) {
 82              Cell side = free(cell, axis);
 83              if (side != null) {
 84                  claim(new Room(pick(RoomType.HALL_U, RoomType.HALL_LANTERN), List.of(cell, side),
axis, false));
 85                  return;
 86              }
 87          }
 88          // 5. Vanilla DY: 2 tall.
 89          if (up != null) {
 90              claim(new Room(RoomType.TALL, List.of(cell, up), null, false));
 91              return;
 92          }
 93          // 6. Sponge room: every wall closed except the floor.
 94          if (onlyFloorOpen(cell)) {
 95              claim(new Room(RoomType.SPONGE, List.of(cell), null, false));
 96              return;
 97          }
 98          // 7. Small room, 1/3 each. Four-pillar and dark rooms get a 50% chance of a center
 99          //    pillar when the floor and ceiling are closed and at least two walls are open.
100          RoomType type = switch (random.nextInt(3)) {
101              case 0 -> RoomType.SMALL_LANTERNS;
102              case 1 -> RoomType.SMALL_PILLARS;
103              default -> RoomType.SMALL_DARK;
104          };
105          boolean pillar = type != RoomType.SMALL_LANTERNS && random.nextBoolean()
106                  && !cell.isOpen(GridDirection.UP) && !cell.isOpen(GridDirection.DOWN) &&
cell.openCount() >= 2;
107          claim(new Room(type, List.of(cell), null, pillar));
108      }
```

Vanilla's priority list, tried in this order. Each `return` means "done, skip the rest".

- **Line 69:** the unit above, if the wall to it is open and it's free.
- **Lines 72–79, the 2×2 room (pipes or dark):** for each axis, in random order: the unit beside it, the unit above, and the unit above the side one must all be free and connected by open walls. Then the 4 units become one big room, 50/50 pipes or dark.
- **Lines 81–87, the 2-wide hall (U or lantern):** a free, open neighbor along either axis.
- **Lines 89–92, the tall room:** the unit above is free and open.
- **Lines 94–97, the sponge room:** only the floor is open.
- **Lines 100–107, a small room:** one of three designs, equally likely. Pillars and dark rooms get the center pillar with a 50% chance, but only when the floor and ceiling are closed and at least two walls are open, as vanilla does. `random.nextBoolean()` is a coin flip.

RoomFitter.java

```
110      /** The neighbor in that direction if the wall to it is open and it's still unused, else null.
*/
111      private Cell free(Cell cell, GridDirection direction) {
112          Cell neighbor = cell.neighbor(direction);
113          return neighbor != null && cell.isOpen(direction) && !claimed[neighbor.id()] ? neighbor :
null;
114      }
115
116      private static boolean onlyFloorOpen(Cell cell) {
117          for (GridDirection d : GridDirection.values()) {
118              if (d != GridDirection.DOWN && cell.isOpen(d)) {
119                  return false;
120              }
121          }
122          return true;
123      }
124
125      private List<GridDirection> shuffledAxes() {
126          return random.nextBoolean() ? AXES : AXES.reversed();
127      }
128
129      private RoomType pick(RoomType a, RoomType b) {
130          return random.nextBoolean() ? a : b;
131      }
132
133      private void claim(Room room) {
134          for (Cell cell : room.cells()) {
135              claimed[cell.id()] = true;
136          }
137          rooms.add(room);
138      }
```

- **Lines 111–114, the key rule:** a room may only span units that the maze connected **with an open wall** and that are still free. That's why rooms never cut through the maze.
- **Lines 125–127:** the two axes in random order, so X and Z are equally likely.
- **Lines 129–131:** a coin flip between two designs.
- **Lines 133–138:** mark a room's units as taken, and add it to the list.

SpongeRooms.java: guaranteeing a sponge room

SpongeRooms.java

```
32      static Cell makeSpongeSpot(MonumentGrid grid, Set<Cell> excluded, Set<Cell> sponges,
RandomGenerator random) {
33          int[] distance = grid.distancesFromEntry();
34          List<Cell> candidates = new ArrayList<>();
35          for (Cell cell : grid.roomCells()) {
36              Cell below = cell.neighbor(GridDirection.DOWN);
37              if (below != null && grid.canOpen(cell, GridDirection.DOWN)
38                      && !excluded.contains(cell) && !sponges.contains(cell) &&
!sponges.contains(below)) {
39                  candidates.add(cell);
40              }
41          }
42          // Dead ends first (rewiring one never removes a loop), then the furthest from the entry.
43          candidates.sort(Comparator.comparing((Cell c) -> c.openCount() != 1)
44                  .thenComparing(Comparator.comparingInt((Cell c) -> distance[c.id()]).reversed()));
45
46          for (boolean allowRepair : new boolean[]{false, true}) {
47              for (Cell cell : candidates) {
48                  if (tryRewire(grid, cell, sponges, allowRepair, random)) {
49                      return cell;
50                  }
51              }
52          }
53          return null;
54      }
```

- **Lines 35–41, candidates:** units with a unit below them (so a floor opening is possible), that aren't the core, the entrance, an existing sponge spot, or directly above one.
- **Lines 43–44, sorting:** first dead ends (`openCount() != 1` sorts the others last, because `false` sorts before `true`), then the furthest from the entrance (`reversed()`).
- **Lines 46–52, two rounds:** round 1 only accepts units that can be rewired without breaking anything; round 2 also allows repairs. It returns the first unit that works, or `null`.

SpongeRooms.java

```
56      private static boolean tryRewire(MonumentGrid grid, Cell cell, Set<Cell> sponges, boolean
allowRepair,
57                                       RandomGenerator random) {
58          boolean[] before = new boolean[GridDirection.values().length];
59          for (GridDirection d : GridDirection.values()) {
60              before[d.ordinal()] = cell.isOpen(d);
61              if (d != GridDirection.DOWN && cell.neighbor(d) != null) {
62                  grid.setOpen(cell, d, false);
63              }
64          }
65          grid.setOpen(cell, GridDirection.DOWN, true);
66
67          List<Wall> repairs = new ArrayList<>();
68          if (grid.allReachEntry() || (allowRepair && reconnect(grid, cell, sponges, repairs,
random))) {
69              return true;
70          }
71
72          for (Wall wall : repairs) {
73              grid.setOpen(wall.cell(), wall.direction(), false);
74          }
75          for (GridDirection d : GridDirection.values()) {
76              if (cell.neighbor(d) != null) {
77                  grid.setOpen(cell, d, before[d.ordinal()]);
78              }
79          }
80          return false;
81      }
```

- **Lines 58–65:** remember the unit's walls, then close all of them except the floor, and open the floor.
- **Line 68:** does everything still reach the entrance? Then it worked. If not, and repairs are allowed, try to reconnect.
- **Lines 72–80:** it didn't work, so put everything back exactly as it was.

SpongeRooms.java

```
 87      private static boolean reconnect(MonumentGrid grid, Cell spot, Set<Cell> sponges, List<Wall>
opened,
 88                                       RandomGenerator random) {
 89          while (true) {
 90              boolean[] reached = grid.reachableFromEntry();
 91              if (allTrue(reached, grid)) {
 92                  return true;
 93              }
 94              List<Wall> options = new ArrayList<>();
 95              for (Wall wall : grid.walls()) {
 96                  Cell a = wall.cell();
 97                  Cell b = wall.other();
 98                  if (reached[a.id()] != reached[b.id()] && !a.isOpen(wall.direction())
 99                          && grid.canOpen(a, wall.direction())
100                          && a != spot && b != spot && !sponges.contains(a) && !sponges.contains(b))
{
101                      options.add(wall);
102                  }
103              }
104              if (options.isEmpty()) {
105                  return false;
106              }
107              Wall wall = options.get(random.nextInt(options.size()));
108              grid.setOpen(wall.cell(), wall.direction(), true);
109              opened.add(wall);
110          }
111      }
```

- **The repair:** closing a unit in the middle of the maze cuts part of the maze off; this reconnects it.
- **Line 89:** `while (true)` repeats until one of the `return`s is reached.
- **Lines 90–93:** everything reachable again? Done.
- **Lines 94–103:** collect closed walls with the connected part on one side and a cut-off part on the other. Walls touching a sponge spot don't count, because those must keep only their floor open.
- **Lines 104–106:** no such wall: give up.
- **Lines 107–109:** open one at random, remember it (so it can be undone), and check again.

MonumentLayout.java: putting it all together

MonumentLayout.java

```
 20      private final long seed;
 21      private final double newestChance;
 22      private final MonumentGrid grid;
 23      private final List<Room> rooms;
 24      private final Room[] roomByCell;
 25
 26      private MonumentLayout(long seed, double newestChance, MonumentGrid grid, List<Room> rooms) {
 27          this.seed = seed;
 28          this.newestChance = newestChance;
 29          this.grid = grid;
 30          this.rooms = rooms;
 31          this.roomByCell = new Room[grid.cells().size()];
 32          for (Room room : rooms) {
 33              for (Cell cell : room.cells()) {
 34                  roomByCell[cell.id()] = room;
 35              }
 36          }
 37      }
```

```
 38
 39      public static MonumentLayout generate(long seed, MazeSettings settings) {
 40          // SplittableRandom mixes the seed well; java.util.Random gives near-identical first values
for seeds 0, 1, 2...
 41          RandomGenerator random = new SplittableRandom(seed);
 42          double newestChance = settings.drawNewestChance(random);
 43
 44          // Like the datapack: each wing is wing_open or wing_closed, 50/50.
 45          MonumentGrid grid = new MonumentGrid(random.nextBoolean(), random.nextBoolean());
 46          // Like vanilla, the core goes in the back two rows of the bottom two floors, at a random
x.
 47          Cell coreAnchor = grid.at(random.nextInt(MonumentGrid.WIDTH - 1), 0, MonumentGrid.ROWS -
2);
 48
 49          GrowingTreeGenerator.carve(grid, random, newestChance);
 50          List<Room> rooms = RoomFitter.fit(grid, coreAnchor, Set.of(), random);
 51
 52          // Too few sponge rooms: rewire units into floor-only spots and fit the rooms again.
 53          Set<Cell> excluded = new HashSet<>(RoomFitter.coreCells(grid, coreAnchor));
 54          excluded.add(grid.entry());
 55          Set<Cell> sponges = new LinkedHashSet<>();
 56          while (count(rooms, RoomType.SPONGE) < settings.minSpongeRooms()) {
 57              Cell spot = SpongeRooms.makeSpongeSpot(grid, excluded, sponges, random);
 58              if (spot == null) {
 59                  break;
 60              }
 61              sponges.add(spot);
 62              rooms = RoomFitter.fit(grid, coreAnchor, sponges, random);
 63          }
 64          return new MonumentLayout(seed, newestChance, grid, rooms);
 65      }
```

- **Lines 20–24:** a finished layout: its seed, newest chance, grid, rooms, and a quick lookup from unit to room.
- **Lines 31–36:** fill that lookup. Dummy cells have no room, so they stay `null`. The constructor is `private`, so a layout can only be made through `generate`.
- **Line 41:** one random generator, created from the seed, is used for *every* random choice. Same seed and same settings means exactly the same layout, which is what makes `reset generated <seed>` and `layout current` possible. `SplittableRandom` mixes nearby seeds properly (the older `java.util.Random` gave almost identical first values for seeds 0, 1, 2…).
- **Line 42:** this monument's newest chance.
- **Line 45:** each wing open or closed, 50/50.
- **Line 47:** the core at a random x from 0 to 3 (it's 2 wide), in rows 2–3, on floors 0–1.
- **Lines 49–50:** carve the maze, then fit the rooms.
- **Lines 53–63:** while there are too few sponge rooms, find a unit to rewire and fit all rooms again with that sponge spot reserved. A `Set` is a collection without duplicates; `LinkedHashSet` also keeps the order, so the result stays identical for the same seed.

MonumentLayout.java

```
93      /**
94       * Open walls that need a doorway (horizontal) or a floor/ceiling opening (vertical) when
95       * building: every open wall between two different rooms, including the wing and penthouse
links.
96       */
97      public List<Wall> openings() {
98          List<Wall> openings = new ArrayList<>();
99          for (Wall wall : grid.walls()) {
100             if (wall.cell().isOpen(wall.direction()) && roomOf(wall.cell()) !=
roomOf(wall.other())) {
101                 openings.add(wall);
102             }
103         }
104         return openings;
105     }
```

- **The walls the builder must actually open in the world:** every open wall **between two different rooms**, including the links to the wings and the penthouse. An open wall *inside* a room needs nothing, because the room template is already one open space.

LayoutPrinter.java: the console map

LayoutPrinter.java

```
20      public static List<String> print(MonumentLayout layout) {
21          List<String> lines = new ArrayList<>();
22          lines.add("Seed " + layout.seed() + " - growing tree, " + Math.round(layout.newestChance()
* 100)
23                  + "% newest, left wing " + (layout.grid().isLeftWingOpen() ? "open" : "closed")
24                  + ", right wing " + (layout.grid().isRightWingOpen() ? "open" : "closed"));
25
26          for (int floor = MonumentGrid.FLOORS - 1; floor >= 0; floor--) {
27              lines.add("Floor " + floor + (floor == 2 ? " (top)" : floor == 0 ? " (bottom)" : ""));
28              for (int row = MonumentGrid.ROWS - 1; row >= 0; row--) {
29                  addIfNotBlank(lines, border(layout, floor, row));
30                  addIfNotBlank(lines, contents(layout, floor, row));
31              }
32              addIfNotBlank(lines, border(layout, floor, -1));
33          }
34          lines.add(summary(layout));
35          lines.add("C core, E entry, S sponge, T tall, U/L 2-wide hall (u/lantern), P/D 2x2
(pipes/dark),"
36                  + " l/i/d small (lanterns/pillars/dark), # center pillar. Gaps in the front corners
lead to the wings"
37                  + " (open wings can connect on floor 0 too).");
38          return lines;
39      }
```

- **Line 22:** the heading with the seed, newest chance and wing designs.
- **Lines 26–33:** floors from top to bottom, and rows from back to front, so the entrance ends up at the bottom of each floor. Each row gives two lines: the wall line above it and the room line.
- **Lines 34–37:** room counts and the legend.

LayoutPrinter.java

```
48      /** The line between {@code row + 1} (above) and {@code row} (below). */
49      private static String border(MonumentLayout layout, int floor, int row) {
50          StringBuilder line = new StringBuilder();
51          for (int x = 0; x < MonumentGrid.WIDTH; x++) {
```

```
52              line.append(corner(layout, x, floor, row));
53              line.append(connected(layout, at(layout, x, floor, row + 1), at(layout, x, floor, row))
? "   " : "---");
54          }
55          return line.append(corner(layout, MonumentGrid.WIDTH, floor,
row)).toString().stripTrailing();
56      }
57
58      private static String contents(MonumentLayout layout, int floor, int row) {
59          StringBuilder line = new StringBuilder();
60          for (int x = 0; x <= MonumentGrid.WIDTH; x++) {
61              Cell left = at(layout, x - 1, floor, row);
62              Cell cell = at(layout, x, floor, row);
63              line.append(connected(layout, left, cell) ? ' ' : '|');
64              if (x < MonumentGrid.WIDTH) {
65                  line.append(cell == null ? "   " : label(layout, cell));
66              }
67          }
68          return line.toString().stripTrailing();
69      }
70
71      private static String label(MonumentLayout layout, Cell cell) {
72          Room room = layout.roomOf(cell);
73          boolean up = opening(layout, cell, GridDirection.UP);
74          boolean down = opening(layout, cell, GridDirection.DOWN);
75          char marker = up && down ? '*' : up ? '^' : down ? 'v' : ' ';
76          return " " + room.type().label() + (room.centerPillar() ? '#' : marker);
77      }
78
79      /** True when there's no wall to draw: open to a different room, or part of the same room.
Blank when neither exists. */
80      private static boolean connected(MonumentLayout layout, Cell a, Cell b) {
81          if (a == null || b == null) {
82              return a == null && b == null;
83          }
84          return layout.roomOf(a) == layout.roomOf(b) || isOpenBetween(a, b);
85      }
```

- **Lines 49–56, a wall line** like `+---+ +---+`: a corner, then `---` for a closed wall or three spaces for an opening or a room's inside.
- **Lines 58–69, a room line** like `| C C | Uv U |`: `|` or a space between neighbors, then each unit's 3-character label.
- **Lines 71–77, one unit's label:** a space, the room letter, then `^` (opening up), `v` (down), `*` (both), or `#` for a small room with the center pillar.
- **Lines 80–85:** no line is drawn between two units of the same room, or where the maze opened the wall.

LayoutPrinter.java

```
117     private static String summary(MonumentLayout layout) {
118         Map<RoomType, Integer> counts = new EnumMap<>(RoomType.class);
119         for (Room room : layout.rooms()) {
120             counts.merge(room.type(), 1, Integer::sum);
121         }
122         return counts.entrySet().stream()
123                 .map(e -> e.getValue() + "x " + e.getKey().name().toLowerCase(Locale.ROOT))
124                 .collect(Collectors.joining(", ", "Rooms: ", ""));
125     }
```

- **The room count:** an `EnumMap` is a map whose keys are enum values, kept in the enum's order. `merge(type, 1, Integer::sum)` means "add 1 to this type's count, starting from 0". `Collectors.joining(", ", "Rooms:`

`", "")` glues the parts together with commas, starting with `Rooms: `.

Step 6: from layout to blocks

Step 5 produced a **layout**. Step 6 turns it into blocks in the world, in two separate parts:

Part	What it does	Touches the world?
`BuildStep`	the "vocabulary": place a template, fill a box, spawn an Elder Guardian	no
`BuildPlanner`	turns a layout into a **list** of those steps, all positions relative to the center	no
`MonumentBuilder`	carries out the list in the world, a bit per tick	yes

Why split it this way: the planner doesn't need a server, so the tests can check thousands of plans (every room covers exactly its own units, no rooms overlap, the rotations match your datapack's hallway offsets, the sponge odds come out right). Only `MonumentBuilder` needs a real server, and it's deliberately simple: it just does what the list says.

BuildStep.java: the three kinds of step

BuildStep.java

```
 7  public sealed interface BuildStep {
 8
 9      /** A block offset from the monument center. */
10      record Offset(int x, int y, int z) {
11          public Offset plus(int dx, int dy, int dz) {
12              return new Offset(x + dx, y + dy, z + dz);
13          }
14      }
15
16      /** Rotations a template can be placed with, the same as /place template's. */
17      enum Rotation {
18          NONE,
19          CLOCKWISE_90,
20          CLOCKWISE_180,
21          COUNTERCLOCKWISE_90
22      }
23
24      /** Blocks the builder places directly. */
25      enum Block {
26          WATER,
27          PRISMARINE,
28          PRISMARINE_BRICKS,
29          WET_SPONGE
30      }
31
32      /**
33       * Places a template (path below structures/monument/maze/). Like /place template, the origin
is
34       * the template's 0,0,0 corner and a rotation turns the template around that corner.
35       */
36      record Place(String template, Offset origin, Rotation rotation) implements BuildStep {
37      }
38
39      /** Fills a box, both corners included, like /fill. */
40      record Fill(Offset from, Offset to, Block block) implements BuildStep {
41      }
42
```

```
43      /** Spawns one of the three Elder Guardians at an exact position (offset from the center). */
44      record SpawnElder(double x, double y, double z) implements BuildStep {
45      }
46  }
```

- **Line 7:** a sealed interface: a build step can only be one of the three records below.
- **Lines 10–14:** a position **relative to the monument's center**, like `~` in a command. `plus` gives a new offset moved by some amount, for example `node.plus(-4, 0, -4)` means `~-4 ~ ~-4` from a node. Because positions are relative, the same plan works at any location.
- **Lines 17–22:** the four rotations, the same as `/place template`'s.
- **Lines 25–30:** the only blocks the builder places itself; everything else comes from templates. These are the plugin's own enums, not Bukkit's, which keeps the planner free of Bukkit so it can be tested without a server. The builder translates them.
- **Line 36, `Place`:** like `/place template <template> <origin> <rotation>`. The origin is the template's 0,0,0 corner.
- **Line 40, `Fill`:** like `/fill <from> <to> <block>`, with both corners included.
- **Line 44, `SpawnElder`:** spawn an Elder Guardian at an exact position. It uses decimals, because a mob can stand anywhere.

BuildPlanner.java: from layout to steps

Where the units are

BuildPlanner.java

```
32      /** Mixed into the layout seed, so the build's own random choices differ from the layout's. */
33      private static final long BUILD_SALT = 0x5EED_B11DL;
34
35      private BuildPlanner() {
36      }
37
38      public static Offset node(Cell cell) {
39          return new Offset(8 * cell.x() - 16, 4 * cell.floor(), 3 - 8 * cell.row());
40      }
```

- **Line 33:** a fixed number mixed into the seed for the build's own random choices (rotations and sponge rolls), so they don't repeat the layout's random choices but are still the same every time for the same seed. `0x` means hexadecimal; `5EED_B11D` is just a playful "seed build".
- **Lines 38–40, a unit's node:** exactly your datapack's node markers. x 0–4 gives −16, −8, 0, 8, 16; floor 0–2 gives 0, 4, 8; row 0–3 gives 3, −5, −13, −21 (the front is the south side, +z). The entrance, unit (2, 0, 0), is at `~0 ~0 ~3`, the same as the datapack's `summon marker ~ ~ ~3 {Tags:[node,entrance]}`.

A unit covers **node−4 to node+3** on x and z (8 blocks each way). Its floor level is the node's y. The node itself is a block corner, like the datapack's markers.

The plan, in your datapack's order

BuildPlanner.java

```
42      public static List<BuildStep> plan(MonumentLayout layout) {
43          RandomGenerator random = new SplittableRandom(layout.seed() ^ BUILD_SALT);
44          List<BuildStep> steps = new ArrayList<>();
45
46          // reset.mcfunction: the empty monument, entrance, wings and top room.
47          steps.add(new Place("monument_empty_nw", new Offset(-29, -1, -29), Rotation.NONE));
48          steps.add(new Place("monument_empty_ne", new Offset(0, -1, -29), Rotation.NONE));
49          steps.add(new Place("monument_empty_sw", new Offset(-29, -1, 0), Rotation.NONE));
50          steps.add(new Place("monument_empty_se", new Offset(0, -1, 0), Rotation.NONE));
51          Room entry = layout.roomOf(layout.grid().entry());
52          steps.add(new Place(RoomType.ENTRY.template(), footprintMin(entry), Rotation.NONE));
53          steps.add(new Place(wing(layout.grid().isRightWingOpen()), new Offset(9, 0, 6),
Rotation.NONE));
54          steps.add(new Place(wing(layout.grid().isLeftWingOpen()), new Offset(-24, 0, 6),
Rotation.NONE));
55          steps.add(new Place("room/penthouse", new Offset(-8, 11, -8), Rotation.NONE));
56
57          // end.mcfunction: the floors between the levels.
58          steps.add(new Fill(new Offset(-22, 3, -25), new Offset(21, 3, 6), Block.PRISMARINE));
59          steps.add(new Fill(new Offset(-15, 7, -15), new Offset(14, 7, 6), Block.PRISMARINE));
60
61          // The rooms, core first.
62          List<Room> rooms = new ArrayList<>(layout.rooms());
63          rooms.remove(entry);
64          rooms.sort(Comparator.comparing((Room r) -> r.type() != RoomType.CORE));
65          for (Room room : rooms) {
66              steps.add(placeRoom(room, random));
67          }
```

- **Line 43:** the build's random generator. `^` (XOR) mixes the salt into the seed.
- **Lines 47–55, your `maze/reset.mcfunction`, offset for offset:** the empty shell (four 29×22×29 quadrants 1 below the center), the entrance room on its unit, the wings (`wing_open` or `wing_closed` as the layout rolled; east at `~9 ~ ~6`, west at `~-24 ~ ~6`), and the top room at `~-8 ~11 ~-8`.
- **Lines 58–59, your `maze/end.mcfunction`:** the prismarine floors between floor 0 and 1 (y+3) and between floor 1 and 2 (y+7).
- **Lines 62–67:** every room's template. The entrance was already placed, so it's left out. Sorting puts the core first, like your `place/core.mcfunction`.

BuildPlanner.java

```
69          for (Room room : rooms) {
70              switch (room.type()) {
71                  case CORE -> coreCornerWalls(room, steps);
72                  case SPONGE -> sponges(node(room.anchor()), random, steps);
73                  case SMALL_PILLARS, SMALL_DARK -> {
74                      if (room.centerPillar()) {
75                          centerPillar(node(room.anchor()), steps);
76                      }
77                  }
78                  default -> {
79                  }
80              }
81          }
82
83          // Openings: floor/ceiling openings first, then the doorways (place/horizontal_connecters).
84          List<Wall> openings = layout.openings();
85          for (Wall wall : openings) {
```

```
            if (wall.direction() == GridDirection.UP) {
                Offset upper = node(wall.other());
                steps.add(new Place("roof_open", upper.plus(-4, -1, -4), Rotation.NONE));
            }
        }
        for (Wall wall : openings) {
            if (wall.direction() != GridDirection.UP) {
                steps.add(doorway(node(wall.cell()), node(wall.other())));
            }
        }

        // The three Elder Guardians: the top room and both wings.
        steps.add(new SpawnElder(0, 12, 0));
        steps.add(new SpawnElder(16, 4, 11));
        steps.add(new SpawnElder(-16, 4, 11));
        return steps;
    }
```

- **Lines 69–81:** the extras inside rooms, after all templates are placed (so no template overwrites them): the core's outer walls, the sponges, and the center pillars. `default -> {}` means "do nothing for every other room type".
- **Lines 85–90, vertical openings:** `roof_open` is an 8×8 floor slab with a 2×2 water hole. It goes in the **floor of the upper unit**, 1 below its node (`~-4 ~-1 ~-4`), exactly like your datapack's `roof_open ~-3.5 ~-1 ~-3.5`. The opening to the penthouse works the same way.
- **Lines 91–95, horizontal openings:** a water doorway, including the ones into the wings.
- **Lines 98–100:** the three Elder Guardians, at your `core.mcfunction` positions: the top room, then the east and west wings. They're the very last step, so the monument is complete before they appear.

Placing and rotating a room

BuildPlanner.java

```
    /** The lowest corner of the blocks a room's units take up. */
    static Offset footprintMin(Room room) {
        int x = Integer.MAX_VALUE;
        int y = Integer.MAX_VALUE;
        int z = Integer.MAX_VALUE;
        for (Cell cell : room.cells()) {
            Offset node = node(cell);
            x = Math.min(x, node.x() - 4);
            y = Math.min(y, node.y());
            z = Math.min(z, node.z() - 4);
        }
        return new Offset(x, y, z);
    }

    /**
     * Places a room so the template exactly covers its units. The 2-wide templates are saved
running
     * along X; along X they're placed as is or turned 180, along Z turned 90 either way, picked at
     * random so the one-sided designs (like the U-shaped hall) face both ways, as in the datapack.
     */
    static Place placeRoom(Room room, RandomGenerator random) {
        Offset min = footprintMin(room);
        if (room.axis() == null) {
            return new Place(room.type().template(), min, Rotation.NONE);
        }
        // Footprint size: 16 along the room's axis, 8 across it.
        int sizeX = room.axis() == GridDirection.EAST ? 16 : 8;
        int sizeZ = room.axis() == GridDirection.EAST ? 8 : 16;
```

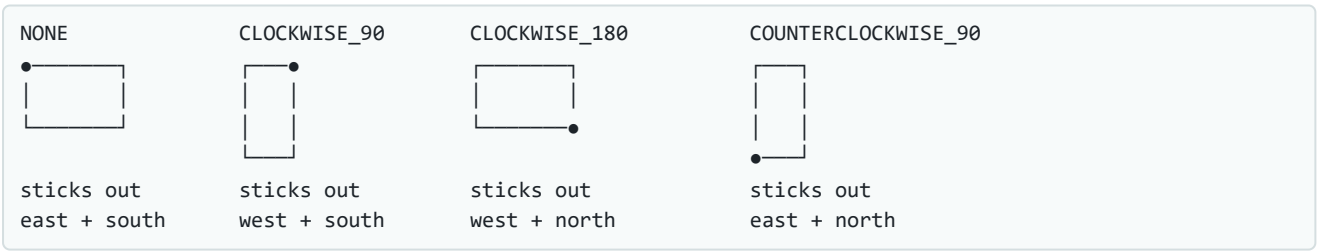
```
135          Rotation rotation = room.axis() == GridDirection.EAST
136                  ? (random.nextBoolean() ? Rotation.NONE : Rotation.CLOCKWISE_180)
137                  : (random.nextBoolean() ? Rotation.CLOCKWISE_90 : Rotation.COUNTERCLOCKWISE_90);
138          return new Place(room.type().template(), rotatedOrigin(min, sizeX, sizeZ, rotation),
rotation);
139      }
```

- **Lines 109–120, the room's corner:** start at the largest possible number, then take the smallest x and z (node −4) and the lowest floor over all the room's units.
- **Lines 129–131:** rooms without an axis are placed as-is, at the room's corner.
- **Lines 133–134:** a 2-wide room's size in the world: 16 along its axis, 8 across.
- **Lines 135–137:** your 2-wide templates are saved running east–west. Along X they're placed as-is or turned 180°; along Z, turned 90° one way or the other. A coin flip picks between the two, so one-sided designs like the U-hall face both ways, just as your datapack's four direction files did.
- **Line 138:** where to put the origin so the rotated template lands exactly on the room.

BuildPlanner.java

```
141      /**
142       * Where to put a template's origin so that, after rotating around it, the template covers the
143       * box starting at {@code min} with the given (already rotated) size.
144       */
145      static Offset rotatedOrigin(Offset min, int sizeX, int sizeZ, Rotation rotation) {
146          return switch (rotation) {
147              case NONE -> min;
148              case CLOCKWISE_90 -> min.plus(sizeX - 1, 0, 0);
149              case COUNTERCLOCKWISE_90 -> min.plus(0, 0, sizeZ - 1);
150              case CLOCKWISE_180 -> min.plus(sizeX - 1, 0, sizeZ - 1);
151          };
152      }
```

`/place template` (and Bukkit's `place`) turn a template **around its origin corner**, so after turning it sticks out in a different direction. Seen from above, with ● as the origin:



This function moves the origin so the turned template still covers exactly the room's box: for CLOCKWISE_90 (sticks out west) the origin moves to the east edge, for COUNTERCLOCKWISE_90 (sticks out north) to the south edge, for CLOCKWISE_180 both. The test `rotatedHallsMatchTheDatapackOffsets` confirms this produces exactly your `west.mcfunction` (`~11.5 ~ ~3.5 180`), `north.mcfunction` (`~-3.5 ~ ~11.5 counterclockwise_90`) and `south.mcfunction` (`~3.5 ~ ~-11.5 clockwise_90`).

The core's walls, sponges, pillars and doorways

BuildPlanner.java

```
154      /**
155       * place/core.mcfunction: a core against the left or right side opens its bottom floor into the
156       * passage between the wings. Offsets are from the core's back-left unit, like the datapack.
157       */
158      private static void coreCornerWalls(Room core, List<BuildStep> steps) {
159          Cell backLeft = core.cells().stream()
160                  .filter(c -> c.floor() == 0)
161
.min(Comparator.comparingInt(Cell::x).thenComparing(Comparator.comparingInt(Cell::row).reversed()))
162                  .orElseThrow();
163          Offset a = node(backLeft);
164          if (backLeft.x() == 0) {
165              steps.add(new Fill(a.plus(-4, 0, -4), a.plus(-4, 2, 8), Block.WATER));
166              steps.add(new Fill(a.plus(-4, 0, -4), a.plus(8, 2, -4), Block.WATER));
167          }
168          if (backLeft.x() == 3) {
169              steps.add(new Fill(a.plus(11, 0, -4), a.plus(11, 2, 8), Block.WATER));
170              steps.add(new Fill(a.plus(-1, 0, -4), a.plus(11, 2, -4), Block.WATER));
171          }
172      }
```

- **Your `place/core.mcfunction`:** a core against the left or right side opens into the passage between the wings.
- **Lines 159–162:** find the core's back-left unit on floor 0, the reference point your datapack used: the smallest x, and among those the largest row. `orElseThrow()` means "there must be one".
- **Lines 164–171:** at the far left or far right, replace the outer side wall and the back wall with water on the bottom floor (3 blocks high), the same fills as your datapack.

BuildPlanner.java

```
174      /**
175       * place/treasure.mcfunction, with vanilla's chances: each of the 36 columns is empty (1/3),
has
176       * one sponge at the top (1/2) or two (1/6). The template's own 4 sponges are cleared first.
177       */
178      private static void sponges(Offset node, RandomGenerator random, List<BuildStep> steps) {
179          steps.add(new Fill(node.plus(-3, 1, -3), node.plus(2, 2, 2), Block.WATER));
180          for (int dx = -3; dx <= 2; dx++) {
181              for (int dz = -3; dz <= 2; dz++) {
182                  if (random.nextInt(3) == 0) {
183                      continue;
184                  }
185                  int bottom = random.nextInt(4) == 0 ? 1 : 2;
186                  steps.add(new Fill(node.plus(dx, bottom, dz), node.plus(dx, 2, dz),
Block.WET_SPONGE));
187              }
188          }
189      }
190
191      /** Vanilla's 2x2 center pillar: prismarine bricks at the bottom and top, prismarine in
between. */
192      private static void centerPillar(Offset node, List<BuildStep> steps) {
193          steps.add(new Fill(node.plus(-1, 0, -1), node.plus(0, 0, 0), Block.PRISMARINE_BRICKS));
194          steps.add(new Fill(node.plus(-1, 1, -1), node.plus(0, 1, 0), Block.PRISMARINE));
195          steps.add(new Fill(node.plus(-1, 2, -1), node.plus(0, 2, 0), Block.PRISMARINE_BRICKS));
196      }
```

- **Line 179:** first clear the template's own 4 fixed sponges, by filling the room's inside (the upper two layers) with water.
- **Lines 180–188:** go through all 36 columns of the 6×6 inside. 1 in 3 stay empty; the others get sponge from the ceiling down, 2 blocks deep with a 1 in 4 chance, otherwise 1 block. That gives vanilla's ⅓ empty, ½ one sponge and ⅙ two sponges.
- **Lines 192–196:** the 2×2 pillar in the room's center (the four blocks around the node): prismarine bricks at the bottom and top, prismarine in the middle.

BuildPlanner.java

```
198      /**
199       * place/horizontal_connecters.mcfunction: a 2 wide, 2 high water doorway through both walls
200       * between two neighboring units.
201       */
202      static Fill doorway(Offset a, Offset b) {
203          if (a.x() != b.x()) {
204              int low = Math.min(a.x(), b.x());
205              int high = Math.max(a.x(), b.x());
206              return new Fill(new Offset(low + 2, a.y(), a.z() - 1), new Offset(high - 3, a.y() + 1,
a.z()), Block.WATER);
207          }
208          int low = Math.min(a.z(), b.z());
209          int high = Math.max(a.z(), b.z());
210          return new Fill(new Offset(a.x() - 1, a.y(), low + 2), new Offset(a.x(), a.y() + 1, high -
3), Block.WATER);
211      }
```

- **Your `place/horizontal_connecters.mcfunction`:** a water doorway between two neighboring units, 2 wide and 2 high, 4 blocks long through both walls.
- **Lines 203–206:** units side by side on x: x runs from `low + 2` to `high − 3` (through A's wall at node+3, B's wall at node−4, and one block of inside on each end); z covers the 2 blocks next to the node; y covers the floor and the layer above it.
- **Lines 208–210:** the same for units side by side on z. `low` and `high` don't depend on which unit comes first, so the doorway is the same from either side.

MonumentBuilder.java: doing it in the world

MonumentBuilder.java

```
26  public final class MonumentBuilder {
27
28      /** Time per tick spent building; the rest of the 50 ms tick is left for the server. */
29      private static final long BUDGET_NANOS = 15_000_000L;
30
31      private final Monuments monuments;
32
33      public MonumentBuilder(Monuments monuments) {
34          this.monuments = monuments;
35      }
36
37      /**
38       * @param spawnElder spawns an Elder Guardian at a location (with the configured stats)
39       * @return completes on the main thread once every step has run
40       */
41      public CompletableFuture<Void> build(MonumentArena arena, List<BuildStep> steps, long seed,
42                                           Consumer<Location> spawnElder) {
43          Deque<BuildStep> queue = new ArrayDeque<>(steps);
44          Random random = new Random(seed);
45          CompletableFuture<Void> done = new CompletableFuture<>();
```

```
46          BukkitTask[] task = new BukkitTask[1];
47
48          task[0] = monuments.getServer().getScheduler().runTaskTimer(monuments.plugin(), () -> {
49              long start = System.nanoTime();
50              try {
51                  do {
52                      BuildStep step = queue.poll();
53                      if (step == null) {
54                          break;
55                      }
56                      run(step, arena, random, spawnElder);
57                  } while (System.nanoTime() - start < BUDGET_NANOS);
58              } catch (RuntimeException e) {
59                  task[0].cancel();
60                  done.completeExceptionally(e);
61                  return;
62              }
63              if (queue.isEmpty()) {
64                  task[0].cancel();
65                  done.complete(null);
66              }
67          }, 1L, 1L);
68          return done;
69      }
```

- **Line 29:** at most 15 milliseconds of building per tick. A tick is 50 ms, so the server keeps 35 ms for everything else. 1 millisecond is 1,000,000 nanoseconds; the `L` marks the number as a `long`.
- **Line 41:** returns a promise that's fulfilled when the build is complete. `MonumentResetter` waits for it before counting sponges and releasing the chunks.
- **Line 42:** `Consumer<Location>` is "something you can hand a location to": the resetter's `spawnElderGuardian`, so the Elder Guardians get your configured stats.
- **Line 43:** the steps in a queue, taken from the front one by one.
- **Line 44:** a random generator for templates with several palettes (Bukkit's `place` needs this older `Random` type).
- **Lines 48–67, a timer that runs every tick:**
  - **Line 49:** note the time at the start of this tick (`System.nanoTime()` is a precise stopwatch).
  - **Lines 51–57:** `do … while` runs the body at least once, then repeats while the condition holds. Take the next step and run it, until the 15 ms are used up or the queue is empty. Because it's "at least once", even one very big step always makes progress. In practice a monument takes a handful of ticks.
  - **Lines 58–62:** if a step crashes, stop and break the promise with that error. The check then fails and is retried a minute later. A half-built monument fixes itself: it has no Elder Guardians yet (they're spawned last), so the retry finds it "cleared" and builds a complete one.
  - **Lines 63–66:** queue empty: stop the timer and fulfil the promise.

MonumentBuilder.java

```
71      private void run(BuildStep step, MonumentArena arena, Random random, Consumer<Location>
spawnElder) {
72          Location center = arena.center();
73          World world = arena.world();
74          switch (step) {
75              case Place(String template, Offset origin, BuildStep.Rotation rotation) ->
76                      monuments.structures().maze(template).place(at(center, origin), true,
rotation(rotation),
77                              Mirror.NONE, -1, 1f, random);
78              case Fill(Offset from, Offset to, BuildStep.Block block) -> {
79                  Material material = material(block);
```

```
                Location a = at(center, from);
                Location b = at(center, to);
                for (int x = Math.min(a.getBlockX(), b.getBlockX()); x <= Math.max(a.getBlockX(),
b.getBlockX()); x++) {
                    for (int y = Math.min(a.getBlockY(), b.getBlockY()); y <=
Math.max(a.getBlockY(), b.getBlockY()); y++) {
                        for (int z = Math.min(a.getBlockZ(), b.getBlockZ()); z <=
Math.max(a.getBlockZ(), b.getBlockZ()); z++) {
                            world.getBlockAt(x, y, z).setType(material, false);
                        }
                    }
                }
            }
            case SpawnElder(double x, double y, double z) ->
spawnElder.accept(center.clone().add(x, y, z));
        }
    }

    private static Location at(Location center, Offset offset) {
        return center.clone().add(offset.x(), offset.y(), offset.z());
    }

    private static StructureRotation rotation(BuildStep.Rotation rotation) {
        return switch (rotation) {
            case NONE -> StructureRotation.NONE;
            case CLOCKWISE_90 -> StructureRotation.CLOCKWISE_90;
            case CLOCKWISE_180 -> StructureRotation.CLOCKWISE_180;
            case COUNTERCLOCKWISE_90 -> StructureRotation.COUNTERCLOCKWISE_90;
        };
    }

    private static Material material(BuildStep.Block block) {
        return switch (block) {
            case WATER -> Material.WATER;
            case PRISMARINE -> Material.PRISMARINE;
            case PRISMARINE_BRICKS -> Material.PRISMARINE_BRICKS;
            case WET_SPONGE -> Material.WET_SPONGE;
        };
    }
```

- **Lines 74–91, one step:** a `switch` on the kind of step, taking its contents out directly.
  - **`Place`:** take the loaded template and place it at center + offset with the rotation: entities included, no mirroring, a random palette, every block.
  - **`Fill`:** set every block in the box. `Math.min` / `Math.max` make the loops work whichever corner is given first. `setType(material, false)` turns off block updates: faster, and placed water stays exactly where it is.
  - **`SpawnElder`:** hand the exact position to the spawn method that was passed in.
- **Lines 94–96:** turns a relative offset into a real location.
- **Lines 98–114:** translate the plugin's own rotation and block names into Bukkit's. This is the only place the two meet.

Step 7: the Elder Guardian reward and the Monument Hoe

These three files are **listeners**: classes whose methods Paper calls when something happens in the game. A few new ideas first:

What you see	Meaning
`implements Listener`	"This class contains event methods." Registered in `Monuments.enable()`.
`@EventHandler`	Marks a method Paper should call. Its parameter type decides *which* event, e.g. `BlockBreakEvent` means "a player broke a block".
`event.setCancelled(true)`	Stops the action: the block isn't broken, the hit doesn't land, and so on.
`priority = …`	When this method runs compared to other plugins: `LOWEST` → `LOW` → `NORMAL` → `HIGH` → `HIGHEST` → `MONITOR`. Later ones see what earlier ones decided and get the final say. `MONITOR` is only for watching the outcome, never for changing it.
`ignoreCancelled = true`	Skip this method if an earlier plugin (like GriefDefender) already cancelled the event.
`UUID`	A player's permanent ID. Names can change; UUIDs never do.

ElderGuardianListener.java: handing out the hoe

This replaces your datapack's `monument_elder_guardian_killed` advancement and `monument_elder_kill.mcfunction`.

ElderGuardianListener.java

```
16  public final class ElderGuardianListener implements Listener {
17
18      private final Monuments monuments;
19      private final MonumentHoe hoe;
20
21      public ElderGuardianListener(Monuments monuments, MonumentHoe hoe) {
22          this.monuments = monuments;
23          this.hoe = hoe;
24      }
25
26      @EventHandler(ignoreCancelled = true)
27      public void onElderGuardianDeath(EntityDeathEvent event) {
28          if (!(event.getEntity() instanceof ElderGuardian elder) ||
!monuments.settings().rewardEnabled()) {
29              return;
30          }
31          Player killer = elder.getKiller();
32          MonumentArena arena = monuments.arena();
33          if (killer == null || arena == null || !arena.contains(elder.getLocation())) {
34              return;
35          }
36          boolean othersAlive = arena.entities(ElderGuardian.class).stream().anyMatch(other ->
!other.equals(elder));
37          if (othersAlive) {
38              return;
39          }
40
41          // The count from when the monument was placed; counted now for a monument placed before
the
```

```
42          // plugin kept track of it.
43          int sponges = monuments.currentMonument().sponges().orElseGet(arena::countSponges);
44          if (sponges == 0) {
45              return;
46          }
47          killer.getInventory().addItem(hoe.create(sponges,
monuments.currentMonument().id())).values()
48                  .forEach(leftover -> killer.getWorld().dropItem(killer.getLocation(), leftover));
49      }
```

- **Lines 16–24:** a listener that knows the monument and the hoe helper. The same `hoe` object is shared with the other listener, so both recognize hoes the same way.
- **Line 27:** called whenever **any** mob dies anywhere, so the first lines filter hard.
- **Line 28:** stop unless it's an Elder Guardian and `reward.enabled` is on. `instanceof ElderGuardian elder` checks the type and names it `elder` in one go.
- **Line 31:** the player who killed it. This also works with arrows and tridents; it's `null` if no player was responsible.
- **Lines 32–35:** stop if no player killed it, the monument isn't available (world not loaded, paused, or switched off), or the Elder Guardian wasn't at the monument (the 40-block sphere or the monument box).
- **Lines 36–39:** are any **other** Elder Guardians still alive at the monument? Then this wasn't the last one. `anyMatch` is true if at least one entry passes the test; `!other.equals(elder)` skips the one that's dying right now.
- **Line 43:** the sponge count saved when the monument was placed, or, if unknown, counted right now. `orElseGet(arena::countSponges)` means "use the saved number, or else run `countSponges`".
- **Lines 44–46:** no sponges, no hoe.
- **Lines 47–48:** create the hoe with that durability and the monument's ID, and put it in the killer's inventory. `addItem` returns whatever didn't fit, so with a full inventory the hoe is dropped at the player's feet instead of being lost. No chat message is sent; the hoe just appears.

MonumentHoe.java: creating and recognizing the hoe

MonumentHoe.java

```
28  public final class MonumentHoe {
29
30      private final NamespacedKey tag;
31      private final NamespacedKey monumentKey;
32
33      public MonumentHoe(Plugin plugin) {
34          this.tag = new NamespacedKey(plugin, "structures");
35          this.monumentKey = new NamespacedKey(plugin, "monument");
36      }
37
38      /**
39       * @param sponges   the hoe's durability, one use per sponge
40       * @param monument  the id of the monument it belongs to
41       */
42      public ItemStack create(int sponges, long monument) {
43          ItemStack hoe = new ItemStack(Material.IRON_HOE);
44          hoe.editMeta(Damageable.class, meta -> {
45              meta.setMaxDamage(sponges);
46              meta.displayName(Component.text("Monument Hoe", NamedTextColor.DARK_AQUA)
47                      .decoration(TextDecoration.ITALIC, false));
48              meta.lore(List.of(Component.text("Mines the sponges of the Ocean Monument.",
NamedTextColor.GRAY)));
49              meta.getPersistentDataContainer().set(tag, PersistentDataType.BOOLEAN, true);
```

```
50              meta.getPersistentDataContainer().set(monumentKey, PersistentDataType.LONG, monument);
51          });
52          // Vanilla's "Can break: Sponge, Wet Sponge" tooltip. Only enforced in Adventure mode;
53          // in Survival MonumentHoeListener does the enforcing.
54          hoe.setData(DataComponentTypes.CAN_BREAK,
ItemAdventurePredicate.itemAdventurePredicate(List.of(
55                  BlockPredicate.predicate()
56                          .blocks(RegistrySet.keySet(RegistryKey.BLOCK, BlockTypeKeys.SPONGE,
BlockTypeKeys.WET_SPONGE))
57                          .build())));
58          return hoe;
59      }
```

- **Lines 30–36:** two **keys** for data stored on the item. A `NamespacedKey` is a name with the plugin's name in front, so it can't clash with other plugins: `resettingstructures:structures` (the "this is a Monument Hoe" tag) and `resettingstructures:monument` (the ID of the monument the hoe came from).
- **Line 43:** a plain iron hoe.
- **Line 44:** `editMeta` opens the item's **meta**: its name, lore, durability and stored data. `Damageable.class` asks for the meta type that has durability.
- **Line 45:** sets the hoe's **maximum durability** to the number of sponges, replacing iron's normal 250. Each use adds 1 damage, and at max damage it breaks, so it breaks exactly on the last sponge.
- **Lines 46–48:** the name "Monument Hoe" in dark aqua (custom names are italic by default; `ITALIC, false` turns that off) and the gray lore line.
- **Lines 49–50:** stores the tag and the monument ID in the item's **persistent data container**, which travels with the item through chests, Ender Chests, bundles and restarts.
- **Lines 54–57:** the vanilla `can_break` component, which makes the tooltip read **"Can break: Sponge, Wet Sponge"**. Read it from the inside out: a set of two block types, a rule "one of these blocks", the list of rules, and put it on the item. Vanilla only *enforces* this in Adventure mode; in Survival it's just the tooltip, and the listener below does the real enforcing.

MonumentHoe.java

```
61      /** Whether an item is a Monument Hoe (has the "structures" tag). */
62      public boolean is(ItemStack item) {
63          return item != null && !item.isEmpty() && item.getPersistentDataContainer().has(tag);
64      }
65
66      /** The id of the monument a hoe belongs to; empty for hoes from before ids were added. */
67      public OptionalLong monument(ItemStack item) {
68          PersistentDataContainerView data = item.getPersistentDataContainer();
69          Long id = data.get(monumentKey, PersistentDataType.LONG);
70          return id == null ? OptionalLong.empty() : OptionalLong.of(id);
71      }
72
73      public static boolean isSponge(Block block) {
74          Material type = block.getType();
75          return type == Material.SPONGE || type == Material.WET_SPONGE;
76      }
```

- **Lines 62–64, "is this a Monument Hoe?":** yes when the item exists, isn't empty, and has the `structures` tag. Recognizing it by the tag, not the name, means renaming can't fool it.
- **Lines 67–71:** the monument ID stored on a hoe; empty for hoes made before IDs existed.
- **Lines 73–76:** whether a block is a sponge, wet or dry.

MonumentHoeListener.java: the hoe rules

MonumentHoeListener.java

```
45  public final class MonumentHoeListener implements Listener {
46
47      private final Monuments monuments;
48      private final MonumentHoe hoe;
49      /** Players whose next durability loss is from a sponge they just mined. */
50      private final Set<UUID> minedSponge = new HashSet<>();
51
52      public MonumentHoeListener(Monuments monuments, MonumentHoe hoe) {
53          this.monuments = monuments;
54          this.hoe = hoe;
55      }
56
57      /** Any click with the hoe. Runs on cancelled events too: right-clicking air arrives cancelled.
*/
58      @EventHandler(priority = EventPriority.LOWEST)
59      public void onInteract(PlayerInteractEvent event) {
60          if (!monuments.isActive()) {
61              return; // no hoe rules while monuments.yml is broken or inactive, so nobody loses a
hoe over it
62          }
63          ItemStack item = event.getItem();
64          if (!hoe.is(item)) {
65              return;
66          }
67          Block clicked = event.getClickedBlock();
68          Location where = clicked != null ? clicked.getLocation() : event.getPlayer().getLocation();
69          if (!usable(item, where)) {
70              event.setCancelled(true);
71              confiscate(event.getPlayer());
72              return;
73          }
74          if (event.getAction() == Action.RIGHT_CLICK_BLOCK || event.getAction() ==
Action.RIGHT_CLICK_AIR) {
75              // No tilling, stripping or other hoe actions; clicking doors and the like still works.
76              event.setUseItemInHand(Event.Result.DENY);
77          }
78      }
```

- **Line 50:** a temporary list of players who **just** mined a sponge with the hoe, used to tell "durability loss from a sponge" (allowed) apart from every other durability loss (blocked).
- **Line 58:** `PlayerInteractEvent` fires on every left or right click, at a block or into the air, once per hand. `LOWEST` priority runs it before anything else, so the smuggling check happens first. There's no `ignoreCancelled`, because clicking into the air arrives already marked "cancelled" by Bukkit and must still count as using the hoe.
- **Lines 60–62:** no hoe rules while `monuments.yml` is broken or the monument is switched off, so nobody loses a hoe over a server mistake.
- **Lines 63–66:** only care about the Monument Hoe, in whichever hand was used.
- **Line 68:** *where* it was used: the clicked block, or the player's position when clicking into the air.
- **Lines 69–73, the smuggling rule:** if it's not usable there (outside the monument, or a hoe from an earlier monument), cancel the click and delete every Monument Hoe the player carries. A hoe brought out in an Ender Chest or bundle is deleted the first time it's clicked with.
- **Lines 74–77:** inside the monument, a right-click doesn't till dirt or do other hoe actions. `setUseItemInHand(DENY)` blocks only the *item's* action, so doors and buttons still work.

MonumentHoeListener.java

```
80      @EventHandler(priority = EventPriority.HIGH, ignoreCancelled = true)
81      public void onBreak(BlockBreakEvent event) {
82          if (!monuments.isActive()) {
83              return;
84          }
85          Player player = event.getPlayer();
86          Block block = event.getBlock();
87          ItemStack item = player.getInventory().getItemInMainHand();
88          if (hoe.is(item)) {
89              if (!usable(item, block.getLocation())) {
90                  event.setCancelled(true);
91                  confiscate(player);
92              } else if (!MonumentHoe.isSponge(block)) {
93                  event.setCancelled(true);
94                  player.sendActionBar(Component.text("The Monument Hoe only mines sponges.",
NamedTextColor.RED));
95              }
96              return;
97          }
98          MonumentArena arena = monuments.arena();
99          if (arena != null && arena.inMonument(block.getLocation()) && MonumentHoe.isSponge(block))
{
100             event.setCancelled(true);
101             player.sendActionBar(Component.text("Sponges can only be mined with the Monument Hoe.",
NamedTextColor.RED));
102         }
103     }
```

- **Line 80:** `HIGH` priority runs after GriefDefender and most protection plugins; `ignoreCancelled = true` does nothing if GriefDefender already said no. Together these keep the rule: the plugin only ever *adds* cancellations, it never allows something GriefDefender blocked.
- **Line 87:** blocks are broken with the main hand.
- **Lines 88–97, breaking with the hoe:** not usable here → cancel and delete all hoes (a backup; the click check normally catches it first). Inside, but not a sponge → cancel, with "The Monument Hoe only mines sponges." on the **action bar** (the line above the hotbar). Inside and a sponge → the break goes ahead.
- **Lines 98–102, breaking without the hoe:** a sponge inside the monument is cancelled, with "Sponges can only be mined with the Monument Hoe."

MonumentHoeListener.java

```
105      /** A sponge really got mined with the hoe: let the durability loss that follows through. */
106      @EventHandler(priority = EventPriority.MONITOR, ignoreCancelled = true)
107      public void onSpongeMined(BlockBreakEvent event) {
108          Player player = event.getPlayer();
109          if (hoe.is(player.getInventory().getItemInMainHand()) &&
MonumentHoe.isSponge(event.getBlock())) {
110              UUID id = player.getUniqueId();
111              minedSponge.add(id);
112              monuments.getServer().getScheduler().runTask(monuments.plugin(), () ->
minedSponge.remove(id));
113          }
114      }
115
116      /** The hoe only wears down from mining sponges: exactly one use per sponge. */
117      @EventHandler(priority = EventPriority.HIGH)
118      public void onDurability(PlayerItemDamageEvent event) {
119          if (!hoe.is(event.getItem())) {
120              return;
121          }
122          if (minedSponge.remove(event.getPlayer().getUniqueId())) {
123              event.setDamage(1);
124          } else {
125              event.setCancelled(true);
126          }
127      }
```

Minecraft first fires "block broken", then right after it "item took damage". The second event doesn't say *why*, so the first method leaves a note and the second reads it:

- **Lines 106–114,** `onSpongeMined`**:** `MONITOR` with `ignoreCancelled` only runs if the sponge break *actually happened*. It puts the player on the list (line 111) and schedules removing them on the next tick (line 112), so the note can never be left lying around.
- **Lines 117–127,** `onDurability`**:** fires whenever the hoe would lose durability. `minedSponge.remove(...)` removes the player from the list *and* says whether they were on it. On the list: allow exactly 1 damage. Not on the list (hitting a mob, anything else): cancel, and the hoe loses nothing.

The hoe therefore loses durability for exactly one thing, mining a sponge, one use each, so it lasts exactly as many sponges as the monument has.

MonumentHoeListener.java

```
129      /** Hitting something with the hoe outside the monument also counts as using it. */
130      @EventHandler(priority = EventPriority.LOWEST)
131      public void onHit(EntityDamageByEntityEvent event) {
132          if (!monuments.isActive()) {
133              return;
134          }
135          if (!(event.getDamager() instanceof Player player)) {
136              return;
137          }
138          ItemStack item = player.getInventory().getItemInMainHand();
139          if (hoe.is(item) && !usable(item, player.getLocation())) {
140              event.setCancelled(true);
141              confiscate(player);
142          }
143      }
144
145      // No repairing, enchanting or combining: the durability has to stay one use per sponge.
146
147      @EventHandler
```

```
148     public void onAnvil(PrepareAnvilEvent event) {
149         if (contains(event.getInventory())) {
150             event.setResult(null);
151         }
152     }
153 
154     @EventHandler
155     public void onGrindstone(PrepareGrindstoneEvent event) {
156         if (contains(event.getInventory())) {
157             event.setResult(null);
158         }
159     }
160 
161     @EventHandler
162     public void onCraft(PrepareItemCraftEvent event) {
163         if (contains(event.getInventory())) {
164             event.getInventory().setResult(null);
165         }
166     }
167 
168     @EventHandler
169     public void onEnchant(PrepareItemEnchantEvent event) {
170         if (hoe.is(event.getItem())) {
171             event.setCancelled(true);
172         }
173     }
```

- **Lines 130–143:** hitting something with the hoe outside the monument also counts as using it, so the hoes are deleted. Inside, hitting is allowed and costs no durability.
- **Lines 147–173:** the "Prepare" events fire when a result would appear in an output slot. Setting it to `null` leaves the slot empty. That blocks every way to repair the hoe (two hoes in an anvil, grindstone or crafting grid), to enchant it (Mending or Unbreaking would break the one-use-per-sponge rule), or to rename it.

MonumentHoeListener.java

```
175     /** In the current monument, and belonging to it (not kept from an earlier one). */
176     private boolean usable(ItemStack item, Location where) {
177         MonumentArena arena = monuments.arena();
178         return arena != null && arena.inMonument(where)
179                 && hoe.monument(item).orElse(0) == monuments.currentMonument().id();
180     }
181 
182     /** Deletes every Monument Hoe the player carries. */
183     private void confiscate(Player player) {
184         PlayerInventory inventory = player.getInventory();
185         ItemStack[] contents = inventory.getContents();
186         boolean removed = false;
187         for (int slot = 0; slot < contents.length; slot++) {
188             if (hoe.is(contents[slot])) {
189                 inventory.setItem(slot, null);
190                 removed = true;
191             }
192         }
193         if (hoe.is(player.getItemOnCursor())) {
194             player.setItemOnCursor(null);
195             removed = true;
196         }
197         if (removed) {
198             player.playSound(player.getLocation(), Sound.ENTITY_ITEM_BREAK, 1f, 1f);
199             player.sendMessage(Component.text("The Monument Hoe crumbles away, it only works inside
the Ocean Monument.",
200                     NamedTextColor.RED));
```

```
201         }
202     }
203
204     private boolean contains(Inventory inventory) {
205         for (ItemStack item : inventory.getContents()) {
206             if (hoe.is(item)) {
207                 return true;
208             }
209         }
210         return false;
211     }
```

- **Lines 176–180, usable** means three things: the monument exists, the spot is **inside the monument box** (the exact 58×22×58 blocks), and the hoe belongs to **this** monument. A hoe from an earlier monument has a different ID (hoes without an ID count as 0), so it isn't usable, even inside.
- **Lines 183–202, deleting every Monument Hoe the player has on them:** every inventory slot (hotbar, main inventory, armor, off-hand) and an item held on the mouse cursor. If anything was removed, play the item-break sound and send the red message. Hoes still in an Ender Chest or bundle are caught as soon as they're taken out and used.
- **Lines 204–211:** used by the anvil, grindstone and crafting checks.

Known, accepted limitations

- **While the monument is paused** (a broken `monuments.yml` at server start) **or switched off** (`active off`, and on a new server), the hoe rules are off, so only GriefDefender's flag applies and anyone could mine the sponges by hand. Both states are clearly shown in the console, `status.txt` and the status command. Switched off, this is deliberate: the location may not have been checked yet, and it could be someone's build.
- **Two hoes are possible** if two Elder Guardians die in the very same tick, because each sees the other as already dead. With one in the penthouse and one in each wing this is very unlikely.

Both were reviewed and deliberately left in.

Step 8: how the tests work

What a test is, and why this plugin has them

A **test** is a small piece of code that runs part of the plugin and checks the result: "if I give `parseTicks` the text `10s`, I must get 200 back". If the result is wrong, the test **fails** and tells you exactly what went wrong.

- **No server needed.** The whole suite (45 tests) runs in about 4.5 seconds. Testing the same things in-game would take hours.
- **Thousands of cases at once.** One test generates 3,000 monuments and checks every one of them.
- **They guard what already works.** If a later change accidentally breaks something, like the sponge guarantee or the rotation math, a test fails straight away instead of a player finding it.

Where they live

```
src/test/java/com/nltarzan/structures/
├── util/          DurationsTest, WeightedTest
├── info/          InfoFilesTest
└── monument/      MonumentArenaTest
    ├── build/     BuildPlannerTest
    ├── command/   MonumentCommandTest
    ├── config/    MonumentConfigTest
    └── layout/    MonumentLayoutTest
```

- **Mirrored packages:** the test folders have the same package names as the code they test, so a test counts as being in that package and can use package-private things like `GrowingTreeGenerator`.
- **Not in the plugin:** tests never end up in the jar (that's what `testImplementation` in `build.gradle.kts` arranges).

JUnit basics, using the smallest test

The tests use **JUnit**, the standard Java test framework.

DurationsTest.java

```
1  package com.nltarzan.structures.util;
2
3  import org.junit.jupiter.api.Test;
4
5  import static org.junit.jupiter.api.Assertions.assertEquals;
6  import static org.junit.jupiter.api.Assertions.assertThrows;
7
8  class DurationsTest {
9
10     @Test
11     void parsesScheduleUnits() {
12         assertEquals(5, Durations.parseTicks("5t"));
13         assertEquals(200, Durations.parseTicks("10s"));
14         assertEquals(24_000, Durations.parseTicks("1d"));
15         assertEquals(1_728_000, Durations.parseTicks("72d"));
16         assertEquals(36_000, Durations.parseTicks("1.5d"));
17     }
18
19     @Test
20     void plainNumberIsTicks() {
21         assertEquals(40, Durations.parseTicks("40"));
```

```
22          assertEquals(40, Durations.parseTicks(" 40 "));
23      }
24
25      @Test
26      void rejectsInvalidDurations() {
27          assertThrows(IllegalArgumentException.class, () -> Durations.parseTicks(""));
28          assertThrows(IllegalArgumentException.class, () -> Durations.parseTicks("abc"));
29          assertThrows(IllegalArgumentException.class, () -> Durations.parseTicks("5m"));
30          assertThrows(IllegalArgumentException.class, () -> Durations.parseTicks("0s"));
31          assertThrows(IllegalArgumentException.class, () -> Durations.parseTicks("-1d"));
32      }
33
34      @Test
35      void formatsRealTime() {
36          assertEquals("5s", Durations.format(100));
37          assertEquals("20m 0s", Durations.format(24_000));
38          assertEquals("24h 0m 0s", Durations.format(1_728_000));
39      }
40  }
```

- **Lines 5–6:** `import static` lets you write `assertEquals(...)` instead of `Assertions.assertEquals(...)`.
- **Line 8:** a test class, named after what it tests plus `Test`.
- **Line 10:** `@Test` marks a method as a test. JUnit finds every method marked like this and runs it.
- **Line 11:** the method name says what it checks, in plain words; when it fails, that name is the first thing you see.
- **Lines 12–16,** **`assertEquals(expected, actual)`:** if the two differ, the test stops and fails with "expected: <200> but was: <…>". If every line passes, the test passes.
- **Lines 27–31,** **`assertThrows(…)`:** passes only if the code **does** throw that error. `() -> …` is a lambda, so JUnit can run the code itself and catch the error. These lines prove that bad input like `5m` is refused instead of quietly turning into a wrong number.

The checks you'll see

Check	Passes when
`assertEquals(a, b)`	`a` and `b` are equal
`assertNotEquals(a, b)`	they're different
`assertTrue(x)` / `assertFalse(x)`	`x` is true / false
`assertNull(x)` / `assertNotNull(x)`	`x` is / isn't "nothing"
`assertSame(a, b)`	`a` and `b` are the very same object (not just equal)
`assertThrows(Error.class, () -> …)`	the code throws that kind of error

Every check can take a last argument: **a message** shown when it fails. The tests use it a lot to say *where* the failure was, for example `"chance 0.5 seed 1234: no sponge room"`.

Two kinds of tests

1. Exact examples: "this input must give exactly this output"

BuildPlannerTest.java

```
57      @Test
58      void rotatedHallsMatchTheDatapackOffsets() {
59          // west.mcfunction: at the west node W, "place template 2x1x1_lantern ~11.5 ~ ~3.5 180".
60          Offset w = new Offset(-8, 0, -5);
61          assertEquals(w.plus(11, 0, 3), BuildPlanner.rotatedOrigin(w.plus(-4, 0, -4), 16, 8,
Rotation.CLOCKWISE_180));
62          // north.mcfunction: at the north node N, "~-3.5 ~ ~11.5 counterclockwise_90".
63          Offset n = new Offset(0, 0, -13);
64          assertEquals(n.plus(-4, 0, 11), BuildPlanner.rotatedOrigin(n.plus(-4, 0, -4), 8, 16,
Rotation.COUNTERCLOCKWISE_90));
65          // south.mcfunction: at the south node S, "~3.5 ~ ~-11.5 clockwise_90" (the room runs north
of S).
66          Offset s = new Offset(0, 0, -5);
67          assertEquals(s.plus(3, 0, -12), BuildPlanner.rotatedOrigin(s.plus(-4, 0, -12), 8, 16,
Rotation.CLOCKWISE_90));
68      }
```

- `rotatedHallsMatchTheDatapackOffsets` takes your datapack's hallway commands and checks the plugin computes **the same** positions.

MonumentConfigTest.java

```
39      @Test
40      void missingOrBrokenLocationIsCaught() throws InvalidConfigurationException {
41          assertProblem("reset:\n  announce: true\n", "has no location: section");
42          // An old config.yml copied over still uses the old section name.
43          assertProblem("monument:\n  world: world\n  x: 224\n  y: 40\n  z: -320\n", "has no
location: section");
44          assertProblem("location:\n  x: 224\n  y: 40\n  z: -320\n", "has no location.world");
45          assertProblem("location:\n  world: ''\n  x: 224\n  y: 40\n  z: -320\n", "has no
location.world");
46          assertProblem("location:\n  world: world\n  y: 40\n  z: -320\n", "has no whole number for
location.x");
47          assertProblem("location:\n  world: world\n  x: 224\n  y: forty\n  z: -320\n", "has no whole
number for location.y");
48          assertProblem("location:\n  world: world\n  x: 224\n  y: 40\n  z: -320.5\n", "has no whole
number for location.z");
49          assertNullProblem("location:\n  world: world\n  x: 224\n  y: 40\n  z: -320\n");
50      }
```

- `missingOrBrokenLocationIsCaught` feeds in small broken pieces of `monuments.yml` and checks each one is caught with the right message. `\n` is a line break inside the text, so each string is a small YAML file.

2. Mass tests: "check thousands of generated monuments against the rules"

The maze is random, so one example proves little. These tests generate **thousands** of layouts and check every rule on every one.

MonumentLayoutTest.java

```
95      @ParameterizedTest
96      @ValueSource(doubles = {0.0, 0.25, 0.5, 0.75, 1.0})
97      void everyLayoutFollowsTheRules(double chance) {
98          MazeSettings settings = MazeSettings.DEFAULT.withNewestChance(chance);
99          for (long seed = 0; seed < SEEDS; seed++) {
100             MonumentLayout layout = MonumentLayout.generate(seed, settings);
101             String where = "chance " + chance + " seed " + seed;
102             MonumentGrid grid = layout.grid();
103
104             assertTrue(grid.allReachEntry(), where + ": every unit, wing and the penthouse must
reach the entry");
105             assertFalse(grid.entry().isOpen(GridDirection.UP), where + ": entry ceiling must stay
closed");
106             // No extra walls are opened: the layout is a tree, N cells joined by N - 1 open walls.
107             assertEquals(grid.cells().size() - 1, openWalls(grid), where + ": layout should have no
loops");
108             long sponges = layout.rooms().stream().filter(r -> r.type() ==
RoomType.SPONGE).count();
109             assertTrue(sponges >= 1, where + ": no sponge room");
110
111             checkEveryUnitHasOneRoom(layout, where);
112             for (Room room : layout.rooms()) {
113                 checkRoom(room, where);
114             }
115         }
116     }
```

- **Lines 95–96, a parameterized test:** `@ParameterizedTest` with `@ValueSource` runs the same test **once per value**: here 5 times, for newest chances 0, 0.25, 0.5, 0.75 and 1. Each run shows up separately, so you'd see "fails at 0.25 only".
- **Line 99:** 3,000 seeds per value, so 15,000 layouts in total.
- **Line 101, the `where` text:** every check gets it, so a failure tells you the exact chance and seed, which you could then look at with `/structure monument layout <seed> <chance>`.
- **Lines 104–113, the rules checked for every layout:** everything can be reached from the entrance; the entrance ceiling is closed; it's a perfect maze (N cells joined by exactly N − 1 open walls); there's at least one sponge room; every unit has exactly one room; and each room obeys its own rules (`checkRoom`).

The seeds are fixed numbers (`0, 1, 2, …`), not random ones, so every run tests exactly the same layouts and a failure can always be reproduced. A test that only *sometimes* fails would be nearly impossible to fix.

Statistical tests: "roughly", with a margin

WeightedTest.java

```
27      @Test
28      void picksRoughlyByWeight() {
29          List<Weighted<String>> entries = List.of(new Weighted<>("a", 30), new Weighted<>("b", 70));
30          Random random = new Random(42);
31          Map<String, Integer> counts = new HashMap<>();
32          for (int i = 0; i < 10_000; i++) {
33              counts.merge(Weighted.pick(entries, random).orElseThrow(), 1, Integer::sum);
34          }
35          int a = counts.getOrDefault("a", 0);
36          assertTrue(a > 2_700 && a < 3_300, "expected ~3000 picks of a, got " + a);
37      }
```

- **The margin:** 30% of 10,000 is 3,000. Anything between 2,700 and 3,300 passes, so normal random variation never causes a failure, but a real bug would (weights being ignored gives about 5,000).

The same idea is used for the bell curve (`newestChanceFollowsTheBellCurve`), the sponge columns (`spongeColumnsFollowVanillaChances`), the wings (`openWingsUseBothFloors`) and the maze character (`newestChanceGoesFromCorridorsToBranches`).

What each test file guards

Test file	Tests	What it protects
`DurationsTest`	4	`1d`, `10s`, `5t` and decimals are read correctly; bad times are refused; times print as `1h 4m 30s`
`WeightedTest`	4	weighted picks follow the weights; weight 0 is never picked; negative weights are refused
`MonumentConfigTest`	4	the default `monuments.yml` is valid; every way of breaking `location:` is caught; broken YAML is detected
`MonumentCommandTest`	2	`commands.txt` lists exactly the subcommands that exist, each with an explanation and correct examples
`InfoFilesTest`	2	`status.txt` and `commands.txt` have the read-only header, aligned values, every command, and no line wider than 90 characters
`MonumentArenaTest`	4	the monument's far corners count as inside; the sphere still works; a small radius still covers the monument
`MonumentLayoutTest`	18	the grid matches vanilla; open wings; all layout rules on 15,000 layouts; the bell curve; the sponge guarantee; the same seed gives the same layout; every room type appears in both directions
`BuildPlannerTest`	7	node positions match your datapack's markers; the rotation math matches your hallway files; every template covers exactly its units; no overlaps; the right build order; doorways; sponge chances

Running them

Command	What it does
`.\gradlew.bat build`	Compiles, **runs all tests**, and builds the jar. If any test fails, the build fails.
`.\gradlew.bat test`	Only runs the tests.
`.\gradlew.bat test --tests "*DurationsTest"`	Only one test class (`*` means "any package").
`.\gradlew.bat test --tests "*MonumentLayoutTest.sameSeed*"`	Only one test method.

Gradle skips tests if nothing changed since the last run; add `--rerun` to force them to run again. After a run, open `build\reports\tests\test\index.html` in a browser: it lists every test, green or red, with the time it took and the full failure message.

What a failure looks like

A real one from this project. When `InfoFilesTest` was added, it failed with:

```
InfoFilesTest > commandsFileListsEveryMonumentCommand() FAILED
    AssertionFailedError: line too long: READ-ONLY: generated by the plugin. Rewritten by the plugin
    on every start, so it always matches the installed version. ==> expected: <true> but was: <false>
```

- **Which test:** `commandsFileListsEveryMonumentCommand`.
- **Which check:** the message `line too long: …` came from the check that every line in `commands.txt` is at most 90 characters.
- **What was wrong:** it even quotes the offending line, the header. Splitting it into two lines made the test pass.

An earlier failure worked the same way: a test about the newest chance failed with "chances should spread over the range", which revealed that `java.util.Random` gives almost identical first values for nearby seeds and led to the switch to `SplittableRandom`.

What the tests can't check

Anything that needs a running server isn't covered: the listeners (the hoe rules, the Elder Guardian reward), `MonumentBuilder` actually placing blocks, chunk loading, commands typed in-game, and GriefDefender working alongside the plugin. That's why the tests concentrate on everything that *decides* what happens (the layout, the plan, the positions, the config checks), and the server-side code is kept as simple as possible. **Those in-game parts still need testing on the server.**

Adding a test yourself

Say you want to be sure that `2.5s` becomes 50 ticks. Add this to `DurationsTest.java`, next to the other tests:

```
    @Test
    void decimalSeconds() {
        assertEquals(50, Durations.parseTicks("2.5s"));
    }
```

Then run `.\gradlew.bat test --tests "*DurationsTest"`. Change the 50 to 51 and run it again to see what a failure looks like.